
STA5090Z Advanced Topics in Regression

Portfolio: Collection of Tutorials, Practials and Extentsion work

Nathan Mang'eng'e
MNGNAT015

Department of Statistical Sciences
University of Cape Town

October 1, 2026

Table of contents

1	Acknowledgements	1
I	Section A	2
2	Splines	3
2.1	The Construction of Splines	3
2.1.1	Question 1	3
2.2	Question 3	10
2.3	Smoothing Splines	13
2.3.1	ISLR Ch. 7 Exercise 1	13
2.3.2	Sketching the basis function questions	14
3	Multidimensional Splines	17
3.0.1	Wood Chapter 3 Exercise 4	17
3.0.2	P-spline penalty in matrix form	18
3.0.3	Choosing λ by minimising the RSS	19
3.0.4	The roughness penalty as $\beta' S \beta$	19
4	Generalised Additive Models (GAMs)	20
4.0.1	Question 2: Exploring <code>gam()</code> in <code>mgcv</code>	20
4.0.2	Question 4: Simulate splines	24
4.0.3	Question 7: Cairo Temperatures	27
5	Multivariate Adaptive Regression Splines (MARS)	32
6	Wavelets	39
6.0.1	Mexican Hat Wavelet	42
6.0.2	Haar father wavelet convolved with a signal	43
6.0.3	The discrete wavelet transform from first principles	44
7	Gaussian Processes	49

	7.1	Problem 1	49
8		Extension I: Wavelet-Based Jump Detection and Volatility Estimation for High-Frequency Data	55
	8.0.1	Mathematical formulation	55
	8.1	Literature Review	56
	8.1.1	Fan and Wang (2007), “ <i>Multi-scale jump and volatility analysis for high-frequency financial data</i> ”, JASA	56
	8.2	Simulation design	56
	8.3	Wavelet Multiresolution Analysis	59
	8.4	Jump detection via the finest-scale wavelet coefficient	61
	8.5	Jump size estimation and data adjustment	63
	8.6	Volatility Analysis and estimator implementation	65
	8.6.1	Numerical Simulation to Test Estimator Consistency	67
	8.7	Appendix	72
	8.7.1	Generating Correlated Sample Paths via Cholesky Decomposition	72
	8.7.2	Application to the Geometric Ornstein-Uhlenbeck Volatility Model	73
	8.8	References	73
9		Extension II: Interactive Construction of Tensor Product B-spline Bases	75
	9.0.1	Algorithmic Implementation in Shiny	76
	9.0.2	Appendix: Application Code	79
	9.0.3	References	83
II		Section B	84
10		Tutorial 1	85
	10.1	Question 1	85
	10.1.1	(a)	85
	10.1.2	(b)	86
	10.1.3	(c)	88
	10.1.4	(d)	89
	10.1.5	(e)	90
	10.1.6	(f)	92
	10.2	Question 2	93
	10.2.1	(a)	93
	10.2.2	(b)	94

	10.2.3 (c)	95
	10.2.4 (d)	95
	10.2.5 (e)	95
	10.2.6 (f)	96
	10.2.7 (g)	96
	10.3 Question 3	96
11	Tutorial 2	99
	11.1 Question 1	99
	11.1.1 (a)	99
	11.1.2 (b)	100
	11.1.3 (c)	100
	11.1.4 (d)	101
	11.2 Question 2	101
	11.2.1 (c)	101
	11.2.2 (d)	102
	11.3 Question 3	103
	11.3.1 (a)	103
	11.3.2 (b)	104
	11.3.3 (c)	105
	11.4 Question 4	106
	11.4.1 (a)	106
	11.4.2 (b)	106
	11.4.3 (c)	106
12	Extension III: Local Likelihood and Locally Linear Poisson Regression	107
	12.0.1 Local Poisson Regression	108
	12.0.2 Case Study: Call Arrivals at a Bank Call Centre	109
	12.0.3 References	113
13	Extension IV: Ill-Posed Inverse Problem in Scalar-on-Function Regression	114
	13.0.1 Regularisation Strategies	115
	13.0.2 Case Study: Tecator Benchmark	118
	13.0.3 References	126
III Supporting Documentation		127
	AI Declaration	128
	Reflective Commentary	129

Learning achievements	129
Methodological challenges	129
Ethical and reproducibility considerations	130
Future areas for development	130
Journal	132

Chapter 1

Acknowledgements

To my cat, and dear friend, Jojo. Thank you for your inspiration and guidance when it came to putting this together.

Part I

Section A

Chapter 2

Splines

2.1 The Construction of Splines

2.1.1 Question 1

Simulate data from the following model, where x is a vector of 100 random values sampled from $U(0, 1)$. Define two knots, one at 0.33 and the other at 0.66.

$$y_i \sim N(\mu_i, 0.5^2), \mu_i = 5 + \sin(3\pi(x_i - 0.6))$$

For each of the following define the basis functions / matrix and plot these functions. Then fit them to the data above and create a plot to illustrate the fitted curves, lines.

- Piecewise constant
- Piecewise linear
- Piecewise linear, and continuous at knots (broken stick)
- Piecewise cubic polynomial
- Piecewise cubic polynomial, and continuous at knots
- Piecewise cubic polynomial, and continuous up to 1st derivative at knots
- Piecewise cubic polynomial, and continuous up to 2nd derivative at knots
- Cubic spline (using truncated power basis)
- Natural spline (for the moment use the `ns()` function in packages `splines`)
- B-spline basis function (for the moment use the `bs()` function in packages `splines`, but see exercise below)

Create another plot where you add confidence bands to one of the fitted splines (natural or B-spline).

Ans:

We thus begin by simulating the data and plotting it to get a feel it.

```
library(latex2exp) #my goat; for TeX
library(splines)
```

```
## Sim data
set.seed(1409)
n <- 100
X <- runif(n)
X <- sort(X)

mu <- 5 + sin( 3*pi * (X - 0.6) )
Y <- rnorm(n, mu, 0.5)

par(mfrow = c(1,2))
plot(X,mu, pch = 20, ylab = expression(mu))
plot(mu,Y, pch = 20, xlab = expression(mu))
```

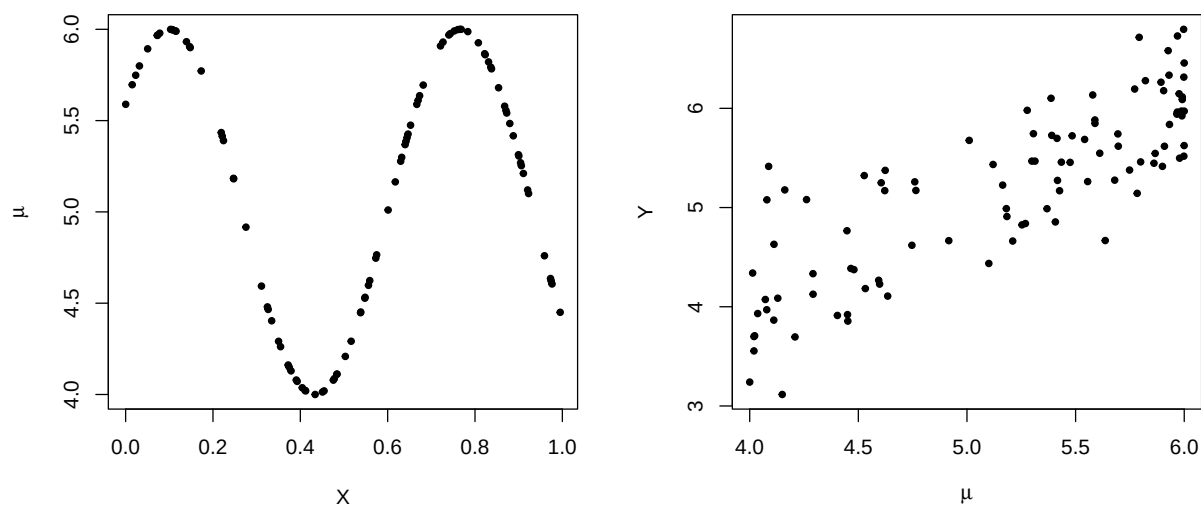
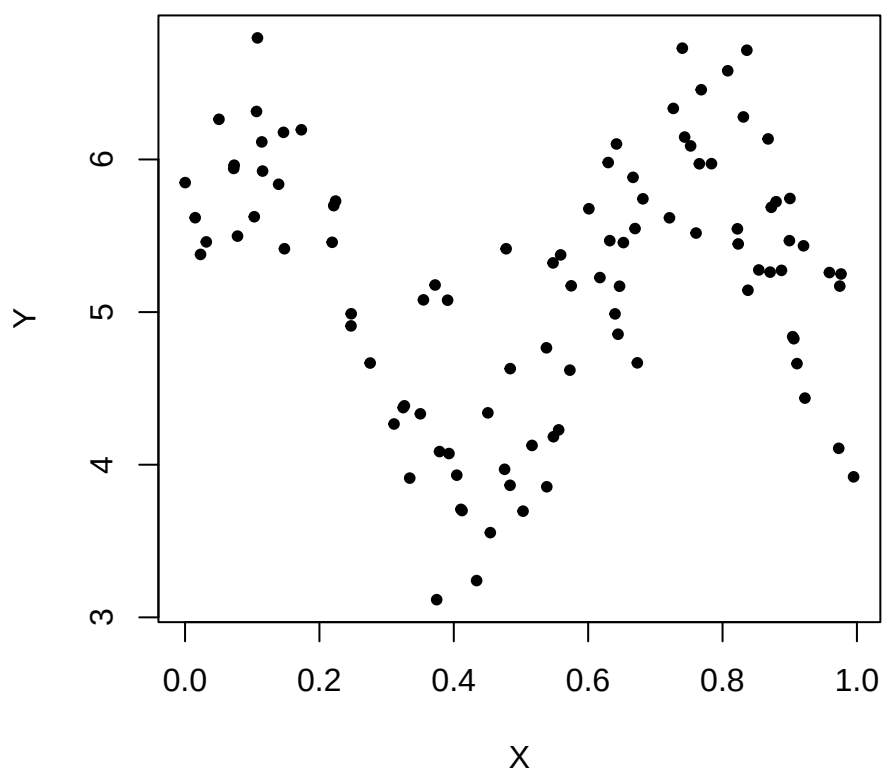


Figure 2.1: Simulated relationships: μ vs X (left) and Y vs μ (right).

```
par(mfrow = c(1,1))
```

Above we clearly see the explicit constructions for μ and Y . Given our uniform sample X and below we see the relationship between X and Y that arises. Nothing surprising to note.

```
plot(X,Y, pch = 20)
```

Figure 2.2: Simulated relationship: Y vs X

```
knots <- c(-Inf, 0.33, 0.66, Inf)

#####
# PIECEWISE CONSTANT
#####
fit1 <- lm(Y ~ cut(X,breaks = knots)) ## cut function breaks our var into
# the relevant regionn; from ISLR.
## fit <- lm(Y ~ I(X < knots[2]) + I(X >= knots[2] & X < knots[3])
# + I(X >= knots[3])) ## I thought this would give me
# piecewise linear and it was actually piecewise constant

preds1 <- predict(fit1)

cf <- coef(fit1)
jumps <- c(cf[1], cf[1] + cf[2], cf[1] + cf[3])
```

```
#####
# PIECEWISE LINEAR
#####
fit2 <- lm(Y ~ X* (I(X < knots[2]) + I(X >= knots[2] & X < knots[3])
            + I(X >= knots[3])))
preds2 <- predict(fit2)

#####
# Broken stick model
#####
fit3 <- lm(Y ~ X + I( (X - knots[2])*(X > knots[2]) )
            + I( (X - knots[3])*(X > knots[3]) ))
preds3 <- predict(fit3)

#####
# piecewise cubic polynomial
#####
fit4 <- lm(Y ~ (X + I(X^2) + I(X^3))*( I(X < knots[2])
            + I(X >= knots[2] & X < knots[3])
            + I(X >= knots[3]) ))
preds4 <- predict(fit4)

#####
# piecewise cubic polynomial (continuous at knots)
#####
fit5 <- lm(Y ~ X + I(X^2) + I(X^3) +
            I((X-knots[2])*(X > knots[2])) +
            I((X-knots[2])^2 *(X > knots[2])) +
            I((X-knots[2])^3 *(X > knots[2])) +
            I((X-knots[3])*(X > knots[3])) +
            I((X-knots[3])^2 *(X > knots[3])) +
            I((X-knots[3])^3 *(X > knots[3]))
            )
preds5 <- predict(fit5)

#####
# piecewise cubic polynomial (continuous 1st deriv)
#####
fit6 <- lm(Y ~ X + I(X^2) + I(X^3) +
            I((X-knots[2])^2 *(X > knots[2])) +
            I((X-knots[2])^3 *(X > knots[2])) +
            I((X-knots[3])^2 *(X > knots[3])) +
            I((X-knots[3])^3 *(X > knots[3]))
            )
```

```

preds6 <- predict(fit6)

#####
# piecewise cubic polynomial (continuous 2nd deriv)
# NB: This is also a cubic spline by definition
#####
fit7 <- lm(Y ~ X + I(X^2) + I(X^3) +
           I((X-knots[2])^3 *(X > knots[2])) +
           I((X-knots[3])^3 *(X > knots[3]))
           )
preds7 <- predict(fit7)

#####
# Natural spline
#####
fit8 <- lm(Y ~ ns(X, knots = c(0.33, 0.66)))
preds8 <- predict(fit8)

#####
# B-spline (bs)
#####
fit9 <- lm(Y ~ bs(X, knots = c(0.33, 0.66)))
preds9 <- predict(fit9)

par(mfrow = c(5, 2))

### plot(X, Y, pch = 20)
### lines(X, preds) ## overlaying with predict, note that since data is
# discrete it doesn't really look like a step function at the jumps
plot(X, Y, pch = 20, main = "Piecewise Constant")
lines(stepfun(knots[2:3], jumps), col = "cyan", lwd = 3)

plot(X, Y, pch = 20, main = "Piecewise Linear")
lines(X, preds2, col = "cyan", lwd = 3) ## unfortunately I do not have the
# energy to figure out how to make it look like a proper piecewise
# function right now with the step functions ;-;

plot(X, Y, pch = 20, main = "Broken Stick Model")
lines(X, preds3, col = "cyan", lwd = 3)

plot(X, Y, pch = 20, main = "Piecewise Cubic Polynomial")
lines(X, preds4, col = "cyan", lwd = 3)

plot(X, Y, pch = 20, main = "Piecewise Cubic (Continuous at Knots)")

```

```
lines(X, preds5, col = "cyan", lwd = 3)

plot(X, Y, pch = 20, main = "Piecewise Cubic (Continuous 1st Deriv)")
lines(X, preds6, col = "cyan", lwd = 3)

plot(X, Y, pch = 20,
      main = "Piecewise Cubic (Continuous 2nd Deriv / Cubic Spline)")
lines(X, preds7, col = "cyan", lwd = 3)

plot(X, Y, pch = 20, main = "Natural Spline")
lines(X, preds8, col = "cyan", lwd = 3)

plot(X, Y, pch = 20, main = "B-spline (bs)")
lines(X, preds9, col = "cyan", lwd = 3)

par(mfrow = c(1, 1)) # Reset grid back to default
```

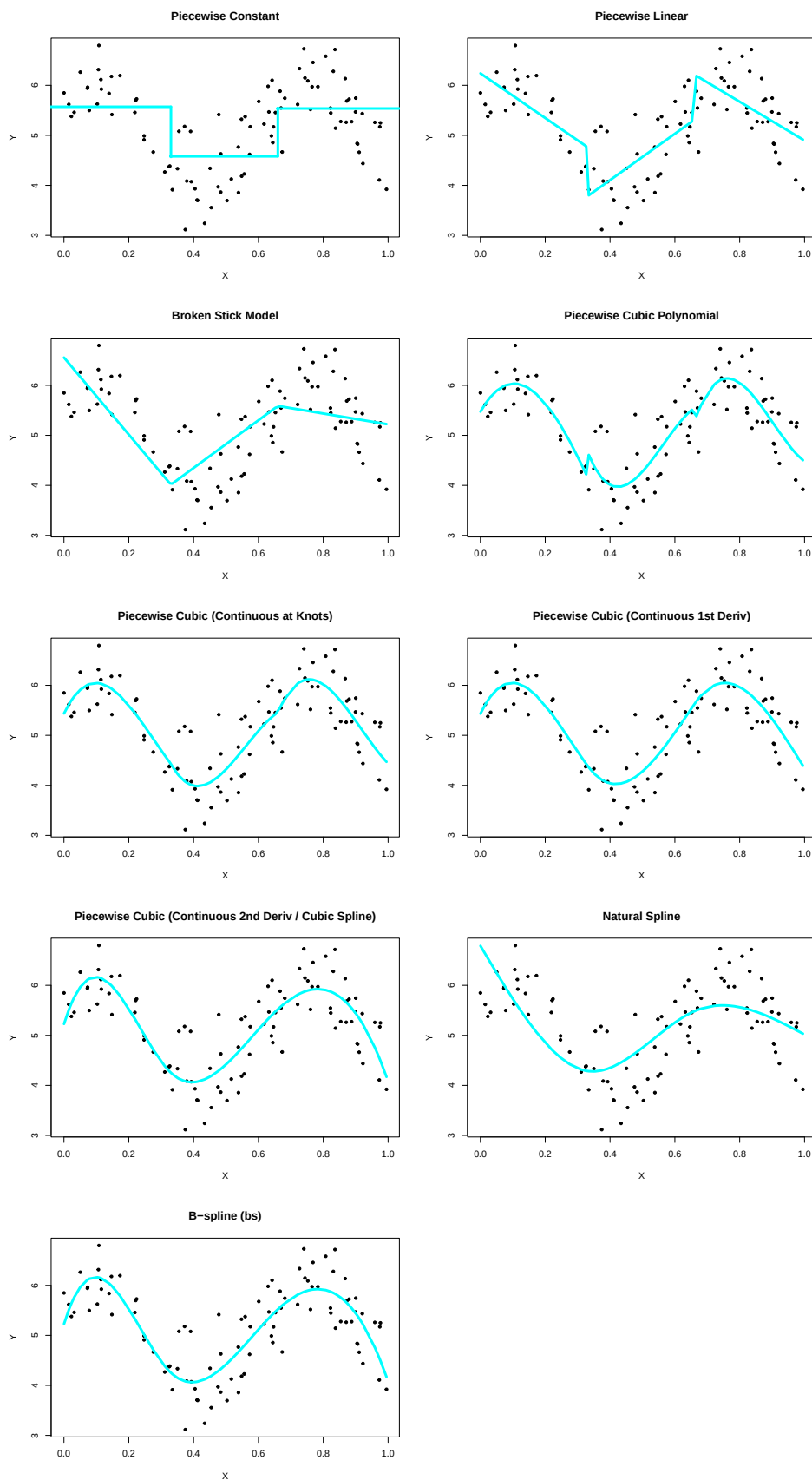


Figure 2.3: Various piecewise polynomial and spline fits, in increasing order of complexity.

We can observe the progression of the splines as we loosen the underlying restrictions, moving gradually from simple step functions all the way through to cubic polynomials. It is worth noting that due to the unnecessary complexity involved in coding the exact mathematical plots, which is not the primary focus of this exercise, we simply overlaid basic plot lines rather than the explicit functions. This explicit plotting approach was only taken for the step function as it was relatively simple to implement. As a consequence of this shortcut, the regression lines appear to show discontinuous jumps at the boundaries for a couple of the models. This occurs because there is not a sufficient amount of data points for the line drawing function to accurately pinpoint the exact shift in the coefficients at the knots, but still, it is not of much issue for this exploratory exercise. It is also worth noting that the piecewise cubic polynomial that is continuous up to the second derivative is mathematically identical to a cubic spline constructed using the truncated power basis, meaning there is no need to redundantly plot both versions. There is not much else to comment on regarding these specific visualisations, but the process serves as a very good introduction to working with splines and understanding how they are practically constructed.

2.2 Question 3

Construct the B-spline basis functions (see ESL Appendix pg. 186): Haar basis, linear, quadratic, cubic B-spline basis functions. x between -1, 1. Create basis matrix X . Plot the basis functions.

Ans:

Following ESL Appendix (p. 186), the B-spline basis comes from the Cox-de Boor recursion. Let $M = \text{order} (\text{degree} + 1)$, interior knots $\xi_1 < \dots < \xi_K$, boundary knots $\xi_0 = -1$, $\xi_{K+1} = 1$. Augment the knots by repeating the boundaries:

$$\tau_1 = \dots = \tau_M = \xi_0, \quad \tau_{M+j} = \xi_j \quad (j = 1, \dots, K), \quad \tau_{M+K+1} = \dots = \tau_{2M+K} = \xi_{K+1}.$$

Order 1 (Haar) is just an indicator of each interval:

$$B_{i,1}(x) = I(\tau_i \leq x < \tau_{i+1}),$$

and each higher order is a weighted average of two neighbours of the order below: 2

$$B_{i,m}(x) = \frac{x - \tau_i}{\tau_{i+m-1} - \tau_i} B_{i,m-1}(x) + \frac{\tau_{i+m} - x}{\tau_{i+m} - \tau_{i+1}} B_{i+1,m-1}(x),$$

with the convention $0/0 = 0$ for the boundary knots. Order $m = 2$ gives a linear basis, $m = 3$ quadratic and $m = 4$ cubic B-splines, and a spline of order M with K interior knots has $K + M$ basis functions. The code below is this recursion built up one order at a time, starting from the Haar matrix, and the plot below is the final order.

```
knots <- c(-0.33, 0.33)
```

```
bsplines_crude <- function(x = seq(-1,1, length.out = 300),
```

```
        knots,
        degree = 3,
        boundary.knots = range(x)){
M <- degree +1
K <- length(knots)
tau = c( rep(boundary.knots[1], M +1), knots, rep(boundary.knots[2], M+1))
# just gonna default with stacking the additional
# nodes at the boundaries

# Restoring the harr_basis function
harr_basis <- function(i, tau, X) {
  if (i == length(tau) - 1) {
    return(as.numeric(X >= tau[i] & X <= tau[i+1]))
  } else {
    return(as.numeric(X >= tau[i] & X < tau[i+1]))
  }
}

B <- sapply(1:(length(tau) - 1), function(i) {
  harr_basis(i = i, tau = tau, X = x)
})

B_init <- as.matrix(B)
m = 2

while(m <= M){

  B_current <- matrix(0, nrow = dim(B_init)[1], ncol = dim(B_init)[2]-1)
  # should be equiv to tau - m on the cols like this

  for(i in 1:(length(tau) - m)){

    # Safety checks for div by 0
    denom1 <- tau[i+m-1] - tau[i]
    denom2 <- tau[i+m] - tau[i+1]

    #if/else func was causing the whole vector to truncate to length 1
    # for some reason so building it manually
    if (denom1 == 0) {
      term1 <- 0
    } else {
      term1 <- ((x - tau[i]) / denom1) * B_init[,i]
    }
    if (denom2 == 0) {
```

```
    term2 <- 0
  } else {
    term2 <- ((tau[i+m] - x) / denom2) * B_init[,i+1]
  }

  B_current[,i] <- term1 + term2
}

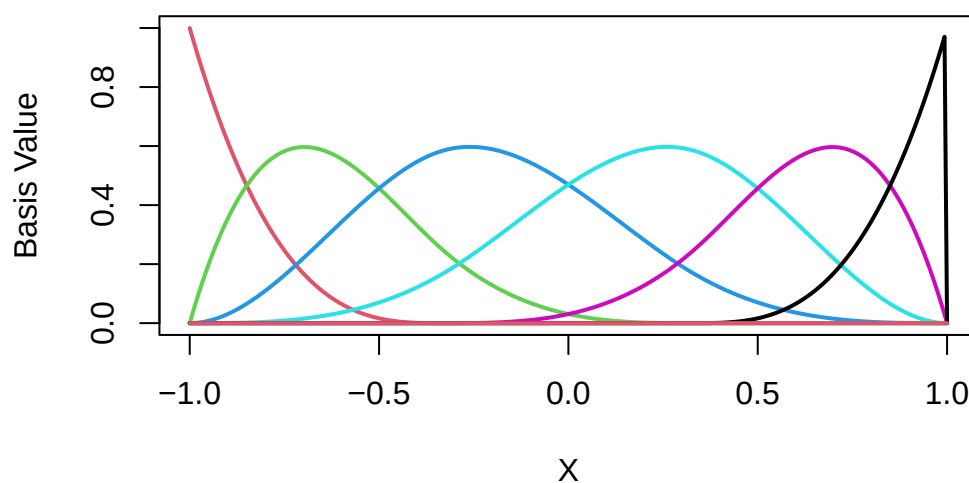
B_init <- B_current
m <- m + 1
}

return(B_current)
}

x_vals <- seq(-1, 1, length.out = 300)
my_knots <- c(-0.33, 0.33)

B_matrix <- bsplines_crude(x = x_vals, knots = my_knots)

matplot(x_vals, B_matrix, type = "l", lwd = 2, lty = 1,
        ylab = "Basis Value", xlab = "X")
```



2.3 Smoothing Splines

In practice (e.g. in R packages) smoothing splines are handled like the regression splines above (basically, a penalized regression spline). A large (enough) number of knots is chosen, and then a roughness penalty is added. The smoothing parameter is often chosen using GCV. So, there is nothing extra you need to do here.

2.3.1 ISLR Ch. 7 Exercise 1

A cubic regression spline with one knot at ξ can be obtained using a basis of the form $x, x^2, x^3, (x - \xi)_+^3$, where $(x - \xi)_+^3 = (x - \xi)^3$ if $x > \xi$ and equals 0 otherwise. We will now show that a function of the form $f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 (x - \xi)_+^3$ is indeed a cubic regression spline, regardless of the values of $\beta_0, \beta_1, \beta_2, \beta_3, \beta_4$.

- Find a cubic polynomial $f_1(x) = a_1 + b_1 x + c_1 x^2 + d_1 x^3$ such that $f(x) = f_1(x)$ for all $x \leq \xi$. Express a_1, b_1, c_1, d_1 in terms of $\beta_0, \beta_1, \beta_2, \beta_3, \beta_4$.
- Find a cubic polynomial $f_2(x) = a_2 + b_2 x + c_2 x^2 + d_2 x^3$ such that $f(x) = f_2(x)$ for all $x > \xi$. Express a_2, b_2, c_2, d_2 in terms of $\beta_0, \beta_1, \beta_2, \beta_3, \beta_4$. We have now established that $f(x)$ is a piecewise polynomial.
- Show that $f_1(\xi) = f_2(\xi)$. That is, $f(x)$ is continuous at ξ .
- Show that $f_1'(\xi) = f_2'(\xi)$. That is, $f'(x)$ is continuous at ξ .
- Show that $f_1''(\xi) = f_2''(\xi)$. That is, $f''(x)$ is continuous at ξ .

Therefore, $f(x)$ is indeed a cubic spline.

Ans:

RTP: f is a cubic spline, i.e. piecewise cubic, with f, f', f'' continuous at ξ , for any $\beta_0, \dots, \beta_4$.

- (a) For $x \leq \xi$, $(x - \xi)_+^3 = 0$,

$$\therefore f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 =: f_1(x),$$

i.e. $a_1 = \beta_0, b_1 = \beta_1, c_1 = \beta_2, d_1 = \beta_3$.

- (b) For $x > \xi$, $(x - \xi)_+^3 = (x - \xi)^3 = x^3 - 3\xi x^2 + 3\xi^2 x - \xi^3$, so

$$\begin{aligned} f(x) &= \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 (x^3 - 3\xi x^2 + 3\xi^2 x - \xi^3) \\ &= \underbrace{(\beta_0 - \beta_4 \xi^3)}_{a_2} + \underbrace{(\beta_1 + 3\beta_4 \xi^2)}_{b_2} x + \underbrace{(\beta_2 - 3\beta_4 \xi)}_{c_2} x^2 + \underbrace{(\beta_3 + \beta_4)}_{d_2} x^3 =: f_2(x). \end{aligned}$$

Both pieces are cubics, so f is a piecewise polynomial with a single knot at ξ .

Comparing the two parts directly. From (a) and (b), the remainder of the answers follow directly

$$\begin{aligned} f_2(x) - f_1(x) &= -\beta_4 \xi^3 + 3\beta_4 \xi^2 x - 3\beta_4 \xi x^2 + \beta_4 x^3 = \beta_4 (x - \xi)^3, \\ f_2'(x) - f_1'(x) &= 3\beta_4 (x - \xi)^2, \quad f_2''(x) - f_1''(x) = 6\beta_4 (x - \xi). \end{aligned}$$

(c) Continuity:

$$f_2(\xi) - f_1(\xi) = \beta_4(\xi - \xi)^3 = 0 \Rightarrow f_1(\xi) = f_2(\xi).$$

(d) First derivative:

$$f'_2(\xi) - f'_1(\xi) = 3\beta_4(\xi - \xi)^2 = 0 \Rightarrow f'_1(\xi) = f'_2(\xi).$$

(e) Second derivative:

$$f''_2(\xi) - f''_1(\xi) = 6\beta_4(\xi - \xi) = 0 \Rightarrow f''_1(\xi) = f''_2(\xi).$$

So f is a cubic spline. ■

Note that this also tells us where the continuity *stops*: $f'''_2 - f'''_1 = 6\beta_4 \neq 0$, so the third derivative jumps by $6\beta_4$ at the knot.

2.3.2 Sketching the basis function questions

Model 1 Question: Suppose we fit a curve with basis functions $b_1(X) = X, b_2(X) = (X - 1)^2 I(X \geq 1)$. (Note that $I(X \geq 1)$ equals 1 for $X \geq 1$ and 0 otherwise). We fit the linear regression model $Y = \beta_0 + \beta_1 b_1(X) + \beta_2 b_2(X) + e$. Sketch the estimated curve between $X = -2$ and $X = 2$. Note the intercepts, slopes, and other relevant information.

Ans: Note that the tutorial sheet drops the coefficient estimates for the first one, but the question in ISLR (Ch. 7 Exercise 3) uses the following estimates: $\hat{\beta}_0 = 1, \hat{\beta}_1 = 1, \hat{\beta}_2 = -2$.

Model 1: $b_1(X) = X, b_2(X) = (X - 1)^2 I(X \geq 1), Y = \beta_0 + \beta_1 b_1(X) + \beta_2 b_2(X) + \epsilon$.

Split at $X = 1$:

$$\begin{aligned} X < 1: \quad \hat{f}(X) &= 1 + X \\ X \geq 1: \quad \hat{f}(X) &= 1 + X - 2(X - 1)^2 \end{aligned}$$

At the knot,

$$\hat{f}(1^-) = 2 = \hat{f}(1^+), \quad \hat{f}'(1^-) = 1, \quad \hat{f}'(1^+) = 1 - 4(X - 1)|_{X=1} = 1,$$

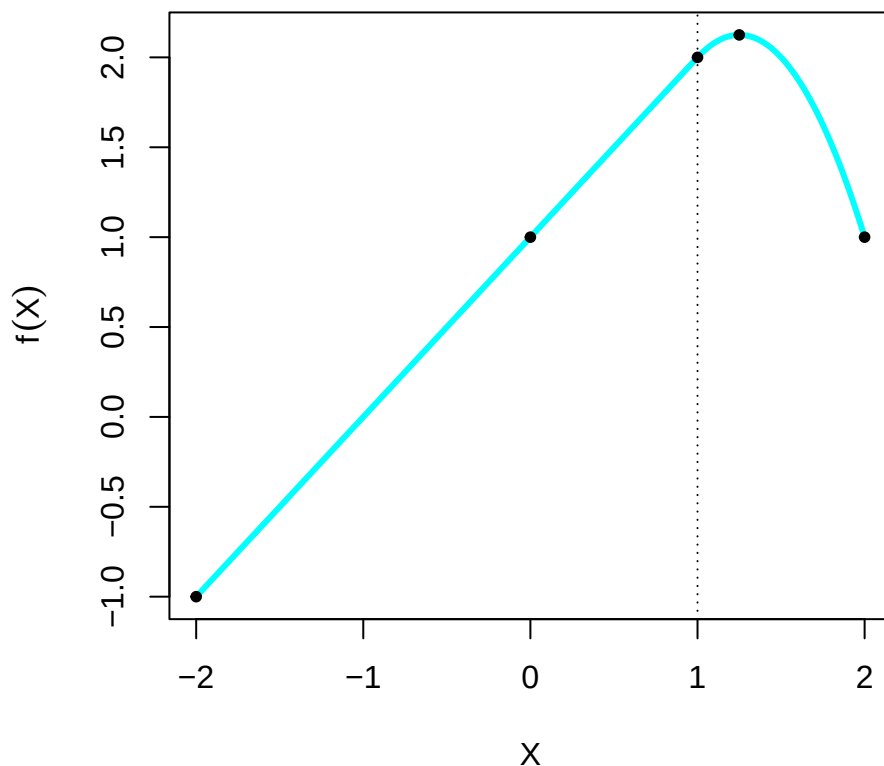
so continuous with continuous slope, while $\hat{f}''$ jumps from 0 to $2\hat{\beta}_2 = -4$. The parabola peaks where $\hat{f}' = 0$:

$$1 - 4(X - 1) = 0 \Rightarrow X = 1.25, \quad \hat{f}(1.25) = 1 + 1.25 - 2(0.25)^2 = 2.125.$$

End points: $\hat{f}(-2) = -1, \hat{f}(0) = 1, \hat{f}(2) = 1 + 2 - 2 = 1$.

```
fA <- function(x) 1 + x - 2*(x - 1)^2*(x >= 1)

xx <- seq(-2, 2, length.out = 500)
plot(xx, fA(xx), type = "l", lwd = 3, col = "cyan", xlab = "X",
      ylab = TeX("$\\hat{f}(X)$"))
abline(v = 1, lty = 3)
points(c(-2, 0, 1, 1.25, 2), fA(c(-2, 0, 1, 1.25, 2)), pch = 20)
```

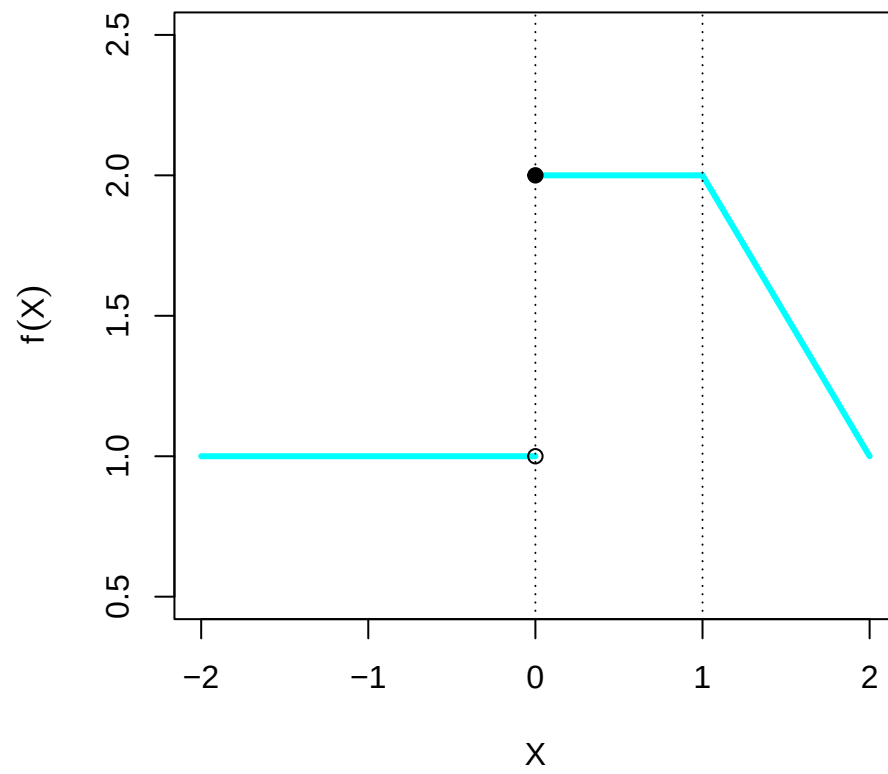


Model 2: $b_1(X) = I(0 \leq X \leq 2) - (X - 1)I(1 \leq X \leq 2)$, $b_2(X) = (X - 3)I(3 \leq X \leq 4) + I(4 < X \leq 5)$, with $\hat{\beta}_0 = 1$, $\hat{\beta}_1 = 1$, $\hat{\beta}_2 = 3$.

On $[-2, 2]$, $b_2 = 0$ since its support is $(3, 5]$, so only b_1 matters:

$$\begin{aligned} X < 0: \quad b_1 &= 0 \Rightarrow \hat{f} = 1 \\ 0 \leq X < 1: \quad b_1 &= 1 \Rightarrow \hat{f} = 2 \\ 1 \leq X \leq 2: \quad b_1 &= 1 - (X - 1) = 2 - X \Rightarrow \hat{f} = 3 - X \end{aligned}$$

```
plot(NA, xlim = c(-2, 2), ylim = c(0.5, 2.5), xlab = "X",
     ylab = TeX("$\\hat{f}(X)$"))
segments(-2, 1, 0, 1, col = "cyan", lwd = 3)
segments(0, 2, 1, 2, col = "cyan", lwd = 3)
segments(1, 2, 2, 1, col = "cyan", lwd = 3)
points(0, 1, pch = 1)      # open
points(0, 2, pch = 19)    # closed
abline(v = c(0, 1), lty = 3)
```



Chapter 3

Multidimensional Splines

3.0.1 Wood Chapter 3 Exercise 4

Show that the β minimizing

$$\|y - X\beta\|^2 + \lambda\beta'P\beta$$

is given by

$$\hat{\beta} = (X'X + \lambda P)^{-1}X'y$$

Ans:

Assume $P \succeq 0$, $\lambda > 0$.

$$S(\beta) := \|y - X\beta\|^2 + \lambda\beta'P\beta = y'y - 2\beta'X'y + \beta'X'X\beta + \lambda\beta'P\beta.$$

Note,

$$\partial_{\beta} a'\beta = a, \text{ and}$$

$$\partial_{\beta} \beta'M\beta = (M + M')\beta,$$

since $\partial_{\beta_k} \sum_{i,j} M_{ij}\beta_i\beta_j = \sum_j M_{kj}\beta_j + \sum_i M_{ik}\beta_i = [M\beta + M'\beta]_k$:

$$\frac{\partial S}{\partial \beta} = -2X'y + 2X'X\beta + \lambda(P + P')\beta = 0 \Rightarrow \left(X'X + \lambda\frac{P + P'}{2}\right)\beta = X'y.$$

$$\therefore \hat{\beta} = \left(X'X + \lambda\frac{P + P'}{2}\right)^{-1}X'y.$$

Thus, assuming P symmetric,

$$\hat{\beta} = (X'X + \lambda P)^{-1}X'y.$$

Let $A := X'X + \lambda P$. A is invertible when X has full column rank, i.e., $\|X\beta\|^2 = 0 \implies \beta = 0$. So for $\beta \neq 0$:

$$\beta' A \beta = \beta' (X'X + \lambda P) \beta = \beta' X'X \beta + \lambda \beta' P \beta = \|X\beta\|^2 + \lambda \beta' P \beta > 0.$$

Thus $A \succ 0$.

$$\frac{\partial S}{\partial \beta} = -2X'y + 2X'X\beta + \lambda(P + P')\beta = -2X'y + 2X'X\beta + 2\lambda P\beta = -2X'y + 2A\beta.$$

$$\implies \text{the Hessian is } \partial^2 S / \partial \beta \partial \beta' = 2A \succ 0.$$

Because the Hessian is positive definite, $S(\beta)$ is strictly convex and the stationary point $\hat{\beta}$ is the unique global minimum. ■

3.0.2 P-spline penalty in matrix form

For P-splines (penalize squared difference between adjacent β_i values, show that

$$P = \sum_{i=1}^{k-1} (\beta_{i+1} - \beta_i)^2$$

can be written as

$$P = \beta' \begin{bmatrix} 1 & -1 & 0 & \dots \\ -1 & 2 & -1 & \dots \\ 0 & -1 & 2 & \dots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix} \beta$$

Ans:

let D be the $(k-1) \times k$ first-difference matrix,

$$D = \begin{pmatrix} -1 & 1 & 0 & \dots & 0 \\ 0 & -1 & 1 & \dots & 0 \\ & & \ddots & \ddots & \\ 0 & \dots & 0 & -1 & 1 \end{pmatrix}, \quad D\beta = (\beta_2 - \beta_1, \dots, \beta_k - \beta_{k-1})'.$$

Then

$$\sum_{i=1}^{k-1} (\beta_{i+1} - \beta_i)^2 = \|D\beta\|^2 = \beta' D' D \beta, \quad P = D' D.$$

And clearly,

$$D' D = \begin{pmatrix} -1 & 0 & \dots & 0 \\ 1 & -1 & \dots & 0 \\ 0 & 1 & \ddots & \vdots \\ \vdots & \vdots & \ddots & -1 \\ 0 & 0 & \dots & 1 \end{pmatrix} \begin{pmatrix} -1 & 1 & 0 & \dots & 0 \\ 0 & -1 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & \dots & 0 & -1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & -1 & 0 & \dots & 0 \\ -1 & 2 & -1 & \dots & 0 \\ 0 & -1 & 2 & \dots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & \dots & \vdots & -1 & 1 \end{pmatrix} = P.$$

■

3.0.3 Choosing λ by minimising the RSS

The ‘obvious’ way to estimate smoothing parameters is by treating smoothing parameters just like the other model parameters, β , and to choose λ to minimize the residual sum of squares for the fitted model. What estimate of λ will such an approach always produce?

Ans:

$\hat{\lambda} = 0$, i.e. the unpenalised fit. $R(\beta) := \|y - X\beta\|^2$, $Q(\beta) := \beta' P \beta$ and let $\hat{\beta}_\lambda = \arg \min_\beta R + \lambda Q$. Then

$$\text{RSS}(\lambda) = R(\hat{\beta}_\lambda) \geq \min_\beta R(\beta) = R(\hat{\beta}_0) = \text{RSS}(0)$$

trivially, since $\hat{\beta}_0$ minimises R over *all* β , and $\hat{\beta}_\lambda$ is just one particular β .

3.0.4 The roughness penalty as $\beta' S \beta$

Show that for any function f , which has a basis expansion

$$f(x) = \sum_j \beta_j b_j(x),$$

it is possible to write

$$\int f''(x)^2 dx = \beta' S \beta,$$

where the coefficient matrix S can be expressed in terms of the known basis functions b_j (assuming that these possess at least two (integrable) derivatives). As usual β is a parameter vector with β_j in its j^{th} element.

Ans:

Write $b''(x) = (b_1''(x), \dots, b_k''(x))'$.

$$\implies f''(x) = \sum_j \beta_j b_j''(x) = b''(x)' \beta = \beta' b''(x),$$

$$f''(x)^2 = \left(\sum_j \beta_j b_j''(x) \right) \left(\sum_l \beta_l b_l''(x) \right) = \sum_{j,l} \beta_j \beta_l b_j''(x) b_l''(x) = \beta' b''(x) b''(x)' \beta.$$

Integrate, swapping the finite sum and the integral (Fubini):

$$\int f''(x)^2 dx = \int \sum_{j,l} \beta_j \beta_l b_j''(x) b_l''(x) dx = \sum_{j,l} \beta_j \beta_l \underbrace{\int b_j''(x) b_l''(x) dx}_{S_{jl}}.$$

Equivalently in matrix form, since β does not depend on x :

$$\int f''(x)^2 dx = \beta' \left[\int b''(x) b''(x)' dx \right] \beta = \beta' S \beta, \quad S_{jl} = \int b_j''(x) b_l''(x) dx. \blacksquare$$

Chapter 4

Generalised Additive Models (GAMs)

```
library(latex2exp)
library(mgcv)
library(splines)
library(gamair)
library(knitr)
```

The first two questions in this section share the same small simulated data set that we saw in the previous chapter for demonstrations.

```
set.seed(1409)
n <- 100
x <- runif(n)
mu <- 5 + sin(3*pi*(x - 0.6))
y <- rnorm(n, mu, 0.5)
dat <- data.frame(x = x, y = y)
xg <- seq(0, 1, length.out = 200) #for plots
```

4.0.1 Question 2: Exploring gam() in mgcv

```
fit <- gam(y ~ s(x), data = dat)
plot(fit, residuals = TRUE, pch = 20, scheme = 1,
     xlab = TeX("$x$"), ylab = TeX("$f(x)$"))
```

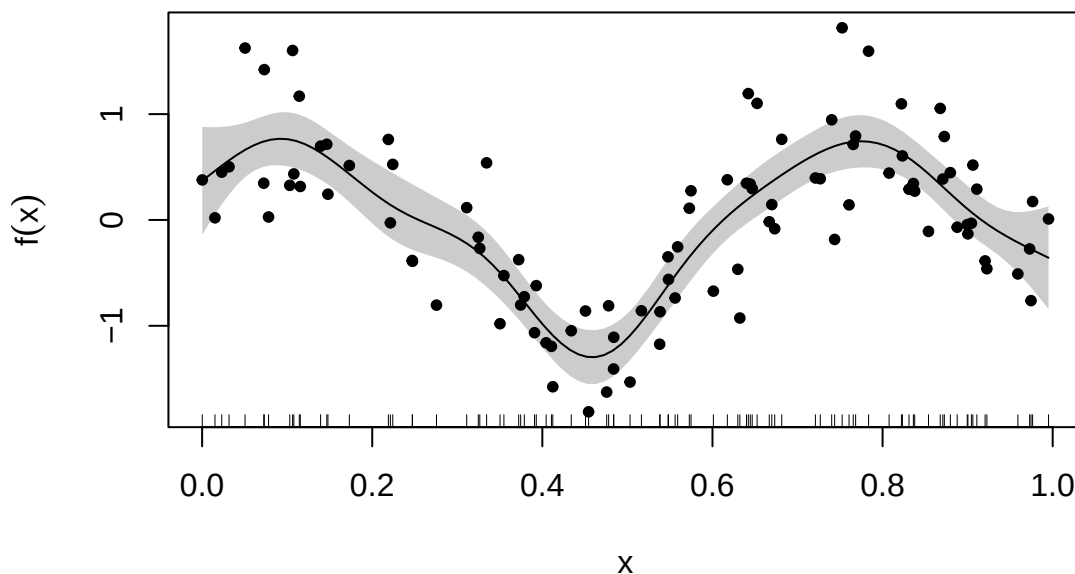


Figure 4.1: Estimated thin plate spline centered around zero, overlaid with partial residuals and a data rug plot.

Above is a simple example of how these estimated smooth functions look. Noticeably, the curve is centered at zero due to an identifiability constraint. If the smooth function was free to shift vertically, we could add a constant to it and subtract that exact same constant from the global intercept without changing the final predictions at all. Thus, this constraint cleanly separates the shape of the curve from the model's overall baseline. With this in mind, the plot above displays only this isolated smooth component alongside its shaded confidence bands, dropping the intercept entirely.

Because the curve is shifted down by removing the intercept, partial residuals are overlaid rather than raw data points. This allows us to visually verify if our centred smooth curve properly tracks the local scatter of the data. We also notice the tick marks along the horizontal axis forming a rug plot to indicate the distribution of the observed data, helping identify regions where the estimated curve relies heavily on the penalty due to a lack of local observations.

4.0.1.1 `s(.)` terms in a model

```
sm <- fit$smooth[[1]] # extract s(x), the only smooth term in this model
class(sm)
```

```
[1] "tprs.smooth" "mgcv.smooth"
```

```

sm$bs.dim      # k; basis dims before the identifiability constraint,

[1] 10

# which makes it k-1
sm$null.space.dim  # dim of the unpenalised part

[1] 1

sm$rank        # of the penalty matrix

[1] 8

ncol(predict(fit, type = "lpmatrix"))  # intercept + smooth coefficients

[1] 10

```

By default, the smooth term is a thin plate regression spline with an initial basis dimension of $k = 10$. Because a sum-to-zero identifiability constraint is applied, the actual number of parameters evaluated for this term drops to $k - 1 = 9$. Consequently, the design matrix for the entire model contains 10 columns in total: one for the parametric intercept, and nine for the constrained smooth. The null space dimension indicates the number of basis components that are entirely unpenalised. The penalty applied during optimisation takes the quadratic form $\lambda \beta' S \beta$. If a specific combination of coefficients β lies in the null space of the penalty matrix S , then $S\beta = \mathbf{0}$, meaning the penalty evaluates to exactly zero regardless of how large the smoothing parameter λ becomes. For the standard wiggleness penalty, $\int f''(x)^2 dx$, we can observe this by setting the penalty to zero:

$$\int f''(x)^2 dx = 0 \implies f''(x) = 0$$

$$\therefore f'(x) = c$$

$$f(x) = cx + d$$

Because the identifiability constraint already removes the constant shift d , only the linear component cx remains. Hence, the null space is of dimension 1 in this case. The function explicitly notes this dimension because it represents the limiting shape of the curve under maximum shrinkage: $\lambda \rightarrow \infty$ will give an infinitely large penalty to *any* curve with a non-zero second derivative. Finally, since the constrained basis has 9 dimensions and 1 lies in the null space, the penalty matrix itself has a rank of 8.

4.0.1.2 Penalty matrices with `fit = FALSE`

```

G <- gam(y ~ s(x), data = dat, fit = FALSE)

dim(G$X)

```

```
[1] 100 10
```

```
S1 <- G$smooth[[1]]$S[[1]] # penalty matrix for s(x)
dim(S1)
```

```
[1] 9 9
```

```
round(eigen(S1, symmetric = TRUE)$values, 5)
```

```
[1] 343.93246 119.14740 98.21020 41.98748 30.62711 6.78790 4.75979
[8] 0.36751 0.00000
```

```
fit_from_G <- gam(G = G)
all.equal(coef(fit), coef(fit_from_G))
```

```
[1] TRUE
```

Setting the `fit` argument to `FALSE` stops the routine before estimation and returns the complete design matrix and penalties that would be used. Examining the eigenvalues of the penalty matrix reveals one zero eigenvalue. This indicates that the linear part of the smooth lies in the null space of the penalty and remains completely unpenalised, consistent with the mathematical property that straight lines have a zero second derivative. The remaining non-linear components of the smooth are shrunk by the penalty. The returned object can be passed directly back into the function to complete the estimation, allowing model setup to be separated from execution.

4.0.1.3 Outer iteration optimization and smoothing parameters

```
fit$method
```

```
[1] "GCV"
```

```
fit$optimizer
```

```
[1] "magic"
```

```
fit$sp # estimated smoothing parameter
```

```
      s(x)
0.00352007
```

The software formulates the entire model as a single penalised regression rather than utilising backfitting. For a Gaussian response, this is penalised least squares, while for other families it employs penalised iteratively re-weighted least squares, solving a weighted penalised regression on pseudo-data at each step. The smoothing parameters, denoted as `sp`, multiply the respective penalty matrices and are chosen via a separate outer optimisation loop. By default, this outer iteration minimises the generalised cross-validation score for an unknown scale or Mallows' C_p for a known scale, with the penalised fitting nested inside.

4.0.1.4 Effective degrees of freedom represent parameter usage

The effective degrees of freedom measure how many parameters a smooth term is actively utilising in the model. This is calculated as the sum of the diagonal elements of the hat matrix that belong to the coefficients of that specific smooth. Each coefficient contributes between zero for complete shrinkage and one for a fully unpenalised parameter. As the smoothing parameter approaches zero, the effective degrees of freedom approach the full basis dimension minus one, whereas as the smoothing parameter grows large, the value collapses toward the dimension of the null space. An effective degrees of freedom value near one indicates that the estimated function is essentially a straight line.

4.0.1.5 Controlling penalty application across terms

There are several ways to alter the default shrinkage behaviour.

- (i) `s(x, k = 10, fx = TRUE)` fixes the smooth as an unpenalised regression spline ($\text{edf} = k - 1$).
- (ii) Entering the variable as an ordinary parametric term: $y \sim x$.
- (iii) `sp = 0` forces the smoothing parameter to be zero for that term.
- (iv) `select = TRUE` adds a penalty on the null space so terms can be shrunk out completely.

4.0.2 Question 4: Simulate splines

4.0.2.1 Simulating the prior distribution of splines

Choose a random set of beta coefficients from $N(0, \sigma^2)$, and plot the resulting splines (10 to 20). One could see this as the ‘prior’ distribution of splines (in a Bayesian context).

Ans:

```
k <- 10
Bg <- bs(xg, df = k, intercept = TRUE)[, 1:k]

par(mfrow = c(1, 3))
for (sigma in c(0.5, 1)) {
  betas <- matrix(rnorm(k * 15, 0, sigma), k, 15)
  matplot(xg, Bg %*% betas, type = "l", lty = 1,
          col = adjustcolor("blue", 0.5),
          xlab = "x", ylab = TeX("$f(x)$"), ylim = c(-3, 3),
          main = TeX(paste("$\\sigma = $ ", sigma)))
}
plot(xg, sqrt(rowSums(Bg^2)), type = "l", lwd = 2, xlab = "x",
     ylab = TeX("$\\sigma_{f(x)} / \\sigma$"),
     main = TeX("SD of $f(x)$ for iid $\\beta$"))
```

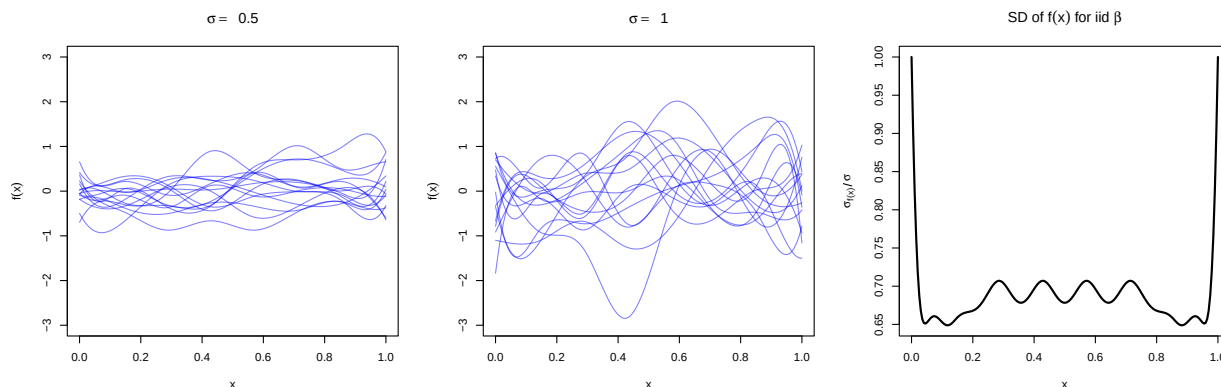


Figure 4.2: Simulating 15 random curves from an independent Gaussian prior on the B-spline coefficients. The first two panels show unpenalised draws with different variances, while the third panel shows the standardised theoretical standard deviation across the domain, highlighting the non-stationarity of the B-spline basis at the boundaries.

```
par(mfrow = c(1, 1))
```

Rather than estimating coefficients from data, the inherent behaviour of the basis can be explored by simulating functions from a prior distribution. Drawing independent coefficients from a zero-mean Gaussian distribution and multiplying them by the basis matrix generates random functions from the unpenalised spline. The curves appear smooth because the underlying B-spline basis functions are piecewise cubic polynomials with continuous derivatives, but because the coefficients are independent, the functions remain as wiggly as the basis dimension permits. The theoretical variance of these curves evaluates to $\text{Var}(f(x)) = \sigma^2 \sum B_j(x)^2$. Dividing this by σ standardises the metric, allowing the structural uncertainty of the basis to be plotted independent of the chosen variance. Because B-splines always sum to one at any point across the active interval, the standard deviation balloons rapidly to one at the endpoints where only a single basis function is active. In the interior, multiple basis functions overlap to sum to one, meaning the sum of their squares is strictly less than one, creating a dip in variance. Consequently, this independent prior is not stationary, which is entirely an artefact of the basis construction. A penalised model corrects this by replacing the independent prior with a structured multivariate normal prior that enforces smoothness

4.0.2.2 Simulating draws using the fitted covariance matrix

Obtain the covariance matrix for the fitted spline (`mgcv`) coefficients, and simulate 10 to 20 splines.

Ans:

Building directly on the previous demonstration, the penalised fit resolves the jaggedness of the independent prior by imposing a structured smoothing prior, $\beta \sim N(0, (\lambda S)^{-1} \sigma^2)$. Because the penalty matrix S is singular, this prior is improper, effectively assigning a flat unpenalised prior to the null space. As highlighted in the course notes, adopting this Bayesian perspective is mathematically necessary for reliable inference. The penalty naturally induces bias in the

penalised terms such that $E(\hat{\beta}) \neq \beta$, meaning that confidence intervals based on the standard frequentist variance, $V_{\beta} = (X'X + \lambda S)^{-1} X'X (X'X + \lambda S)^{-1} \sigma^2$, exhibit poor coverage. The solution is to construct Bayesian confidence intervals. As per the notes, combining the structured prior with the data likelihood yields the Bayesian posterior distribution for the coefficients: $\beta \mid y \sim N(\hat{\beta}, (X'WX + \sum_j \lambda_j S_j)^{-1} \sigma^2)$. The variance-covariance function extracts this posterior covariance matrix directly from the fitted model.

```
fit4 <- gam(y ~ s(x), data = dat, method = "REML")
Xp    <- predict(fit4, newdata = data.frame(x = xg), type = "lpmatrix")
Vb    <- vcov(fit4)

# Generate 15 posterior draws
post15 <- Xp %*% t(rmvn(15, coef(fit4), Vb))

# Calculate analytical standard errors for the shaded band
pr <- predict(fit4, newdata = data.frame(x = xg), se.fit = TRUE)
upr <- pr$fit + qnorm(0.975) * pr$se.fit
lwr <- pr$fit - qnorm(0.975) * pr$se.fit

plot(dat, pch = 20, col = "grey50", xlab = "x", ylab = "y")
# Add the shaded analytical confidence band
polygon(c(xg, rev(xg)),
        c(upr, rev(lwr)),
        col = adjustcolor("blue", 0.2),
        border = NA)

matlines(xg, post15, lty = 1,
         col = adjustcolor("blue", 0.35))

lines(xg, drop(Xp %*% coef(fit4)), col = "red", lwd = 3)
lines(xg, 5 + sin(3*pi*(xg - 0.6)), lty = 2, lwd = 2)

legend("topleft",
      legend = c("posterior draws", "fitted", "95% CI", "true mean"),
      lty = c(1, 1, NA, 2), pch = c(NA, NA, 15, NA),
      col = c("blue", "red", adjustcolor("blue", 0.2), "black"),
      lwd = c(1, 3, NA, 2), bty = "n")
```

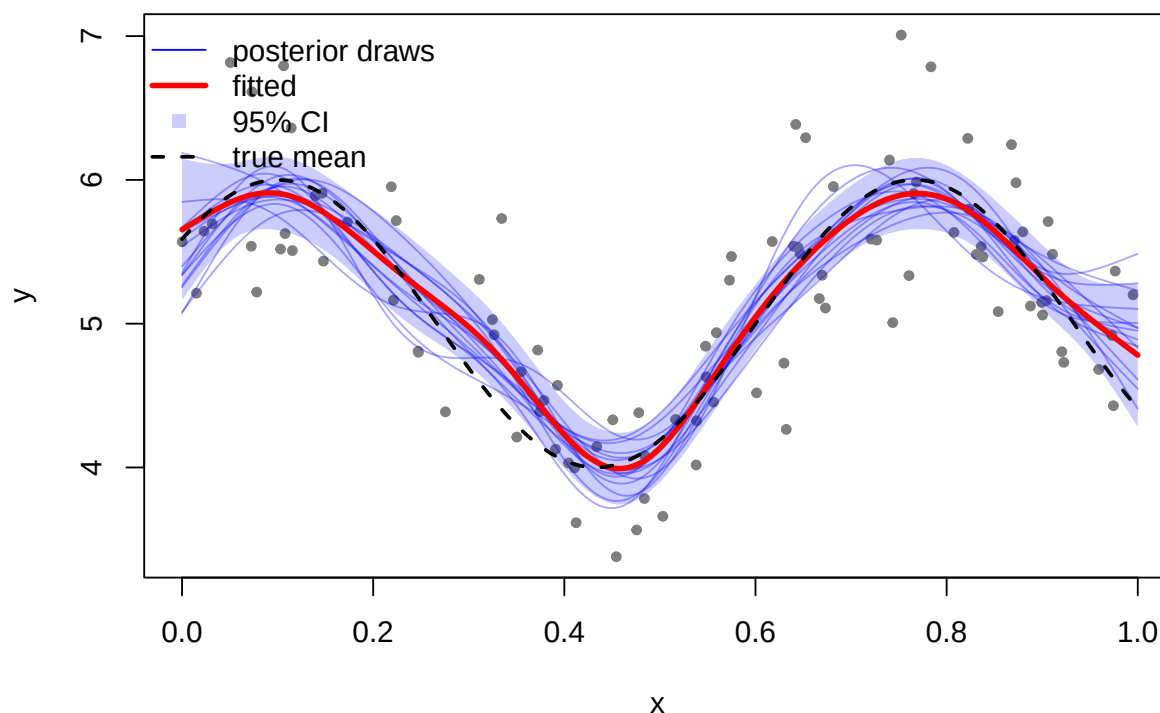


Figure 4.3: Posterior draws from the fitted GAM overlaid on the analytical 95% confidence interval.

These are draws of the curve representing uncertainty about f , rather than new observations which would need σ^2 added on top. One caveat to this approach is that the standard posterior covariance matrix conditions on the estimated smoothing parameters $\hat{\lambda}$ as if they were fixed and known constants. Consequently, the uncertainty involved in estimating $\hat{\lambda}$ itself is ignored, though invoking the variance-covariance function with the unconditional argument set to true computes an adjusted version that incorporates this extra variability.

4.0.3 Question 7: Cairo Temperatures

Fit an appropriate GAM to the cairo data set (in gamair). The data set contains daily temperature over nearly a decade. Your model should include a long-term trend and seasonal changes. You don't have to worry about autocorrelation here. Illustrate with confidence intervals.

Ans:

The cairo dataset from the gamair package contains average daily temperatures in Fahrenheit

recorded over roughly a decade, starting from January 1995. With 3780 consecutive observations, the dataset is substantially larger than the simulated examples explored previously. The primary response variable is the daily average temperature, while the predictors of interest are the continuous time elapsed in days and the recurring day of the year.

```
data(cairo)

plot(cairo$time, cairo$temp, type = "l", col = adjustcolor("blue", 0.8),
     xlab = "Days elapsed (Time)", ylab = "Average Temperature (F)")
```

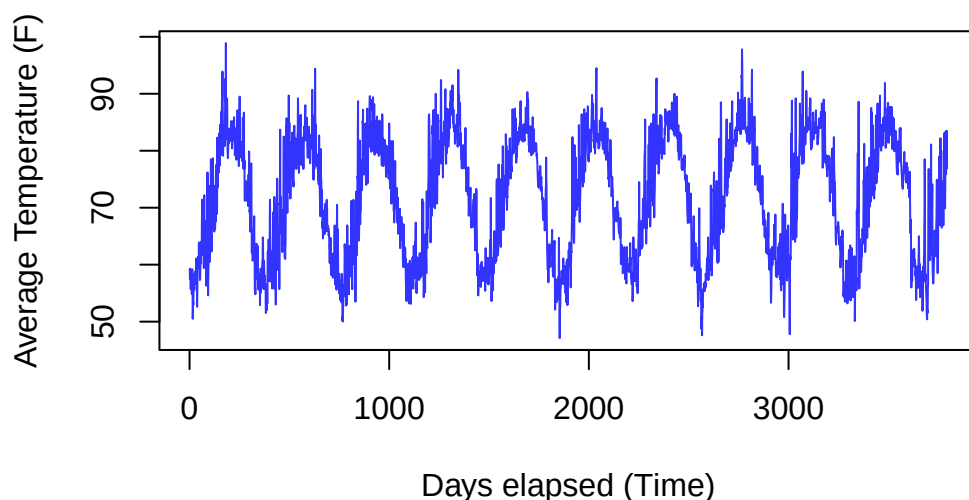


Figure 4.4: Average daily temperatures in Cairo from January 1995 onwards.

Visual inspection reveals a regular periodic oscillation. It does appear to be an annual cycle, eyeballing that 3 years $\approx$ 1200 days, in which 3 full cycles occur. Intuitively one would expect peaks during the summer months and drops sharply into the winter troughs. There is no immediate visual evidence from the raw plot to suggest any long-term drift in the baseline temperature, nor any obvious long-term change in the magnitude of the cyclical peaks and troughs across the decade. This stability suggests that while the seasonal variation is exceptionally strong, any overarching long-term climate trend will likely be subtle or entirely non-existent.

To capture the full climate profile, the model incorporates two distinct smooth terms. A standard thin plate regression spline is applied to the continuous time variable to capture any overarching long-term climate trend. Simultaneously, a cyclic cubic regression spline is applied to the day of the year to properly model the recurring seasonal variation.

Examining the plotted smooth components below confirms the initial visual intuition. The

rug plot along the horizontal axis merges into a solid line, indicating a highly dense and consistent data distribution, which we already know since the data is daily. Consequently, the shaded Bayesian confidence intervals are exceptionally narrow for both terms. The long-term trend across elapsed time evaluates to a nearly flat line hovering around zero, exhibiting only minor perturbations. This confirms no substantial overall climate effect or temperature shift occurred across this specific decade. In contrast, the seasonal term displays a very clear, concave-down relationship peaking predictably around day 210: 29 July, middle of summer in the Northern hemisphere. Also as expected we see the coldest days are in the last month of the year heading into the beginning months of the following year, i.e. winter.

```
fit_cairo <- gam(temp ~ s(time) + s(day.of.year, bs = "cc", k = 12),
                 data = cairo, method = "REML")

par(mfrow = c(2, 1), mar = c(4, 4, 1, 1))

plot(fit_cairo, select = 1, scheme = 1, col = "blue",
     xlab = "Days elapsed (Time)", ylab = TeX("$f(\\text{Time})$"))
plot(fit_cairo, select = 2, scheme = 1, col = "red",
     xlab = "Day of the Year", ylab = TeX("$f(\\text{Day})$"))
```

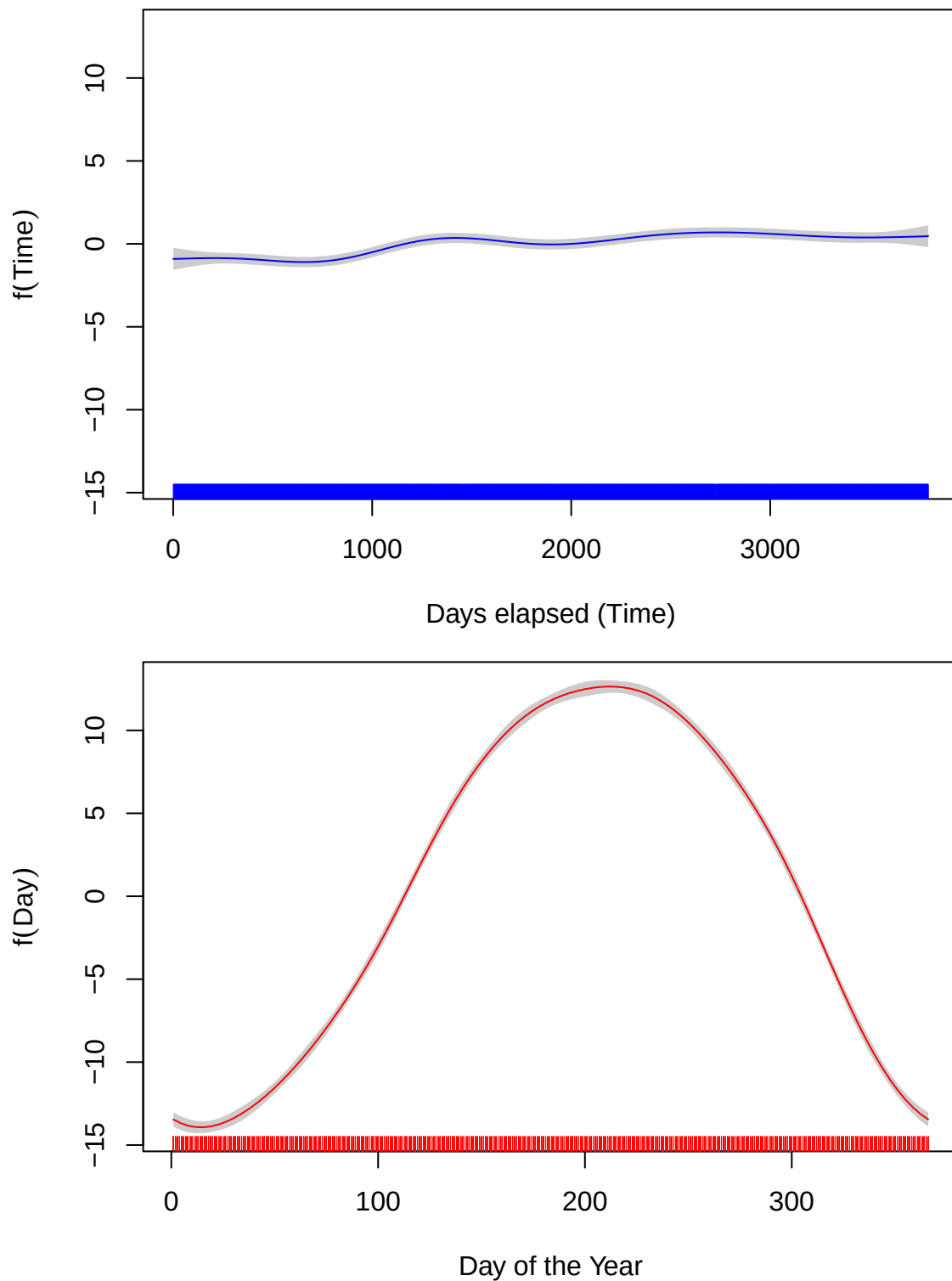


Figure 4.5: Estimated long-term trend and cyclic seasonal effect for Cairo daily temperatures, illustrated with 95% shaded Bayesian confidence intervals.

```
# Reset parameters back to default  
par(mfrow = c(1, 1), mar = c(5, 4, 4, 2) + 0.1)
```

Chapter 5

Multivariate Adaptive Regression Splines (MARS)

```
library(earth)
library(mgcv)
library(latex2exp)
library(knitr)
```

The Spambase dataset contains binary indicators of whether an email is considered unsolicited commercial spam, alongside continuous predictors measuring the frequencies of specific words and characters, as well as the lengths of consecutive capital letter sequences.

```
spam_data <- read.csv("data/MARS/spambase.data", header = FALSE)

# Response distribution
kable(table("Spam Indicator" = spam_data$V58))
```

Table 5.1: Distribution of the binary response (V58)

Spam.Indicator	Freq
0	2788
1	1813

```
#|tbl-cap: "summary statistics for a selection of predictors."
```

```
sel_vars <- c(1:5, 55:57)
stats <- data.frame(
  Prop_Zero = colMeans(spam_data[, sel_vars] == 0),
```

```

Mean      = colMeans(spam_data[, sel_vars]),
Max       = apply(spam_data[, sel_vars], 2, max)
)
rownames(stats) <- paste0("V", sel_vars)
kable(stats, digits = 3, col.names = c("Proportion of Zeros", "Mean",
                                       "Maximum"))

```

	Proportion of Zeros	Mean	Maximum
V1	0.771	0.105	4.54
V2	0.805	0.213	14.28
V3	0.590	0.281	5.10
V4	0.990	0.065	42.81
V5	0.620	0.312	10.00
V55	0.000	5.192	1102.50
V56	0.000	52.173	9989.00
V57	0.000	283.289	15841.00

Most predictors in this dataset count the frequencies of specific words or characters. Because the majority of these tracked terms do not appear in a typical email, these features exhibit extremely high proportions of exact zero values, as demonstrated by the first five predictors in the table, alongside a long right tail. Fortunately, the binary response variable maintains a healthy balance between non-spam (2788) and spam (1813) classifications, avoiding the predictive complications associated with severe class imbalance. After partitioning the data into a 70/30 train/testing split, an initial MARS model is fitted using a degree of one to establish a simple additive baseline without interaction terms. This configuration automatically applies a penalty of two per knot, which the package documentation suggests is the optimal default setting to avoid subverting the standard selection algorithm.

```

spam_data <- read.csv("data/MARS/spambase.data", header = FALSE)

set.seed(1409)
idx <- sample(nrow(spam_data), size = round(0.7 * nrow(spam_data)))
train <- spam_data[idx, ]
test <- spam_data[-idx, ]

fit <- earth(V58 ~ ., data = train, degree = 1, penalty = 2)

```

5.0.0.1 Evaluating term selection and penalisation criteria

The model constructs its functional basis through a two-stage algorithm: a greedy forward pass that sequentially adds terms to maximally reduce the residual sum of squares, followed by a backward pass that prunes terms based on the generalised cross-validation score. The generalised cross-validation heavily penalises the residual sum of squares using the effective number of parameters, which is directly scaled by the specified per-knot penalty.

```

par(mfrow = c(1, 2))
plot(fit$rss.per.subset, type = "l", lwd = 2,
     xlab = "Number of terms", ylab = "RSS",
     main = "RSS of best model of each size")
plot(fit$gcv.per.subset, type = "l", lwd = 2,
     xlab = "Number of terms", ylab = "GCV",
     main = "GCV of best model of each size")
abline(v = which.min(fit$gcv.per.subset), lty = 2, col = "cyan")

```

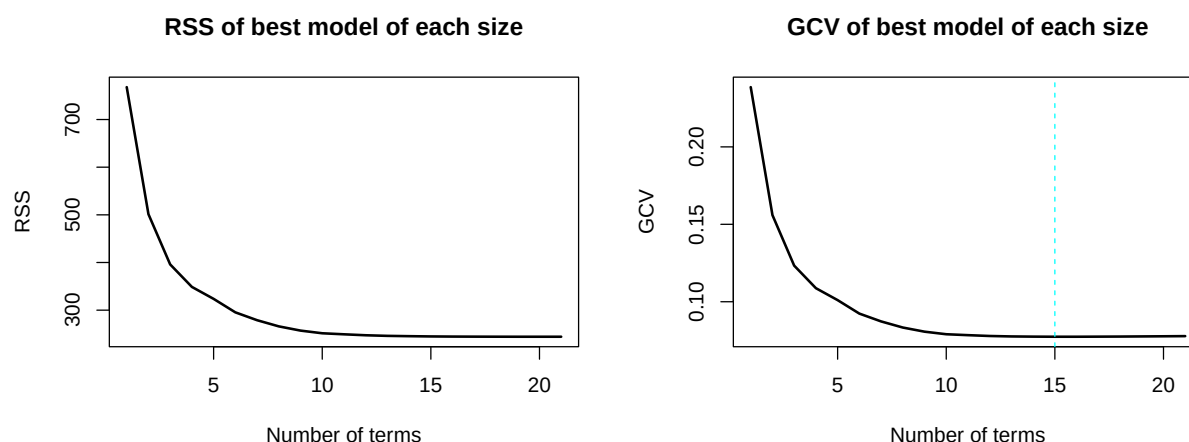


Figure 5.1: The term selection process. The residual sum of squares decreases monotonically during the forward pass, while the generalised cross-validation curve reaches a minimum to identify the optimal model size during the backward pruning pass.

```

par(mfrow = c(1, 1))

```

Plotting these criteria against the number of terms illustrates this regularisation dynamic. The residual sum of squares decreases monotonically as terms are added during the unconstrained forward pass. Conversely, the generalised cross-validation curve eventually penalises the increasing complexity and turns upward, creating a distinct minimum that dictates the optimal final model size selected by the backward pass.

5.0.0.2 The mechanical impact of structural arguments

The algorithm builds the model through a two-stage process, and the structural arguments govern the constraints placed on each phase. The initial forward pass is dictated by parameters that control the expansion of the functional basis. The degree argument restricts the maximum order of interactions, with a degree of one enforcing a strictly additive model, while higher values allow for complex tensor products of hinge functions. To prevent uncontrolled expansion, the forward pass is bounded by the `nk` argument, which sets a hard upper limit on the total number of terms generated before pruning begins. Furthermore, the `thresh` parameter

ensures computational efficiency by prematurely terminating the forward pass if the marginal improvement to the coefficient of determination falls below a specified tolerance, meaning the algorithm will naturally stop adding terms even if the maximum term limit has not been reached.

Once the forward pass concludes, the backward pass systematically prunes the model to optimise the bias-variance tradeoff. This phase is heavily influenced by the **penalty** parameter, which determines the cost per knot incorporated into the generalised cross-validation score. A higher penalty forces the algorithm to penalise complexity more aggressively, inherently favouring a more parsimonious final model without fundamentally altering the initial sequence of terms generated by the forward pass. The overall pruning strategy is dictated by the **pmethod** argument, which defaults to standard backward elimination but can be switched to k-fold cross-validation or disabled entirely. Finally, the **nprune** argument provides a direct mechanism to enforce a strict upper bound on the number of terms retained in the final model, overriding the algorithmic optimum if a smaller structural footprint is explicitly required.

5.0.0.3 Extending the framework to logistic classification

Because the response variable represents a binary outcome, a formal logistic framework is more appropriate than treating the problem as a linear probability model. The algorithm adapts to classification tasks by first fitting a standard Gaussian model to execute the term selection and pruning phases. It subsequently refits the retained basis functions using a binomial likelihood.

```
fit_l1 <- earth(V58 ~ ., data = train, degree = 1,
               glm = list(family = binomial))
fit_l2 <- earth(V58 ~ ., data = train, degree = 2,
               glm = list(family = binomial))

auc <- function(p, y) {
  r <- rank(p); n1 <- sum(y == 1); n0 <- sum(y == 0)
  (sum(r[y == 1]) - n1 * (n1 + 1) / 2) / (n1 * n0)
}

clip <- function(p) pmin(pmax(p, 1e-12), 1 - 1e-12)
metrics <- function(p, y, label) {
  data.frame(model = label,
             acc     = round(mean((p > 0.5) == y), 4),
             auc     = round(auc(p, y), 4),
             logloss = round(-mean(y * log(clip(p)) + (1 - y) * log(1 - clip(p))), 4))
}
```

5.0.0.4 Benchmarking predictive performance against a GAM

Benchmarking the regression spline framework against a generalised additive model illustrates the contrasting mechanisms each methodology employs to handle zero-inflated, right-skewed data. Because standard continuous spline bases allocate their flexibility evenly across the

entire data domain, providing raw, heavily skewed inputs inadvertently forces the spline to concentrate its evaluation capacity around the massive pile of zeros. To counteract this vulnerability, a logarithmic transformation is universally applied to the predictors prior to fitting the generalised additive model. Furthermore, an extra shrinkage penalty is activated on the null space of the basis, allowing smooth terms to be completely shrunk out of the model if they do not sufficiently contribute to the final fit. In contrast, the hinge functions inherent to the competing methodology naturally place their interior knots exactly at observed data points, allowing them to inherently process the raw skewed data directly without requiring external transformation.

R

```
lg <- function(d) { d[paste0("V", 1:57)] <- log1p(d[paste0("V", 1:57)]); d }
train_g <- lg(train)
test_g  <- lg(test)

nuniq <- sapply(train[paste0("V", 1:57)], function(v) length(unique(v)))
terms <- ifelse(nuniq >= 10,
                sprintf("s(V%d, k = 5)", 1:57),
                sprintf("V%d", 1:57))
form  <- as.formula(paste("V58 ~", paste(terms, collapse = " + ")))

gam_raw_path <- "data/MARS/fit_gam_raw.rds"
if (file.exists(gam_raw_path)) {
  fit_gam_raw <- readRDS(gam_raw_path)
} else {
  fit_gam_raw <- gam(form, data = train, family = binomial,
                    method = "REML", select = TRUE)
  saveRDS(fit_gam_raw, gam_raw_path)
}

gam_log_path <- "data/MARS/fit_gam_log.rds"
if (file.exists(gam_log_path)) {
  fit_gam_log <- readRDS(gam_log_path)
} else {
  fit_gam_log <- gam(form, data = train_g, family = binomial,
                    method = "REML", select = TRUE)
  saveRDS(fit_gam_log, gam_log_path)
}

p_gam_raw <- drop(predict(fit_gam_raw, newdata = test, type = "response"))
p_gam_log <- drop(predict(fit_gam_log, newdata = test_g, type = "response"))
p_m1      <- drop(predict(fit_l1, newdata = test, type = "response"))
p_m2      <- drop(predict(fit_l2, newdata = test, type = "response"))
```

```

tab_bench <- rbind(
  metrics(p_gam_raw, test$V58, "GAM (raw inputs, select)"),
  metrics(p_gam_log, test$V58, "GAM (log1p, select)"),
  metrics(p_m1,      test$V58, "MARS degree 1 + logit"),
  metrics(p_m2,      test$V58, "MARS degree 2 + logit")
)
rownames(tab_bench) <- NULL
kable(tab_bench, digits = 4)

```

Table 5.3: Comparative predictive accuracy across generalised additive models and MARS variants on the held-out test set.

model	acc	auc	logloss
GAM (raw inputs, select)	0.9391	0.9784	0.2002
GAM (log1p, select)	0.9565	0.9867	0.1402
MARS degree 1 + logit	0.9384	0.9801	0.1771
MARS degree 2 + logit	0.9449	0.9830	0.1595

Evaluating the benchmark metrics highlights the performance trade-offs between the two non-linear modelling structures. The generalised additive model fitted on the logarithmically transformed data is the unequivocal best performer across all criteria, achieving the highest overall accuracy and area under the curve, alongside the lowest log-loss. It heavily benefits from its fully penalised smooth terms and the rigorous logarithmic preprocessing required to manage distributions with a high concentration of zero values.

Stripping away this preprocessing exposes the structural vulnerability of continuous splines on raw, skewed data. While the raw-input GAM marginally outperforms the strictly additive MARS configuration (degree 1) in pure classification accuracy and AUC, it suffers the highest log-loss of all evaluated models, indicating severe instability and miscalibration in its predicted probabilities. Furthermore, allowing the MARS framework to capture pairwise interactions (degree 2) enables it to bridge the performance gap and comfortably surpass the raw-input GAM across the board. Ultimately, the core theoretical trade-off centres on workflow efficiency: the multivariate adaptive regression splines approach achieves highly competitive classification natively, bypassing the manual transformations the generalised additive model strictly requires to function on heavily skewed inputs. While not a definitive rule of thumb, this highlights that model selection remains highly dependent on the specific nature of the data and the analytical goals of the project.

5.0.0.5 Visualising non-linear smooth components

Extracting the most significant smooth components from the GAM w.r.t. the log-transformed data provides visual confirmation of data structures structurally captured even on the transformed scale, let alone the raw scale.

```

st <- summary(fit_gam_log)$s.table
top <- order(st[, "p-value"])[1:6]

par(mfrow = c(2, 3))
for (i in top) {
  plot(fit_gam_log, select = i, scale = 0, scheme = 1, col = "blue", las = 1,
       ylab = TeX(paste0("$f(V", as.numeric(gsub("[^0-9]", "",
                                                rownames(st)[i])), ")$"))))
}

```

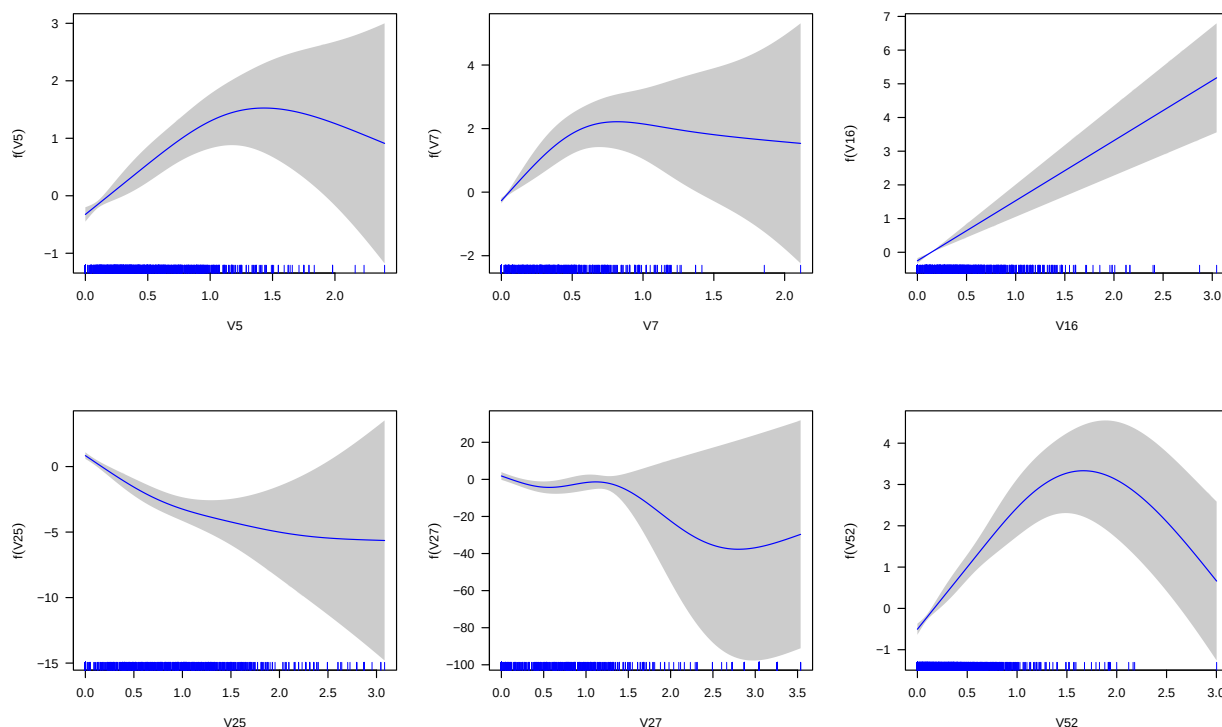


Figure 5.2: The six most statistically significant smooth functions estimated by the penalised generalised additive model, illustrating the non-linear relationships identified on the transformed inputs.

```

par(mfrow = c(1, 1))

```

Observing the rug plots along the horizontal axes reveals that the underlying data distribution remains heavily concentrated within the lower third of the range. As the predictor values increase, the data becomes increasingly sparse towards the right tail of the domain. This lack of empirical support causes the confidence interval bands to balloon rapidly in these upper regions. Although the exact point of divergence and the magnitude of the widening vary among the individual predictors, this escalating structural uncertainty is a noticeable and consistent characteristic across all the visualised smooth components.

Chapter 6

Wavelets

```
library(wavethresh)
library(splines)
library(knitr)
```

Daubechies Wavelets

Ingrid Daubechies developed a family of compactly supported wavelets, named according to their filter numbers. Plot the Daubechies mother wavelets for filter numbers 1 to 10 and evaluate the underlying filter coefficients and vanishing moments.

Ans: The Daubechies family is defined by the number of vanishing moments, denoted by the filter number N . The associated filter contains $2N$ coefficients, with $N = 1$ corresponding to the standard Haar wavelet. Plotting the first ten mother wavelets from the extremal phase family illustrates the fundamental trade-off in wavelet construction: as N increases, the wavelet becomes progressively smoother but its spatial support widens to $2N - 1$, sacrificing strict time localisation for improved frequency resolution

```
par(mfrow = c(5, 2), mar = c(3, 3, 2, 1))
for (N in 1:10) {
  draw.default(filter.number = N, enhance = T, family = "DaubExPhase",
    main = paste("N =", N), xlab = "", ylab = "")
}
```

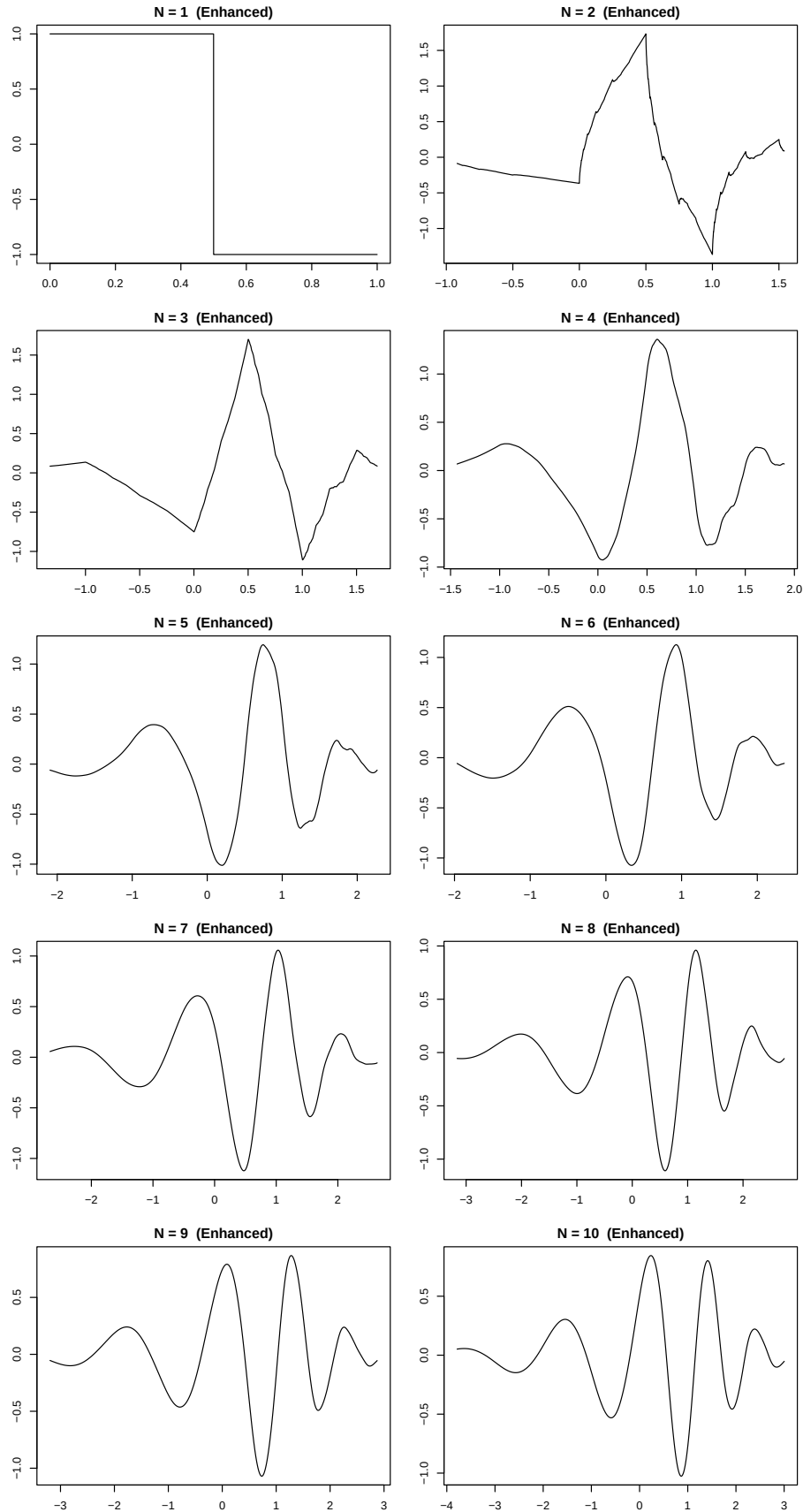


Figure 6.1: Daubechies mother wavelets for filter numbers 1 through 10, demonstrating the structural progression from the discontinuous Haar step function to increasingly smooth, continuous oscillations.

```
par(mfrow = c(1, 1))
```

Because there is no closed-form expression for most Daubechies wavelets, they are numerically approximated using their filter coefficients h_n , which define the wavelet via the two-scale dilation equations:

$$\phi(x) = \sqrt{2} \sum_n h_n \phi(2x - n), \quad \psi(x) = \sqrt{2} \sum_n g_n \phi(2x - n)$$

where $g_n = (-1)^n h_{2N-1-n}$. To achieve an orthonormal basis with sparse representation capabilities, these coefficients must strictly satisfy normalisation, orthogonality to even shifts, and exactly N vanishing moments. Evaluating these mathematical constraints programmatically confirms that the extracted filter weights reconstruct the desired structural properties.

```
check_filter <- function(N, family = "DaubExPhase") {
  h <- filter.select(filter.number = N, family = family)$H
  h_len <- length(h)
  n <- 0:(h_len - 1)

  sum_h <- round(sum(h), 8)
  sum_h2 <- round(sum(h^2), 8)

  # Iterate through all possible even shifts (2k) and calculate
  # the inner product
  max_even_shift <- 0
  for (k in 1:(N - 1)) {
    shift_amount <- 2 * k

    original_segment <- h[1:(h_len - shift_amount)]
    shifted_segment <- h[(1 + shift_amount):h_len]

    inner_prod <- sum(original_segment * shifted_segment)
    max_even_shift <- max(max_even_shift, abs(inner_prod))
  }
  even_shift_val <- signif(max_even_shift, 3)

  # Vanishing moments
  moments <- sapply(0:N, function(p) {
    numerator <- abs(sum((-1)^n * (n^p) * h))
    denominator <- sum(abs((n^p) * h))
    numerator / denominator
  })

  # The first N moments should be practically zero
```

```

num_vanishing <- sum(moments[1:N] < 1e-6)
next_moment <- signif(moments[N + 1], 3)

data.frame(
  N = N,
  length = h_len,
  sum_h = sum_h,
  sum_h2 = sum_h2,
  even_shift_innerprod = even_shift_val,
  num_vanishing = num_vanishing,
  next_moment = next_moment
)
}

tab_filters <- do.call(rbind, lapply(2:10, check_filter))
rownames(tab_filters) <- NULL

kable(tab_filters)

```

Table 6.1: Verification of Daubechies filter constraints, confirming normalisation, orthogonality, and exactly N vanishing moments.

N	length	sum_h	sum_h2	even_shift_innerprod	num_vanish- ing	next_mo- ment
2	4	1.414214	1	0	2	0.423000
3	6	1.414214	1	0	3	0.186000
4	8	1.414214	1	0	4	0.084600
5	10	1.414214	1	0	5	0.034500
6	12	1.414214	1	0	6	0.013000
7	14	1.414214	1	0	7	0.004740
8	16	1.414214	1	0	8	0.001780
9	18	1.414214	1	0	9	0.000680
10	20	1.414214	1	0	10	0.000245

6.0.1 Mexican Hat Wavelet

The Mexican Hat mother wavelet function is defined as $\psi(t) = (1 - t^2) \exp(-t^2/2)$. How would you translate and dilate this to obtain daughter wavelets?

Ans:

Daughter wavelets are mathematically generated from a continuous mother wavelet $\psi(t)$ by applying dilation and shift translation operators. The standard transformation formula is:

$$\psi_{j,k}(t) = 2^{j/2} \psi(2^j t - k)$$

where j represents the dyadic dilation integer that shrinks or expands the wavelet's frequency scale, k represents the translation integer shifting the wavelet along the time domain, and the $2^{j/2}$ factor ensures that the L_2 norm of the daughter wavelet remains strictly normalised such that $\|\psi_{j,k}\| = \|\psi\|$. Substituting the Mexican Hat equation into this general formula yields the specific daughter wavelet generation function:

$$\psi_{j,k}(t) = 2^{j/2} [1 - (2^j t - k)^2] \exp\left(-\frac{(2^j t - k)^2}{2}\right)$$

6.0.2 Haar father wavelet convolved with a signal

Create a signal ($Y = \sin(x)$) on 16 equally spaced points in $[0, 2\pi]$

Convolve the Haar father wavelet with the signal (2 points wavelet). Scale the wavelet so that it has L2 norm = 1. R function: `convolve`. Plot and check.

Ans:

```
X <- seq(0, 2*pi, length.out = 16)
Y <- sin(X)
```

```
haar_father <- rep(1/sqrt(2), 2)
sqrt(sum(haar_father^2)) # L2 norm = 1
```

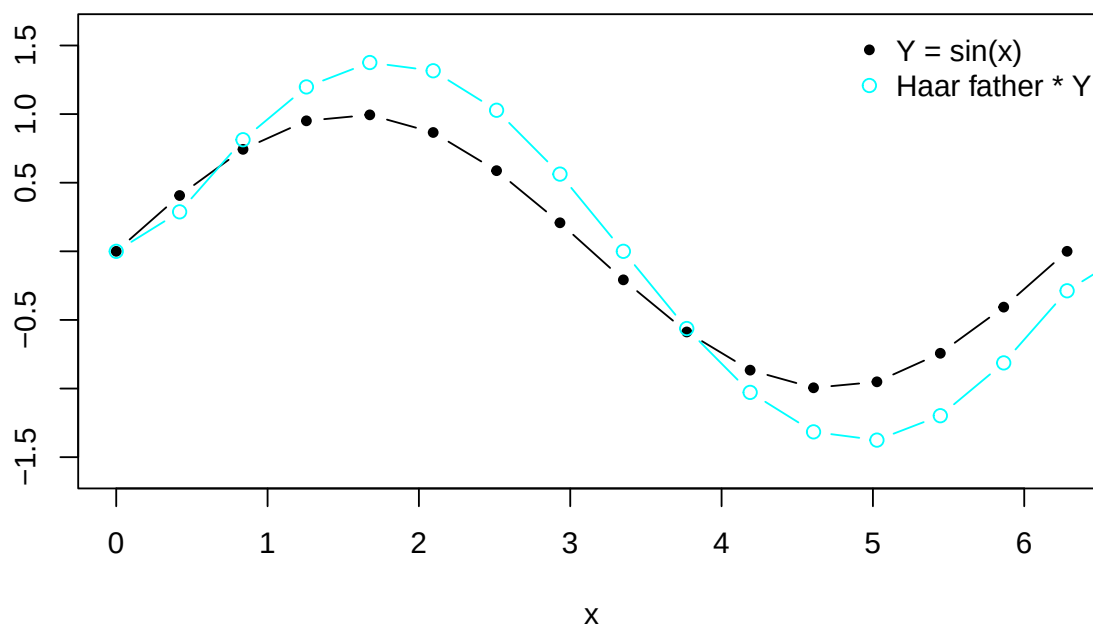
```
[1] 1
```

```
conv_Y <- convolve(Y, haar_father, type = "open")
```

```
conv_Y
```

```
[1] 5.367439e-17 2.876062e-01 8.130890e-01 1.197981e+00 1.375732e+00
[6] 1.315606e+00 1.027999e+00 5.626427e-01 6.791953e-16 -5.626427e-01
[11] -1.027999e+00 -1.315606e+00 -1.375732e+00 -1.197981e+00 -8.130890e-01
[16] -2.876062e-01 -1.828603e-16
```

```
plot(X, Y, type = "b", pch = 20, xlab = "x", ylab = "", ylim = c(-1.6, 1.6))
lines(c(X, X[16] + diff(X)[1]), conv_Y, type = "b", pch = 1, col = "cyan")
legend("topright", c("Y = sin(x)", "Haar father * Y"), pch = c(20, 1),
      col = c("black", "cyan"), bty = "n")
```



Note that the output is not on the same scale as Y : it peaks at about 1.4 ($\sqrt{2}$), well above 1. This occurs because the sum of two adjacent values, each close to 1 at the peak, is divided by $\sqrt{2}$ rather than by 2. This specific normalisation ensures the transform is mathematically orthonormal rather than a simple geometric average.

6.0.3 The discrete wavelet transform from first principles

Evaluating a single level of the discrete wavelet transform requires convolving the signal with the scaling filter (low-pass) and the wavelet filter (high-pass), followed by a downsampling step that retains only every second value. Using the unscaled Haar filters $(1, 1)$ and $(1, -1)$ on the specific vector $y = (1, 1, 7, 9, 2, 8, 8, 6)$ from the lecture material explicitly demonstrates this mechanical process.

```
xx      <- c(1, 1, 7, 9, 2, 8, 8, 6)
father  <- c(1, 1)
mother  <- c(1, -1)

# sums of pairs
A_raw <- convolve(xx, father, type = "open")[c(2, 4, 6, 8)]

A_raw
```

```
[1]  2 16 10 14
```

```
D_raw <- convolve(xx, mother, type = "open")[c(2, 4, 6, 8)]
D_raw
```

```
[1] 0 -2 -6 2
```

Because R's `convolve(..., type = "open")` function natively computes a statistical cross-correlation rather than a formal mathematical convolution, it does not reverse the order of the filter coefficients before sliding them across the data. This is not an issue for the father filter (1, 1), as its reflection is identical to itself. However, for an antisymmetric high-pass filter like (1, -1), failing to reverse the order scrambles the formal convolution. This is easily resolved by manually wrapping the filter in `rev()` to reverse its elements prior to computation, completely bypassing the issue. Also worth noting that for an asymmetrical filter like the mother, using its reverse in the function is equivalent to the negative of the regular `convolve`.

```
D_raw <- convolve(xx, rev(mother), type = "open")[c(2, 4, 6, 8)]
D_raw
```

```
[1] 0 2 6 -2
```

Scaling both resulting downsampled vectors by $1/\sqrt{2}$ yields the correct orthonormal coefficients.

6.0.3.1 Implementing the full pyramidal algorithm

Extending this single-step logic allows for the construction of a complete one-dimensional discrete wavelet transform function. The explicit algorithm iterates through a specified number of filter bank passes, continuously downsampling the low-pass approximation to generate progressively coarser resolutions.

```
dwt <- function(X, L, H, n) {
  A <- list()
  D <- list()

  A[[1]] <- X # Init

  for (i in 1:n) {
    current_A <- A[[i]]
    # Downsampling indices to retain every second value
    idx <- seq(2, length(current_A), by = 2)

    # High-pass filter and downsample
    D[[i]] <- convolve(current_A, rev(H), type = "open")[idx]

    # Low-pass filter and downsample
    A[[i+1]] <- convolve(current_A, rev(L), type = "open")[idx]
  }
}
```

```
# Package results: An, Dn, Dn-1, ..., D1
res <- list(An = A[[n+1]])
for (i in n:1) {
  res[[paste0("D", i)]] <- D[[i]]
}
return(res)
}

# simulated noisy signal
set.seed(1409)
N <- 1024
Xs <- seq(0, 4*pi, length.out = N)
X <- sin(Xs) + rnorm(N, mean = 0, sd = 0.2)

L_haar <- rep(1/sqrt(2), 2)
H_haar <- c(1/sqrt(2), -1/sqrt(2))

res <- dwt(X, L_haar, H_haar, 4)

# ecomposition stack
par(mfrow = c(6, 1), mar = c(0.5, 4, 0.5, 1), oma = c(1, 1, 1, 1))

components <- list("X" = X,
                   "A4" = res$An,
                   "D4" = res$D4,
                   "D3" = res$D3,
                   "D2" = res$D2,
                   "D1" = res$D1)

for (name in names(components)) {
  plot(components[[name]], type = "l", xaxt = "n", yaxt = "n",
        ann = FALSE, col = 'blue')
  mtext(name, side = 2, line = 1, las = 2, cex = 1.1)
}
```

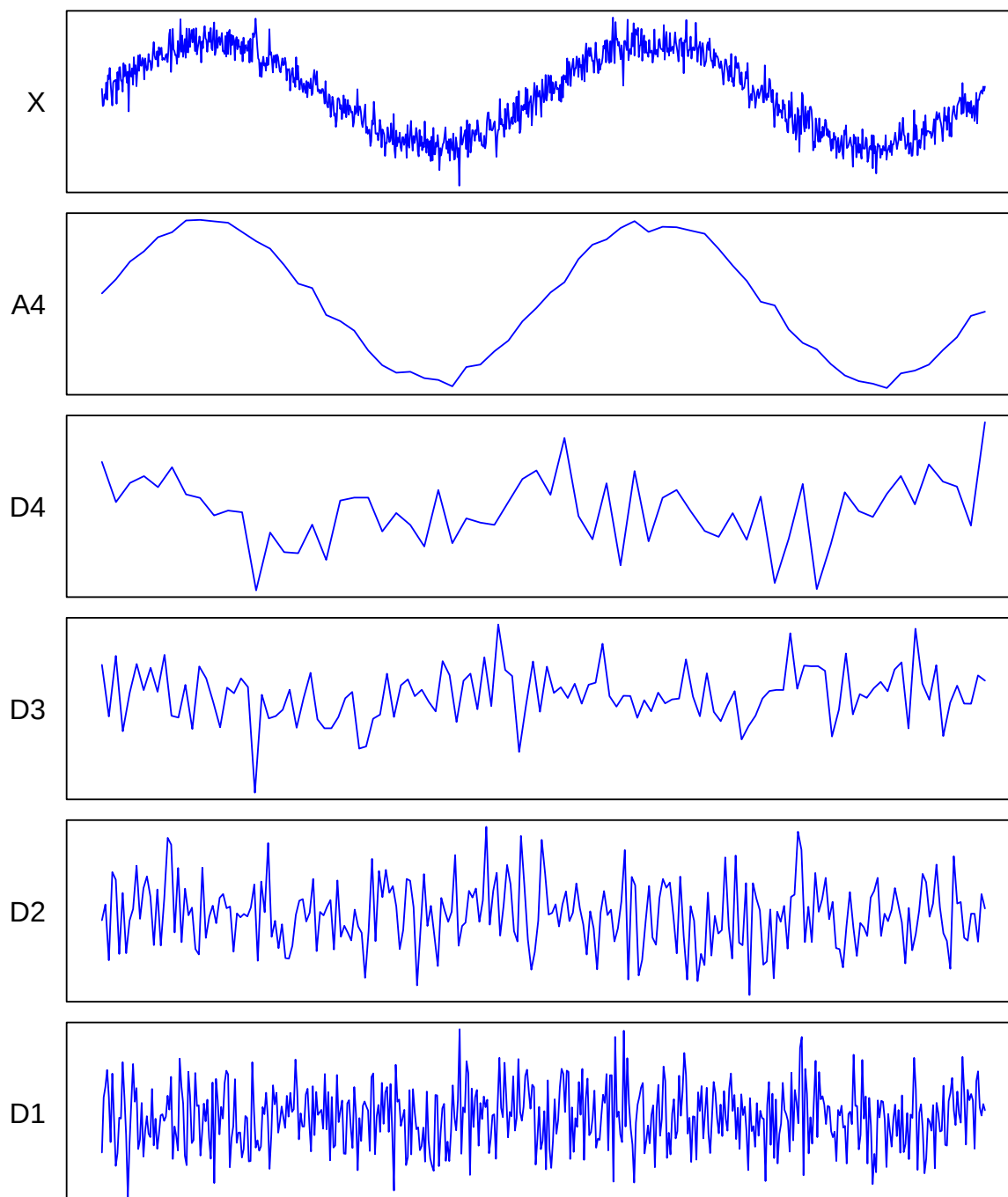


Figure 6.2: Multiresolution decomposition using the pyramidal algorithm, displaying the original signal X alongside the final low-pass approximation (A_4) and the sequential levels of high-pass detail (D_1 through D_4)

```
par(mfrow = c(1, 1), mar = c(5, 4, 4, 2) + 0.1, oma = c(0, 0, 0, 0))
```

Structuring the output panels in a stacked layout strips away arbitrary dimensional limits and

explicitly isolates the isolated wavelet variations, demonstrating how the algorithm progressively filters ambient high-frequency noise downward into the detail tiers, D1-D4, preserving the macroscopic structural continuity of the original signal in the terminal approximation, A4.

Chapter 7

Gaussian Processes

```
library(cmdstanr)
library(knitr)
```

7.1 Problem 1

Consider the following six observations of a response y at a single real input x .

x	0.9	3.8	5.2	6.1	7.5	9.6
y	0.1	1.2	2.1	1.1	1.5	1.2

Use a Bayesian approach to fit a Gaussian process model to these data. Simulate from the posterior and illustrate. Carefully explain what the Stan program does. Use equations.

Ans:

```
x_obs <- c(0.9, 3.8, 5.2, 6.1, 7.5, 9.6)
y_obs <- c(0.1, 1.2, 2.1, 1.1, 1.5, 1.2)

plot(x_obs, y_obs, pch = 20, xlim = c(0, 10), ylim = c(-1, 3),
      xlab = "x", ylab = "y")
```

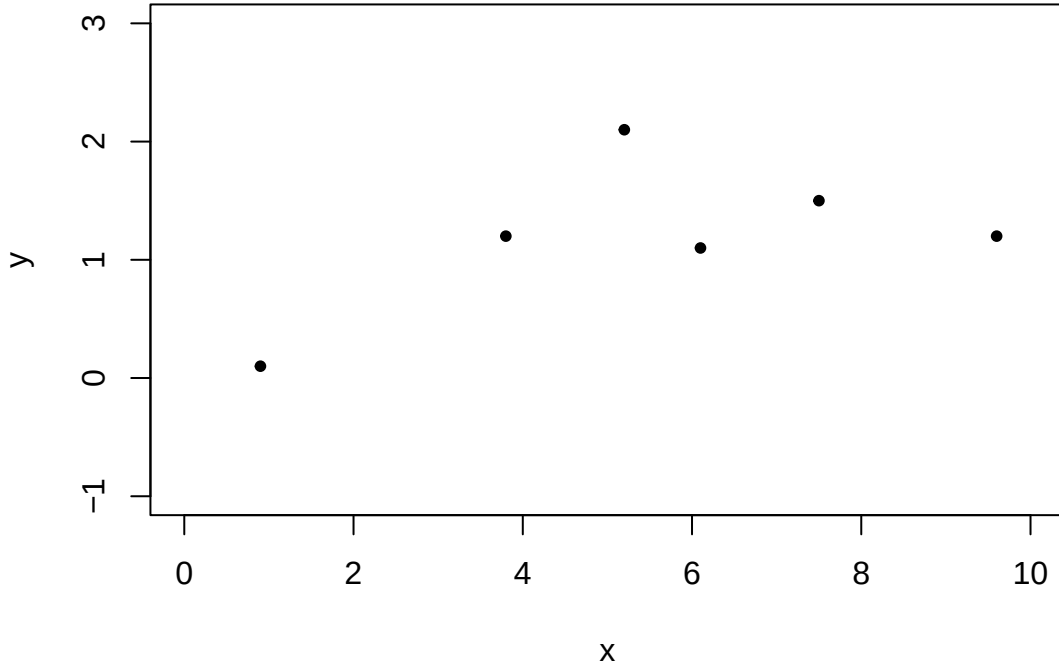


Figure 7.1: The six observations.

The model treats the observations as noisy evaluations of an unknown function f , which is given a Gaussian process prior with zero mean and a squared exponential covariance function,

$$y_i = f(x_i) + \varepsilon_i, \quad \varepsilon_i \sim N(0, \sigma^2), \quad f \sim \mathcal{GP}(0, k(x, x')), \quad k(x, x') = \alpha^2 \exp\left(-\frac{(x - x')^2}{2\rho^2}\right).$$

Here α is the marginal standard deviation of the function, ρ is the length scale that controls how quickly the correlation between $f(x)$ and $f(x')$ decays with distance, and σ is the standard deviation of the observation error. A Gaussian process is characterised by the fact that any finite collection of function values is multivariate normal, so for the $n = 6$ observed inputs the function values satisfy $\mathbf{f} \sim N(\mathbf{0}, K)$ with $K_{ij} = k(x_i, x_j)$.

In matrix form:

$$\mathbf{y} = \mathbf{f} + \boldsymbol{\varepsilon}$$

Given $\mathbf{f} \sim N(\mathbf{0}, K)$ and independent errors $\boldsymbol{\varepsilon} \sim N(\mathbf{0}, \sigma^2 I_n)$:

$$E[\mathbf{y}] = E[\mathbf{f}] + E[\boldsymbol{\varepsilon}] = \mathbf{0} + \mathbf{0} = \mathbf{0}$$

Since $\mathbf{f}$ and $\boldsymbol{\varepsilon}$ are independent, $\text{Cov}(\mathbf{f}, \boldsymbol{\varepsilon}) = \mathbf{0}$:

$$\text{Var}(\mathbf{y}) = \text{Var}(\mathbf{f}) + \text{Var}(\boldsymbol{\varepsilon}) = K + \sigma^2 I_n$$

Since linear combinations of normal variables remain normal, integrating out $\mathbf{f}$ yields the marginal distribution of the data given the hyperparameters $\theta = (\rho, \alpha, \sigma)$:

$$\mathbf{y} \mid \theta \sim N(\mathbf{0}, C), \quad C = K + \sigma^2 I_n.$$

This is the only likelihood that the sampler has to evaluate. The Bayesian part of the problem is then to place priors on θ and to sample from $p(\theta \mid \mathbf{y}) \propto p(\mathbf{y} \mid \theta) p(\theta)$ using Hamiltonian Monte Carlo. The function values at new inputs are obtained afterwards by drawing from their conditional distribution given the data and each sampled value of θ .

The Stan programme is the `gp1.stan` file found on the course amathuba, with no alterations made.

The Stan program executes this model across three primary blocks. The transformed data block standardises the observed response and the N_2 new predictive inputs:

$$\tilde{y}_i = \frac{y_i - \bar{y}}{s_y}, \quad \tilde{x}_i = \frac{x_i - \bar{x}}{s_x}, \quad \tilde{x}_j^* = \frac{x_j^* - \bar{x}}{s_x}.$$

Standardisation centres the function on the sample mean and places the hyperparameters on a unit scale, allowing the priors $\rho, \alpha, \sigma \sim N^+(0, 1)$ to remain sensible regardless of the original data units. The model block constructs the covariance matrix C and evaluates the marginal likelihood. Instead of directly inverting C , the program computes its lower-triangular Cholesky factor $C = LL^\top$ for computational speed and numerical stability. The log-density is then evaluated directly using this factor:

$$\log p(\tilde{\mathbf{y}} \mid \theta) = -\frac{1}{2} \tilde{\mathbf{y}}^\top (LL^\top)^{-1} \tilde{\mathbf{y}} - \frac{1}{2} \log |LL^\top| - \frac{n}{2} \log 2\pi.$$

The generated quantities block runs once per posterior draw of θ to sample the latent function at the new inputs. Because Gaussian processes are jointly normal, the new function values $\mathbf{f}^*$ and the observed data satisfy:

$$\begin{pmatrix} \tilde{\mathbf{y}} \\ \mathbf{f}^* \end{pmatrix} \sim N \left(\mathbf{0}, \begin{pmatrix} C & K_* \\ K_*^\top & K_{**} \end{pmatrix} \right),$$

where K_* is the $n \times N_2$ cross-covariance matrix and K is the covariance among the new inputs. Conditioning on the observed data yields the predictive distribution:

$$\mathbf{f}^* \mid \tilde{\mathbf{y}}, \theta \sim N(\boldsymbol{\mu}^*, \Sigma^*)$$

$$\boldsymbol{\mu}^* = K_*^\top C^{-1} \tilde{\mathbf{y}}, \quad \Sigma^* = K_{**} - K_*^\top C^{-1} K_*$$

To compute the covariance without inverting C , the program defines $V = L^{-1} K_*$. Substituting $C = LL^\top$ into the predictive variance equation yields:

$$K_*^\top C^{-1} K_* = K_*^\top (LL^\top)^{-1} K_* = K_*^\top L^{-\top} L^{-1} K_* = (L^{-1} K_*)^\top (L^{-1} K_*) = V^\top V$$

$$\implies \Sigma^* = K_{**} - V^\top V.$$

A small constant (10^{-9}) is added to the diagonal of Σ^* to ensure the matrix remains numerically stable before drawing the new function values. Because one function is drawn for each sampled value of θ , the resulting collection of draws forms a sample from the posterior predictive distribution $p(\mathbf{f}^* \mid \tilde{\mathbf{y}})$, passing forward hyperparameter uncertainty into the function estimates.

A structural property of the provided code requires the response to be standardised before it is passed to Stan as the likelihood in the model block evaluates the supplied `y_obs`, but the priors and predictions rely on the internally standardised `yn`. The returned draws are then converted back to the original scale in R via $f^* = s_y \tilde{f}^* + \bar{y}$ and $\sigma_y = s_y \sigma$.

We standardise the response, compile the program, and sample across a grid of 100 inputs ($\in [0, 10]$). With only $n = 6$ observations, the posterior distribution of the hyperparameters is highly uncertain. To prevent the sampler from making numerical errors when exploring this tricky space, we increase `adapt_delta` to 0.95, which forces the algorithm to take smaller, more cautious steps.

```
x_grid <- seq(0, 10, length.out = 100)

y_mean <- mean(y_obs)
y_sd    <- sd(y_obs)
y_std   <- (y_obs - y_mean) / y_sd

data_list <- list(N_obs = length(x_obs),
                  y_obs = y_std,
                  x_obs = x_obs,
                  N2    = length(x_grid),
                  x2     = x_grid)
```

```
gp_model <- cmdstan_model("stan/gp1.stan")

gp_fit <- gp_model$sample(data = data_list,
                          seed = 123,
                          chains = 4,
                          parallel_chains = 2,
                          adapt_delta = 0.95,
                          iter_sampling = 2000,
                          refresh = 0)
```

Running MCMC with 4 chains, at most 2 in parallel...

Chain 1 finished in 1.1 seconds.
 Chain 2 finished in 1.0 seconds.
 Chain 4 finished in 1.0 seconds.
 Chain 3 finished in 1.0 seconds.

All 4 chains finished successfully.
 Mean chain execution time: 1.0 seconds.
 Total execution time: 2.4 seconds.

```
hyper_summary <- gp_fit$summary(c("rho", "alpha", "sigma"))
hyper_summary <- as.data.frame(hyper_summary[, c("variable", "mean", "median",
                                                "sd", "q5", "q95",
                                                "rhat", "ess_bulk")])

hyper_summary$variable <- c("$\\rho$", "$\\alpha$", "$\\sigma$")
colnames(hyper_summary) <- c("Parameter", "Mean", "Median", "SD",
                             "5%", "95%", "$\\hat{R}$", "$ESS_{bulk}$")

kable(hyper_summary, digits = 3, row.names = FALSE, escape = FALSE)
```

Table 7.2: Posterior summaries of the hyperparameters on the standardised scale.

Parameter	Mean	Median	SD	5%	95%	$\hat{R}$	ESS_{bulk}
ρ	0.697	0.560	0.574	0.043	1.825	1.005	1628.317
α	0.647	0.590	0.442	0.068	1.461	1.006	1827.798
σ	0.853	0.833	0.369	0.225	1.497	1.013	1302.255

```
rho_draws <- as.numeric(gp_fit$draws("rho", format = "matrix"))
sigma_draws <- as.numeric(gp_fit$draws("sigma", format = "matrix"))

# back to the original scale of x and y
rho_original <- rho_draws * sd(x_obs)
```

```

sigma_original <- sigma_draws * y_sd

n_divergent <- sum(gp_fit$diagnostic_summary(quiet = TRUE)$num_divergent)
### gives a val of 6

rho_draws    <- as.numeric(gp_fit$draws("rho", format = "matrix"))
alpha_draws  <- as.numeric(gp_fit$draws("alpha", format = "matrix"))
sigma_draws  <- as.numeric(gp_fit$draws("sigma", format = "matrix"))

# Transform back to the original scale of x and y
rho_orig    <- rho_draws * sd(x_obs)
alpha_orig  <- alpha_draws * y_sd
sigma_orig  <- sigma_draws * y_sd

orig_summary <- data.frame(
  Parameter = c("$\\rho$ (length scale)", "$\\alpha$ (marginal SD)",
    "$\\sigma$ (noise SD)"),
  Median    = c(median(rho_orig), median(alpha_orig), median(sigma_orig)),
  Q5        = c(quantile(rho_orig, 0.05), quantile(alpha_orig, 0.05),
    quantile(sigma_orig, 0.05)),
  Q95       = c(quantile(rho_orig, 0.95), quantile(alpha_orig, 0.95),
    quantile(sigma_orig, 0.95))
)

kable(orig_summary, digits = 3,
  col.names = c("Parameter", "Median", "5%", "95%"),
  row.names = FALSE, escape = FALSE)

```

Table 7.3: Posterior summaries of the hyperparameters converted to the original data scale.

Parameter	Median	5%	95%
ρ (length scale)	1.687	0.131	5.494
α (marginal SD)	0.384	0.044	0.952
σ (noise SD)	0.543	0.147	0.975

The potential scale reduction factors ($\hat{R}$) are close to 1.00 and the effective sample sizes (ESS_{bulk}) are adequate, confirming that the chains converged and mixed well. The sampler recorded 6 divergent transitions across all chains, which is a minor consequence of navigating a highly uncertain parameter space with limited data. We also note that the posteriors of the three hyperparameters are predictably wide given the $n = 6$ sample size.

Chapter 8

Extension I: Wavelet-Based Jump Detection and Volatility Estimation for High-Frequency Data

In high-frequency finance, traditional realised volatility estimators are upwardly biased because they capture both the continuous diffusion and discrete price jumps. It is with no surprise that along the years there have been many applications to separate the micro and macro structures that comprise the time series to correct for such jumps.

To this end, we implement an application of wavelets by closely following a foundational paper that structures a computational pipeline to adjust for price jumps.

8.0.1 Mathematical formulation

We model the latent logarithmic asset price $X_t = \ln S_t$ as an Itô semimartingale. Using a Merton-style structure,

$$dX_t = \mu_t dt + \sigma_t dW_t + J_t dN_t,$$

where W_t is a standard Brownian motion, N_t is a Poisson counting process, and J_t denotes the random jump magnitude. The total quadratic variation over $[0, T]$ decomposes into a continuous integrated-variance term and a discontinuous jump-variation term:

$$[X, X]_T = \int_0^T \sigma_s^2 ds + \sum_{0 < s \leq T} (\Delta X_s)^2 =: \Theta + \Psi.$$

The actually observed price additionally carries market microstructure noise,

$$Y_t = X_t + \epsilon_t, \quad \epsilon_t \stackrel{iid}{\sim} N(0, \eta^2), \quad \epsilon_t \perp X_t.$$

The object of primary economic interest is typically isolating Θ , the persistent continuous risk that is fundamental to asset pricing and standard volatility modelling. In contrast, Ψ represents transient, unpredictable shocks, and ϵ_t acts merely as observational friction. The methodological challenge is therefore constructing an estimator that recovers Θ while remaining robust to both the discrete jumps and the microstructure noise.

8.1 Literature Review

8.1.1 Fan and Wang (2007), “*Multi-scale jump and volatility analysis for high-frequency financial data*”, JASA

Fan and Wang’s starting point is that existing estimators had been built for one contaminant at a time: two-scale/multi-scale realized volatility handle noise but assume a continuous price process, while bipower variation handles jumps but assumes no noise. No existing method, at the time, handled both simultaneously.

Their solution is a three-step procedure. First, they wavelet-transform the observed noisy price path and show that, at a sufficiently fine resolution level, the wavelet coefficient at a jump location is bounded away from zero by an order of magnitude larger than at any continuous point, even in the presence of noise, so that a simple universal threshold isolates jump locations consistently. Second, jump sizes are estimated by averaging observations just before and after each detected jump, giving an estimator of the jump variation Ψ that converges at rate $n^{-1/4}$, which is the best attainable rate given how rough a jump-diffusion sample path is. Third, having subtracted the estimated jumps from the data, they show the “jump-adjusted” series behaves, to first asymptotic order, like data from a pure continuous diffusion contaminated only by noise, meaning that existing noise-robust estimators can simply be applied to the adjusted series and inherit their usual convergence rates without any loss from the jump-removal step. They additionally propose a purely wavelet-based realized-variance estimator (WRV) and prove that, under the Haar wavelet specifically, WRV is numerically identical to the jump-adjusted two-scale realised volatility (JTSRV) for a matching partition size. Furthermore, an application to one-minute EUR–USD and JPY–USD data over seven months of 2004 finds that jumps are common, with several per day on many days, and that ignoring them materially inflates estimated volatility.

8.2 Simulation design

Following Fan and Wang (2007, p. 18) as closely as possible, spot volatility follows a Geometric Ornstein–Uhlenbeck process,

$$d \log \sigma_t^2 = -(0.6802 + 0.10 \log \sigma_t^2) dt + 0.25 dW_{1,t}, \quad \text{Corr}(W_t, W_{1,t}) = -0.62,$$

simulated by an Euler scheme at $n = 1440$ minute-bars over a 24-hour trading day. The continuous part of the log-price is $dX_t^c = \sigma_t dW_t$. To mimic the jump process, between zero

and three jumps are added at $U(0, 1)$ locations with $N(0, (1/30)^2)$ sizes, and the whole path is observed with additive $N(0, \eta^2)$ noise, $\eta \in [0, 0.001]$. These parameters are chosen in the paper to match realistic FX microstructure dynamics. Because we work throughout in log-price, the implied price level $S_t = e^{X_t}$ is strictly positive by construction. It is to be noted that simulating such a process requires the use of Cholesky decomposition in order to create Brownian motions dW_t and $dW_{1,t}$ that are correlated as such. A detailed explanation is found in Appendix A, and an example of a simulated day is shown below.

```
library(ggplot2)
library(latex2exp)
library(knitr)

#####
# Merton jump-diffusion simulation + Haar MODWT
#####

# simulating the specified dynamics; we take the normal samples
# and create the brownian motions as specified in the appendix
simulate_day <- function(n = 1440, n_jumps = 1, eta = 0.0005) {
  dt <- 1 / n
  rho <- -0.62

  z1 <- rnorm(n)
  z2 <- rnorm(n)

  dW <- sqrt(dt) * z1
  dW1 <- sqrt(dt) * (rho * z1 + sqrt(1 - rho^2) * z2)

  logvar <- numeric(n + 1)
  logvar[1] <- -6.8020 # start at mean of the process
  for (i in 1:n) {
    logvar[i + 1] <- logvar[i] - (0.6802 + 0.10 * logvar[i]) * dt + 0.25 * dW1[i]
  }
  sigma <- sqrt(exp(logvar[-1])) # spot vol at each minute, length n

  ##  $dX^c_t$  as stipulated
  dXc <- sigma * dW
  Xc <- c(0, cumsum(dXc)) # as if we intergrated both sides;  $x_0 = 0$ 

  ## jump part:  $n\_jumps$  uniform locations on  $(0, 1]$ ,  $N(0, (1/30)^2)$  sizes
  jump_times <- sort(sample(2:(n + 1), n_jumps))
  jump_sizes <- rnorm(n_jumps, mean = 0, sd = 1 / 30)

  Xd <- numeric(n + 1) #Xd = discontinuous jump part
  for (l in seq_along(jump_times)) {
```

```

    Xd[jump_times[1]:(n + 1)] <- Xd[jump_times[1]:(n + 1)] + jump_sizes[1]
  }

  X <- Xc + Xd                                # true (jump-diff) log price
  eps <- rnorm(n + 1, sd = eta)
  Y <- X + eps                                # observed, noisy log price

  list(n = n, Xc = Xc, Xd = Xd, X = X, Y = Y, eps = eps,
       jump_times = jump_times, jump_sizes = jump_sizes,
       sigma = sigma, theta_true = sum(sigma^2 * dt)) # true integrated variance
}

```

```

set.seed(1408)
day <- simulate_day(n = 1440, n_jumps = 1, eta = 0.0006)
tt <- (0:day$n) / day$n

par(mfrow = c(3, 1), mar = c(2, 4, 2, 1), oma = c(2, 0, 0, 0))
# oma to add some space at the bottom. mar to add some space on
# the left side, bring the plots closer, less space needed on
# the right, and to make it more compact at top and bottom

# Plot 1; True cont path
plot(tt, day$Xc, type = "l", col = "navy", lwd = 1.2,
     main = TeX("True continuous path  $X^c_t$ "), ylab = "Log-price",
     xlab = "")
grid(nx = NULL, ny = NULL, col = "lightgray", lty = "dotted")
abline(v = tt[day$jump_times], col = "firebrick", lty = "dashed", lwd = 1.5)

# Plot 2; True jump-diff path
plot(tt, day$X, type = "l", col = "navy", lwd = 1.2,
     main = TeX("True jump-diffusion path  $X_t$ "), ylab = "Log-price",
     xlab = "")
grid(nx = NULL, ny = NULL, col = "lightgray", lty = "dotted")
abline(v = tt[day$jump_times], col = "firebrick", lty = "dashed", lwd = 1.5)

# Plot 3; observed noisy path
plot(tt, day$Y, type = "l", col = "navy", lwd = 1.2,
     main = TeX("Observed noisy path  $Y_t$ "), ylab = "Log-price",
     xlab = "Time of day")
grid(nx = NULL, ny = NULL, col = "lightgray", lty = "dotted")
abline(v = tt[day$jump_times], col = "firebrick", lty = "dashed", lwd = 1.5)

```

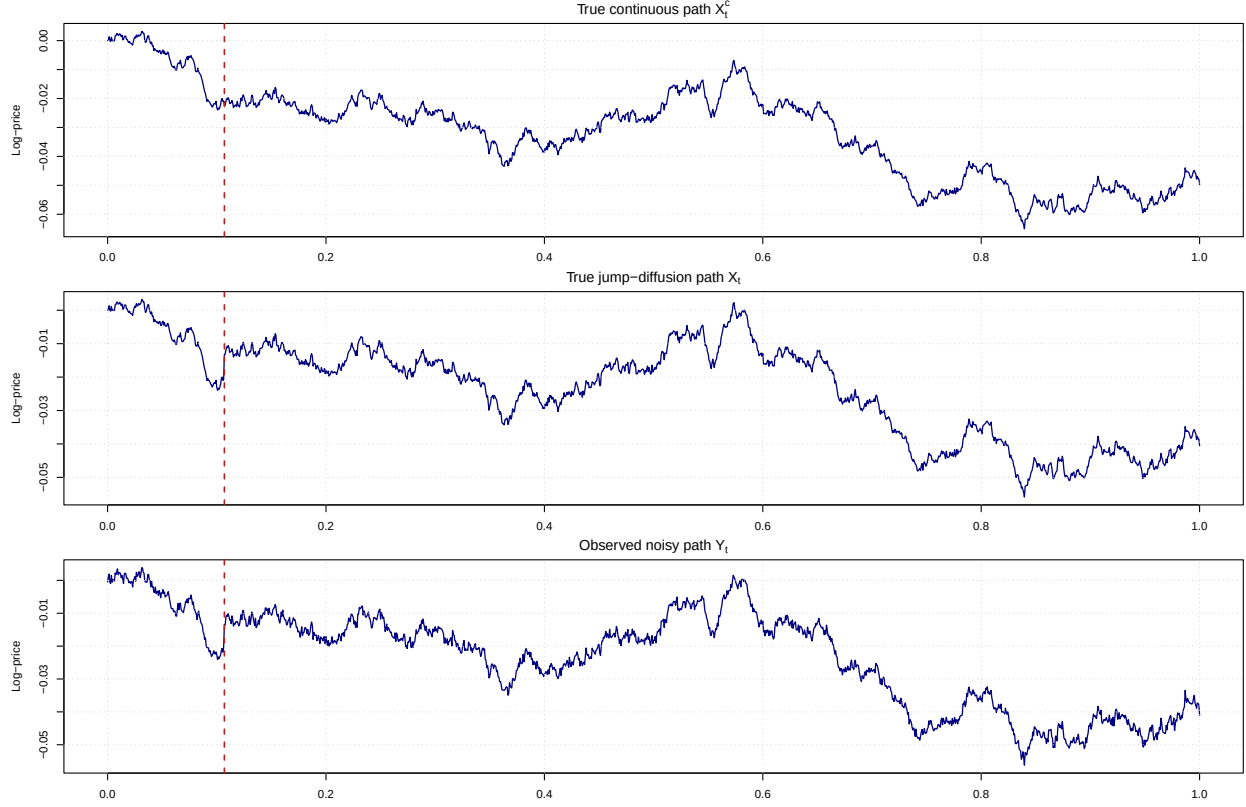


Figure 8.1: One simulated trading day: the latent continuous path, the true jump-diffusion path, and the noisy observed path actually available to an estimator.

```
par(mfrow = c(1, 1), mar = c(5, 4, 4, 2) + 0.1, oma = c(0, 0, 0, 0))
```

8.3 Wavelet Multiresolution Analysis

The standard Discrete Wavelet Transform (DWT) relies on downsampling at each scale, which strictly restricts the sample size to a power of two ($N = 2^J$) and makes the transform shift-variant. The Maximal Overlap Discrete Wavelet Transform (MODWT) overcomes these limitations by omitting the downsampling step. Consequently, the MODWT can process arbitrary sample sizes and is strictly translation-invariant. This shift-invariance is critical in high-frequency financial econometrics, as it ensures that the volatility decomposition is not artificially distorted by the arbitrary starting point of the time series. To implement the decomposition efficiently in R, we build the MODWT pyramid algorithm from first principles. In the standard literature (Percival and Walden, 2000, p.159-204), the MODWT at level j is defined generally for a filter of width L via circular convolution:

$$\mathcal{W}_{j,t} = \sum_{l=0}^{L-1} \tilde{h}_l \mathcal{V}_{j-1,t-l2^{j-1} \bmod n}$$

$$\mathcal{V}_{j,t} = \sum_{l=0}^{L-1} \tilde{g}_l \mathcal{V}_{j-1, t-l2^{j-1} \bmod n}$$

For the specific case of the Haar wavelet, the filter width is strictly $L = 2$, and the rescaled MODWT wavelet and scaling filter vectors are $\tilde{h} = (1, -1)^T/2$ and $\tilde{g} = (1, 1)^T/2$. Defining the original time series vector as $y = (y_1, \dots, y_n)^T$, the wavelet coefficients as $W_{j,t}$, and the scaling coefficients as $V_{j,t}$, the transform is implemented by applying the pyramid algorithm pseudocode outlined in Percival and Walden (2000, p. 178) with $V_0 \equiv y$ and circular boundary handling.

The decomposition isolates variation at scale 2^{j-1} , that is in this case, an investment horizon of that many minutes, and satisfies the discrete Parseval identity:

$$\sum_t y_t^2 = \sum_{j=1}^J \sum_t W_{j,t}^2 + \sum_t V_{J,t}^2$$

This ensures no energy is created or destroyed by the transform; it is strictly reallocated across scales. This identity is the reason a rescaled filter is used, because the standard orthonormal Haar filter $(1, -1)^T/\sqrt{2}$ does not satisfy it once the transform is undecimated. A numerical sanity check is performed using this identity to verify that the transformation preserves energy correctly.

Manual Haar MODWT pyramid algorithm

```
haar_modwt <- function(y, J) {
  N <- length(y)
  W <- vector("list", J)
  V <- y

  # Rescaled MODWT Haar filters
  h_tilde <- c(1/2, -1/2)
  g_tilde <- c(1/2, 1/2)
  L <- length(h_tilde)

  for (j in 1:J) {
    Wj <- numeric(N)
    Vj <- numeric(N)

    for (t in 0:(N - 1)) {
      k <- t
      Wj[t + 1] <- h_tilde[1] * V[k + 1]
      Vj[t + 1] <- g_tilde[1] * V[k + 1]

      for (n in 1:(L - 1)) {
        k <- k - 2^(j - 1)
```

```

    if (k < 0) {
      k <- k %% N # Circular boundary handling
    }
    Wj[t + 1] <- Wj[t + 1] + h_tilde[n + 1] * V[k + 1]
    Vj[t + 1] <- Vj[t + 1] + g_tilde[n + 1] * V[k + 1]
  }
}
W[[j]] <- Wj
V <- Vj
}
list(W = W, V = V, J = J)
}

```

8.4 Jump detection via the finest-scale wavelet coefficient

Fan and Wang (2007, p.9-11) highlight that at the finest resolution level, the wavelet coefficient of a Hölder-continuous diffusion path decays like $2^{-j(\alpha+1/2)}$ near any ordinary point. For a brownian motion, $\alpha \approx 1/2$. However, while near a jump it stays bounded below by order $2^{-j/2}$; an order of magnitude larger.

Concretely, the finest-scale ($j = 1$) Haar coefficient is just a rescaled first difference,

$$W_{1,t} = \frac{Y_t - Y_{t-1}}{2},$$

and a jump at t makes $|W_{1,t}|$ stand out sharply from the noise floor. Detection uses a universal threshold calibrated by an estimate of the noise scale in the paper, given by:

$$D_n = \hat{d}\sqrt{2\log n}, \quad \hat{d} = \frac{\text{median}(|W_{1,t}|)}{0.6745}.$$

Any t with $|W_{1,t}| > D_n$ is flagged as a jump location $\hat{\tau}$. One important implementation detail to notes as well is that a trading day is not periodic. Thus, the MODWT recursion must not wrap the last observation onto the first when computing $W_{1,1}$. A funny thing than an earlier version of this code did was use circular boundary handling everywhere, which led to a produced a spurious “jump” at $t = 1$ on every single simulated day. The fix is simply to leave $W_{1,1}$ undefined and exclude it from thresholding.

```

#### Jump detection
detect_jumps <- function(Y, W1 = NULL) {
  n <- length(Y)
  if (is.null(W1)) {
    # Haar coefficient (Y_t - Y_{t-1})/2, non-circular

```

```

    #a trading day has a start and an end,
    # so we must not wrap t=1 onto t=n.
    W1 <- c(NA, (Y[2:n] - Y[1:(n - 1)]) / 2)
  }
  d <- median(abs(W1), na.rm = TRUE) / 0.6745
  Dn <- d * sqrt(2 * log(n)) # threshold
  flagged <- which(!is.na(W1) & abs(W1) > Dn)
  list(W1 = W1, threshold = Dn, jump_idx = flagged)
}

jd <- detect_jumps(day$Y)

plot(tt, jd$W1, type = "n",
     ylab = expression(W["1,t"]),
     xlab = "Time of day")

mtext("Dashed = universal threshold; dotted = true jump; red dot = flagged",
     side = 3, line = 0.25, cex = 0.85)

grid(nx = NULL, ny = NULL, col = "lightgray", lty = "dotted")

# Wavelet coefficients
lines(tt, jd$W1, col = "grey40", lwd = 1.2)

# threshold
abline(h = c(-1, 1) * jd$threshold, col = "firebrick", lty = "dashed",
       lwd = 1.5)

# True jump location
abline(v = tt[day$jump_times], col = "steelblue", lty = "dotted", lwd = 1.5)

# Flagged jump points (solid red dots)
points(tt[jd$jump_idx], jd$W1[jd$jump_idx], col = "firebrick", pch = 19,
       cex = 1.2)

```

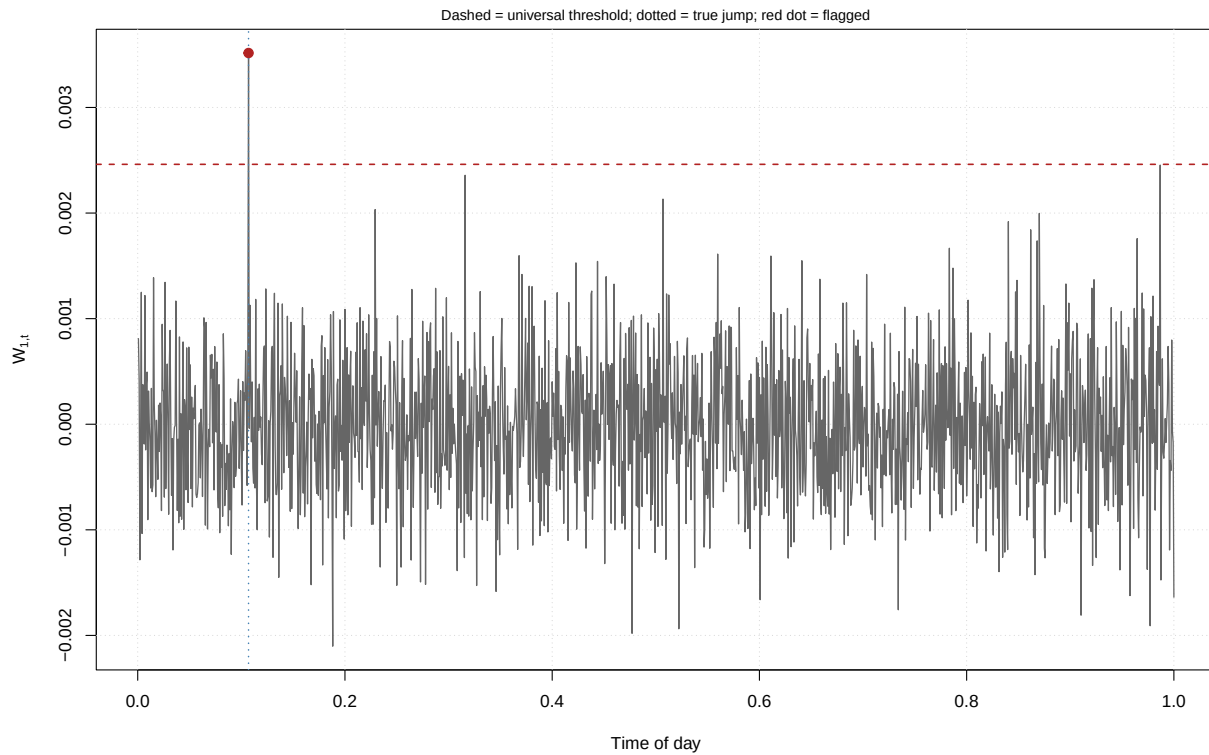


Figure 8.2: Wavelet coefficients of the finest scale on the simulated day from Figure 1. The jump produces a single coefficient far outside the noise band. We see the coefficient spikes well above the dashed threshold band at the exact point where the true jump actually happened

8.5 Jump size estimation and data adjustment

Given a detected location $\hat{\tau}$, the jump size is estimated by the difference of local averages just after and just before it,

$$\hat{L} = \bar{Y}_{\hat{\tau}+} - \bar{Y}_{\hat{\tau}-}, \quad \bar{Y}_{\hat{\tau}\pm} := \text{mean of } Y \text{ over a window of width } \delta_n \text{ just after/before } \hat{\tau},$$

with $\delta_n \sim n^{-1/2}$, implemented as a window of $\lfloor \sqrt{n}/4 \rfloor$ observations either side. The estimated jump is then subtracted from every subsequent observation to form the jump-adjusted series,

$$Y_{t_i}^* = Y_{t_i} - \sum_{\hat{\tau}_\ell \leq t_i} \hat{L}_\ell.$$

```
#### Jump size estimation + data adjustment
adjust_for_jumps <- function(Y, jump_idx, delta = NULL) {
```

```

n <- length(Y)
if (is.null(delta)) delta <- max(2, round(sqrt(n) / 4))

jump_idx <- sort(unique(jump_idx))
jump_sizes <- numeric(length(jump_idx))

Xd_hat <- numeric(n)
for (i in seq_along(jump_idx)) {
  tau <- jump_idx[i]
  lo_after <- min(n, tau + delta)
  hi_before <- max(1, tau - delta - 1)
  after <- Y[tau:lo_after]
  before <- Y[max(1, tau - delta):hi_before]
  if (length(before) == 0) before <- Y[max(1, tau - 1)]
  Lhat <- mean(after) - mean(before)
  jump_sizes[i] <- Lhat
  Xd_hat[tau:n] <- Xd_hat[tau:n] + Lhat
}
Ystar <- Y - Xd_hat
list(Ystar = Ystar, jump_idx = jump_idx, jump_sizes = jump_sizes,
     jump_variation = sum(jump_sizes^2))
}

adj <- adjust_for_jumps(day$Y, jd$jump_idx)

y_range <- range(c(day$Xc, day$Y, adj$Ystar), na.rm = TRUE)
y_limits <- c(y_range[1], y_range[2] + diff(y_range) * 0.15)

plot(tt, day$Xc, type = "n", ylim = y_limits,
     ylab = "Log-price", xlab = "Time of day")

# subtitle with dynamic jump sizes
subtitle_text <- sprintf("True jump size = %.4f, estimated = %.4f",
                        day$jump_sizes, adj$jump_sizes)
mtext(subtitle_text, side = 3, line = 0.25, cex = 0.85)

# Grid
grid(nx = NULL, ny = NULL, col = "lightgray", lty = "dotted")

# Observed noisy path
lines(tt, day$Y, col = "grey70", lwd = 1.2)

```

```
#True continuous path
lines(tt, day$Xc, col = "black", lwd = 1.2)

# Jump-adjusted path
lines(tt, adj$Ystar, col = "#c1272d", lwd = 1.2)

# 8. Legend
legend("top",
      legend = c(expression("True continuous" ~  $X[t]^c$ ),
                  expression("Observed" ~  $Y[t] \sim (\text{noise} + \text{jump})$ ),
                  expression("Jump-adjusted" ~  $Y[t]^*$ )),
      col = c("black", "grey70", "#c1272d"),
      lty = 1, lwd = 1.5, horiz = TRUE, bty = "n", cex = 1)
```

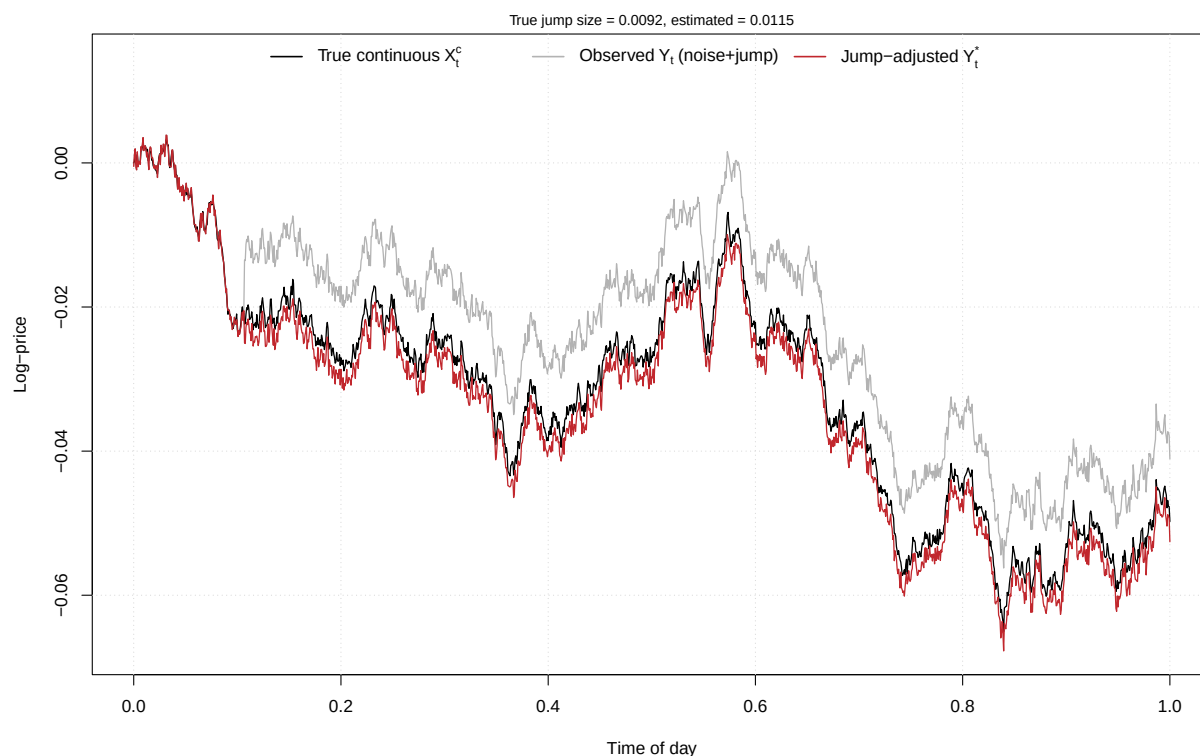


Figure 8.3: The jump-adjusted series (red) closely tracks the latent continuous path (black), while the raw observed series (grey) sits permanently offset after the jump.

8.6 Volatility Analysis and estimator implementation

Four families of estimators are implemented, directly translating the theoretical frameworks from Fan and Wang (2007, p.12–17) into code. Each of TSRV, MSRV, and WRV is computed

twice: once on the raw observed path Y to establish their naive baseline behaviour under jump contamination, and once on the jump-adjusted path Y^* to yield the robust JTSRV, JMSRV, and jump-adjusted WRV estimators.

1. **RV (Realized Volatility):** $\sum_i (Y_{t_i} - Y_{t_{i-1}})^2$. While consistent for integrated volatility under a frictionless, continuous model, this naive estimator breaks down when applied to noisy high-frequency data. The accumulated microstructure noise completely dominates the true variance, rendering it inconsistent.
2. **Average Subsampled Realized Volatility (ASRV) / Two-Scale Realized Volatility (TSRV):** ASRV mitigates noise by partitioning the data into K slower-sampled sub-grids: $[Y, Y]^{(K)} = \frac{1}{K} \sum_i (Y_{t_{i+K}} - Y_{t_i})^2$. The TSRV estimator applies a strict bias correction to this by subtracting the scaled finest-resolution variance: $\widehat{\Theta}_K = [Y, Y]^{(K)} - \frac{1}{K} [Y, Y]^{(1)}$. This subtraction explicitly cancels the microstructure noise variance bias.
3. **Multi-Scale Realized Volatility (MSRV):** To achieve an optimal $n^{-1/4}$ convergence rate, MSRV takes a variance-minimising weighted average of ASRVs over a grid of scales $K_m = m + C$, $m = 1, \dots, M$, where $C = \lfloor \sqrt{n} \rfloor$. The implementation below utilizes the exact analytical weights a_m and boundary-correction ζ defined by the authors.
4. **Wavelet Realized Volatility (WRV):** A multi-scale variance estimator that filters out microstructure noise by discarding the finest J_n resolution levels. The bias-corrected estimator is defined as:

$$\widehat{\Theta}_W = \text{WRV} - 2^{-J_n} [Y, Y]^{(1)}$$

Rather than calculating and summing the squared wavelet coefficients across all remaining coarse scales, the implementation exploits the discrete Parseval identity established in Section 5. Because the transform strictly reallocates energy without loss, the variance of all scales coarser than J_n (as well as the final residual) is perfectly encapsulated by the energy of the scaling coefficients at the exact cut-off level:

$$\sum_{j=J_n+1}^J \sum_t W_{j,t}^2 + \sum_t V_{J,t}^2 = \sum_t V_{J_n,t}^2$$

This algebraic equivalence allows the raw wavelet variance to be computed as $\text{WRV} = 2^{J_n} \sum_t V_{J_n,t}^2$. Fan and Wang establish that when the Haar wavelet is used, this estimator is algebraically identical to the TSRV estimator evaluated with a partition number of $K = 2^{J_n}$. The custom MODWT algorithm is deployed here to natively evaluate the undecimated data without requiring complex downsampling or shift-averaging logic.

```
RV <- function(y) sum(diff(y)^2)
```

```
ASRV <- function(y, K) {
```

```

n <- length(y)
diffs <- y[(K + 1):n] - y[1:(n - K)]
sum(diffs^2) / K
}

TSRV <- function(y, K) ASRV(y, K) - ASRV(y, 1) / K

MSRV <- function(y, M = 11) {
  n <- length(y)
  C <- floor(sqrt(n))
  Km <- (1:M) + C
  a_m <- 12 * Km * ((1:M) - M / 2 - 0.5) / (M * (M^2 - 1))
  zeta <- (M + C) * (C + 1) / ((n + 1) * (M - 1))
  asrv_vals <- sapply(Km, function(K) ASRV(y, K))
  sum(a_m * asrv_vals) + zeta * (ASRV(y, Km[1]) - ASRV(y, Km[M]))
}

WRV_modwt <- function(y, Jn) {
  r <- diff(y)
  out <- haar_modwt(r, Jn)
  K <- 2^Jn

  # scale by K to map the filter variance back to the log-price scale.
  wrv_val <- K * sum(out$V^2)

  # Bias correction
  wrv_val - (1 / K) * ASRV(y, 1)
}

```

8.6.1 Numerical Simulation to Test Estimator Consistency

The single-day example above serves as a diagnostic. However, to prove that these algorithms are robust estimators of the true latent continuous variance Θ , we must examine their asymptotic distributions under both frictions across a Monte Carlo design. To validate statistical consistency, we simulate $R = 200$ replications across a uniform grid of 21 microstructure-noise levels ($\eta \in [0, 0.001]$). We evaluate three distinct jump configurations (zero, one, and three jumps per day) while holding the underlying geometric Ornstein-Uhlenbeck volatility process fixed at the Fan and Wang calibration.

For each replication, we record the squared error of each estimator relative to the known integrated variance Θ . The errors are then averaged across the 200 replications to obtain the Mean Squared Error (MSE). Following the presentation convention of the original paper, all reported MSEs are scaled by 10^4 .

Seven estimators of Θ are evaluated, split into two categories. The unadjusted family (RV, TSRV at $K = 5$ and $K = 15$, and MSRV) estimates variance directly from the raw, contaminated path Y . The jump-adjusted family (JTSRV at $K = 15$, JMSRV, and WRV) estimates variance from the cleaned path Y^* after the wavelet thresholding has been applied. By Theorem 5 of Fan and Wang, WRV computed with the Haar wavelet at level $J_n = 7$ is algebraically identical to TSRV evaluated at $K = 2^7 = 128$. We exploit this equivalence inside the Monte Carlo loop to bypass rebuilding the full wavelet pyramid at every replication.

```
## Monte Carlo across a uniform grid of noise levels
```

```
eta_grid    <- seq(0, 0.001, length.out = 21)
jump_cases  <- c(0L, 1L, 3L)
R_mc        <- 200
MSE_SCALE   <- 1e4

est_labels  <- c(
  RV        = "RV",
  TSRV_5    = "TSRV (K = 5)",
  TSRV_15   = "TSRV (K = 15)",
  MSRV      = "MSRV",
  JTSRV_15  = "JTSRV (K = 15)",
  JMSRV     = "JMSRV",
  WRV       = "WRV (Jn = 7)"
)

mc_cell <- function(eta, nj, R = R_mc) {
  est_names <- names(est_labels)
  errs <- matrix(NA_real_, R, length(est_names),
    dimnames = list(NULL, est_names))
  for (r in seq_len(R)) {
    sim <- simulate_day(n = 1440, n_jumps = nj, eta = eta)
    jd  <- detect_jumps(sim$Y)
    adj <- adjust_for_jumps(sim$Y, jd$jump_idx)
    errs[r, ] <- c(
      RV        = RV(sim$Y),
      TSRV_5    = TSRV(sim$Y, 5L),
      TSRV_15   = TSRV(sim$Y, 15L),
      MSRV      = MSRV(sim$Y),
      JTSRV_15  = TSRV(adj$Ystar, 15L),
      JMSRV     = MSRV(adj$Ystar),
    )
  }
}
```

```

    WRV      = TSRV(adj$Ystar, 128L)
  ) - sim$theta_true
}
errs
}

set.seed(1409)
mc_res <- do.call(rbind, lapply(jump_cases, function(nj) {
  do.call(rbind, lapply(eta_grid, function(e) {
    m <- mc_cell(e, nj)
    data.frame(nj = nj, eta = e,
               estimator = factor(colnames(m), levels = names(est_labels)),
               mse = MSE_SCALE * colMeans(m^2))
  })))
}))

est_order <- names(est_labels)
est_cols <- c("grey60", "cornflowerblue", "darkorchid4", "tan4",
             "darkorange", "firebrick", "mediumseagreen")

# Use layout matrix instead of mfrow for robust rendering in Quarto
layout(matrix(1:3, nrow = 1))
par(mar = c(4.2, 4.6, 2, 0.6))

for (i in seq_along(jump_cases)) {
  nj <- jump_cases[i]
  sub <- mc_res[mc_res$nj == nj, ]
  ylo <- min(sub$mse[sub$mse > 0])
  yhi <- max(sub$mse)

  plot(NULL,
       xlim = range(eta_grid),
       ylim = log10(c(ylo, yhi)) + c(-0.15, 0.15),
       xlab = TeX("$\\eta$"),
       ylab = TeX("$\\log_{10}\\,(MSE\\times 10^4)$"),
       cex.lab = 1.0)

  for (e in est_order) {
    s <- sub[sub$estimator == e, ]
    cl <- est_cols[which(est_order == e)]
    lines(s$eta, log10(s$mse), col = cl, lwd = 2.5)
    points(s$eta, log10(s$mse), col = cl, pch = 1, cex = 0.65)
  }
}

```

```

mtext(sprintf("%d jump%s", nj, if (nj == 1) "" else "s"),
      side = 3, line = 0.4, cex = 0.95, font = 2)

if (i == 1) {
  legend("topleft",
        legend = est_labels[est_order],
        col     = est_cols,
        lwd     = 2.5, bty = "n", cex = 0.85, seg.len = 1.5)
}
}

```

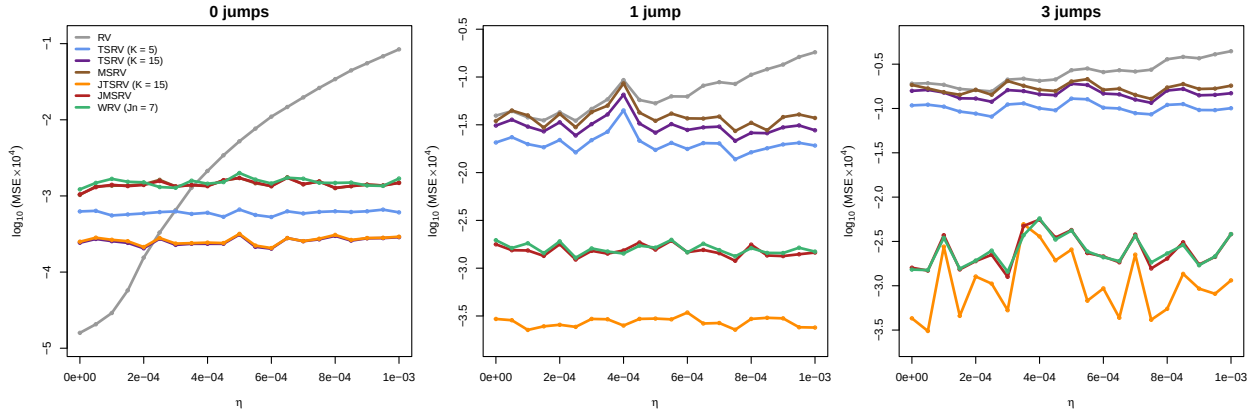


Figure 8.4: Monte Carlo MSE ($\times 10^4$) of each volatility estimator against the microstructure noise level, for zero (left), one (middle) and three (right) jumps. The vertical axis is logarithmic, meaning values approaching negative infinity indicate highly accurate, near-zero errors. RV and the unadjusted estimators are destroyed by jumps, while the jump-adjusted family isolates the latent integrated variance.

The 1x3 panel plot successfully replicates the behavioral distributions identified by Fan and Wang, demonstrating how the estimators rank against the true Θ . The vertical axis is plotted on a $\log_{10}$ scale; therefore, highly accurate estimators with an MSE of less than 1 will drop into the negative values, and trajectories approaching negative infinity represent mathematically superior estimates.

In the continuous, zero-jump case (left panel), the primary friction is purely observational noise. When η is near zero, the naive RV estimator performs optimally, producing the lowest error. However, it skyrockets as noise is introduced, scaling with the fourth power of η . Because there are no structural jumps to remove, the unadjusted TSRV ($K = 15$) and the jump-adjusted JTSRV ($K = 15$) process the exact same data and overlap fully at the very bottom of the plot as the most accurate estimators. The $K = 5$ partition of TSRV follows slightly above them, demonstrating that sampling frequency impacts accuracy. Sitting distinctly above these are JMSRV, WRV, and MSRV, remaining highly stable across the entire noise grid.

The introduction of structural discontinuities radically alters this hierarchy. In the one-jump scenario (middle panel), the unadjusted multi-scale estimators are visibly destroyed by the jump, forming clear vertical translations far above the truth: TSRV ($K = 5$) outperforms TSRV ($K = 15$), which outperforms MSRV. RV starts as the worst estimator overall but continues its upward trajectory rather than tracking smoothly with the other unadjusted variants. Conversely, the jump-adjusted family successfully decouples from the jump variance and drops to the bottom of the log-scale. JTSRV ($K = 15$) clearly establishes itself as the best estimator for Θ , followed distinctly by WRV and JMSRV, which closely track each other.

Under three jumps (right panel), the exact same ordering of the estimators remains intact. The four unadjusted estimators exhibit very consistent error trajectories despite increases in the noise level because the massive jump magnitude completely dominates the sample variance. While the three jump-adjusted estimators display more jagged trajectories as a consequence of isolating three simultaneous jumps across a smaller sample size of only 200 replications, they remain substantially superior.

```
eta_show <- 5e-4
sub      <- mc_res[mc_res$eta == eta_show, ]
tab_mat  <- matrix(NA_real_, nrow = length(est_labels),
                  ncol = length(jump_cases),
                  dimnames = list(unname(est_labels[names(est_labels)]),
                                sprintf("%d jump%s", jump_cases,
                                ifelse(jump_cases == 1, "", "s"))))
for (i in seq_along(est_labels)) {
  for (j in seq_along(jump_cases)) {
    tab_mat[i, j] <- sub$mse[sub$estimator == names(est_labels)[i] &
                             sub$nj       == jump_cases[j]]
  }
}
kable(as.data.frame(tab_mat), digits = 6)
```

Table 8.1: Monte Carlo MSE ($\times 10^4$) at the mid-grid noise level $\eta = 5.0 \times 10^{-4}$.

	0 jumps	1 jump	3 jumps
RV	0.005225	0.053067	0.270310
TSRV ($K = 5$)	0.000662	0.017265	0.129299
TSRV ($K = 15$)	0.000311	0.026144	0.190021
MSRV	0.001726	0.034807	0.201870
JTSRV ($K = 15$)	0.000317	0.000297	0.002559
JMSRV	0.001719	0.001564	0.004250
WRV ($J_n = 7$)	0.002000	0.001641	0.004191

Extracting the mid-grid noise level ($\eta = 5 \times 10^{-4}$) provides concrete confirmation of the visual output. Under one jump, the mean squared error of the unadjusted multi-scale estimators increases by two full orders of magnitude, a gap that widens to three orders of magnitude

under three jumps. Since the absolute scale of these errors remains small, their actual economic impact would require careful examination within a specific downstream pricing or hedging pipeline. However, there is clear quantitative evidence that the unadjusted estimators absorb structural breaks as a pronounced upward bias. The jump-adjusted variants avoid this, remaining reasonably stable across all configurations. While these promising results are currently contained within a strictly controlled simulation, they successfully validate the underlying mathematics. The logical next step for this pipeline would be to move beyond idealised Gaussian frameworks and stress-test these wavelet-thresholding estimators under realistic assumptions against live, unpredictable market data.

8.7 Appendix

8.7.1 Generating Correlated Sample Paths via Cholesky Decomposition

In order to accurately simulate the underlying dynamics of the asset and its stochastic volatility, it is necessary to introduce statistical dependence between their respective random shocks. The leverage effect dictates that asset returns and volatility increments do not evolve in isolation; rather, they are negatively correlated. We achieve this coupled evolution by applying a Cholesky decomposition to a vector of independent Gaussian increments.

Theorem a.1 (Cholesky Decomposition)

Let $\Sigma \in \mathbb{R}^{d \times d}$ be a symmetric positive-semidefinite matrix. Then there exists a unique lower-triangular matrix L with positive diagonal entries such that $\Sigma = LL^\top$.

Remark (Generating Correlated Gaussian Vectors)

Let $Z \sim \mathcal{N}(0, I_d)$ be a random vector with independent standard normal components. Defining $X = LZ$, we obtain $X \sim \mathcal{N}(0, \Sigma)$ since $\text{Cov}(X) = LI_dL^\top = LL^\top = \Sigma$. Hence, X is a jointly normal vector whose covariance structure is fully characterised by Σ ; that is, its components are marginally Gaussian and pairwise correlated according to Σ_{ij} (Mavuso, 2019).

Remark (Application to Correlated Shock Generation)

Let ε_t denote a random vector whose components are independent $\mathcal{N}(0, 1)$ variables. Applying the Cholesky factor L of the empirical covariance matrix Σ , we define $\varepsilon_t^{corr} = L\varepsilon_t$. Then $\varepsilon_t^{corr} \sim \mathcal{N}(0, \Sigma)$, and its components are therefore jointly normal with the same covariance structure as the observed market proxies. When multiple proxies are simulated together, each time step involves one random shock per asset. If these components were independent, each asset would evolve in isolation, producing unrealistic paths that ignore shared market behaviour. Transforming the independent Gaussian vector ensures that each simulated proxy inherits the appropriate degree of co-movement.

8.7.2 Application to the Geometric Ornstein-Uhlenbeck Volatility Model

In the simulation design of this project, we require exactly two correlated Brownian motions to drive the jump-diffusion price process and the Geometric Ornstein-Uhlenbeck (GOU) spot volatility process. Let W_t and $W_{1,t}$ denote the respective standard Brownian motions, with a specified correlation $\rho = -0.62$.

The target covariance matrix for the standard normal shocks over a time increment dt is:

$$\Sigma = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}$$

Applying Theorem A.1, the lower-triangular Cholesky factor L such that $\Sigma = LL^\top$ is trivially found to be:

$$L = \begin{pmatrix} 1 & 0 \\ \rho & \sqrt{1 - \rho^2} \end{pmatrix}$$

Let Z_1 and Z_2 be independent standard normal random variables drawn at each time step. By multiplying the Cholesky factor L with the independent vector $Z = (Z_1, Z_2)^\top$, we obtain the correlated standard normal shocks:

$$\begin{pmatrix} \Delta W_t \\ \Delta W_{1,t} \end{pmatrix} = \sqrt{dt} \begin{pmatrix} 1 & 0 \\ \rho & \sqrt{1 - \rho^2} \end{pmatrix} \begin{pmatrix} Z_1 \\ Z_2 \end{pmatrix}$$

This yields the explicit system of equations used in the computational simulation:

$$\begin{aligned} \Delta W_t &= \sqrt{dt} Z_1 \\ \Delta W_{1,t} &= \sqrt{dt} (\rho Z_1 + \sqrt{1 - \rho^2} Z_2) \end{aligned}$$

By construction, $\text{Var}(\Delta W_{1,t}) = dt(\rho^2 + 1 - \rho^2) = dt$, ensuring it remains a valid Brownian increment, while the covariance simplifies to $\mathbb{E}[(\sqrt{dt}Z_1)(\sqrt{dt}(\rho Z_1 + \sqrt{1 - \rho^2}Z_2))] = \rho dt$, successfully locking the correlation at the desired leverage parameter.

8.8 References

- Fan, J. Wang, Y. (2007). Multi-scale jump and volatility analysis for high-frequency financial data. *Journal of the American Statistical Association*, 102(480), 1349–1362.
- Mavuso, M. (2019), Order out of chaos i. Lecture notes, University of Cape Town.

- Percival, D.B. and Walden, A.T. 2000. *Wavelet methods for time series analysis*. Cambridge: Cambridge University Press

Chapter 9

Extension II: Interactive Construction of Tensor Product B-spline Bases

```
library(splines)
library(knitr)
```

This extension explores the construction of multidimensional splines, specifically focusing on how two one-dimensional B-spline bases combine to form a tensor product surface. While radial basis functions (such as thin-plate splines) offer isotropic smoothing in multiple dimensions, they assume that one unit of distance in X_1 is equivalent to one unit of distance in X_2 . When covariates operate on entirely different scales or units, a tensor product spline is the mathematically appropriate alternative, allowing for asymmetric flexibility across differing dimensions.

To formalise this setting according to the multidimensional spline framework, suppose there are two predictors $X \in \mathbb{R}^2$. Let $b_j(X_1)$ be a set of m_1 basis functions for the first predictor, and $h_k(X_2)$ be a set of m_2 basis functions for the second. The $m_1 \times m_2$ dimensional tensor product basis is defined by all pairwise products:

$$g_{jk}(X) = b_j(X_1) \otimes h_k(X_2) = b_j(X_1) h_k(X_2), \quad j = 1, \dots, m_1, \quad k = 1, \dots, m_2$$

A two-dimensional surface is then represented as a linear combination of these product functions: $g(X) = \sum_{j=1}^{m_1} \sum_{k=1}^{m_2} \theta_{jk} g_{jk}(X)$. Evaluated across n observations, the new basis matrix G becomes an $n \times (m_1 \times m_2)$ matrix, where each column is the element-wise multiplication of a column from the marginal X_1 basis and a column from the marginal X_2 basis.

Because the model remains strictly linear in the coefficients θ_{jk} , it can be estimated using ordinary linear model machinery. However, this structure exposes the exact mechanical source of the curse of dimensionality due to the total number of parameters growing multiplicatively.

To make this mechanism visible, this extension translates the matrix algebra of tensor products into an interactive Shiny tool, allowing for the real-time visualization of how the choice of degree and basis size in each margin reshapes the resulting multidimensional surface.

9.0.1 Algorithmic Implementation in Shiny

The interactive application bridges the mathematical template provided by the course slides with the reactive design principles of Wickham (2021). As expected the overall design is supposed to form a two a two-part system: a user interface that captures inputs, and a server that dictates the downstream computational flow. Rather than executing sequentially like a standard script, the server operates as a declarative reactive graph. It specifies precisely how outputs depend on inputs and updates only the necessary mathematical pathways when a user interacts with the tool.

This framework directly informs the implementation logic of the tensor product visualizer. In our logic, the core mathematical routines for basis construction and plotting are defined strictly outside the server environment. Then, the server acts solely to connect the user interface to these external functions. The four slider inputs feed into a reactive expression that checks mathematical constraints, ensuring that the number of basis functions strictly exceeds the polynomial degree.

The core algorithmic focus remains the assembly of the tensor product matrix G . The marginal bases are computed using standard uniform interior knots on $[0, 1]$. To map the continuous formulation into a discrete evaluation grid, the tensor product is assembled via a double loop iterating over the columns of the marginal bases.

```
# Evaluation grid on [0, 1] shared by both margins
ng <- 30
gg <- seq(0, 1, length.out = ng)

# B-spline basis with equally spaced interior knots and an intercept
marg_basis <- function(x, m, degree) {
  nk <- m - degree - 1
  kn <- seq(0, 1, length.out = nk + 2)[-c(1, nk + 2)]
  B <- bs(x, knots = kn, degree = degree, intercept = TRUE,
          Boundary.knots = c(0, 1))
  matrix(B, nrow = length(x), ncol = m)
}

# Row-wise tensor product: column (j, k) is B1[, j] * B2[, k]
tensor_basis <- function(B1, B2) {
  m1 <- ncol(B1)
  m2 <- ncol(B2)
  G <- matrix(NA, nrow = nrow(B1), ncol = m1 * m2)
  ccol <- 1
  for (j in 1:m1) {
```

```

for (k in 1:m2) {
  G[, ccol] <- B1[, j] * B2[, k]
  ccol <- ccol + 1
}
}
G
}

```

The full application is given in the appendix and is run locally with `shiny::runApp()`. The left column shows the two marginal bases, $b_j(x)$ above and $h_k(z)$ below, and the right column shows the tensor product basis as a grid of surfaces in which row j and column k contain $g_{jk}(x, z) = b_j(x) h_k(z)$. Moving a slider changes the marginal basis on the left and rebuilds the grid on the right at once, so the relationship between the two is direct: the row of the grid varies with the x basis and the column varies with the z basis. Three configurations were saved from the application and are discussed in turn.

9.0.1.1 Cubic margins with no interior knots

```
include_graphics("pracs/x2/figures/x1_cubic_4x4.png")
```

Tensor product of B-spline basis functions

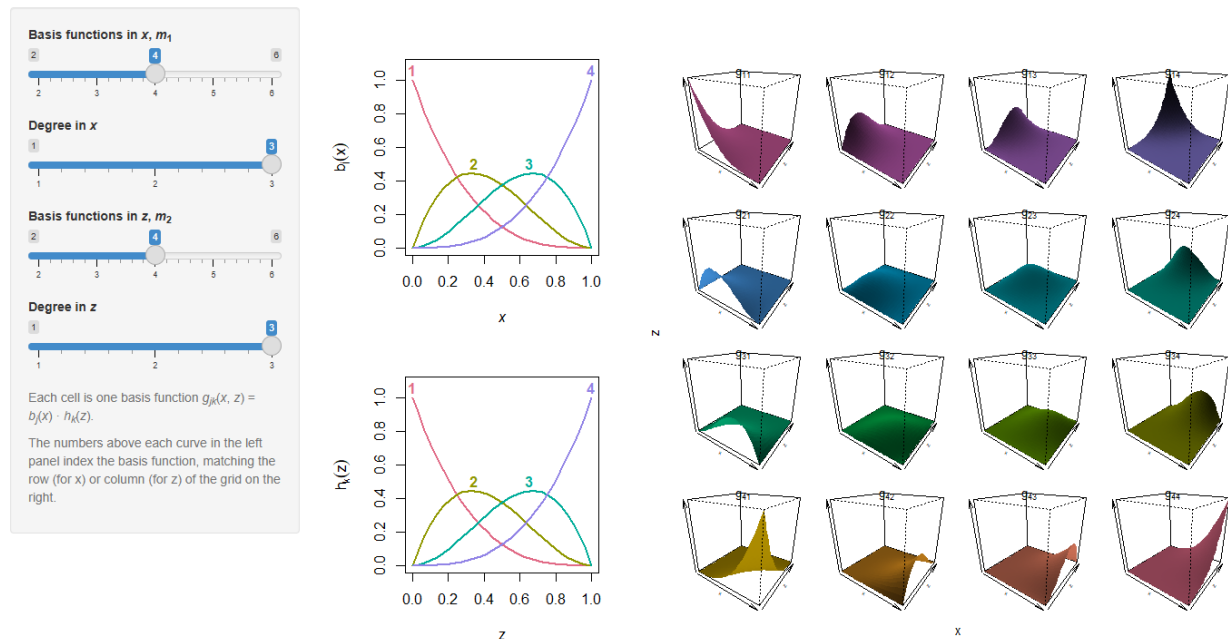


Figure 9.1: Application output with $m_1 = m_2 = 4$ and cubic degree in both margins. Rows of the grid index the basis function in x and columns index the basis function in z .

The number of interior knots K in a B-spline basis is governed by the structural relationship $K = m - (d + 1)$, where m is the number of basis functions and d is the polynomial degree.

With $m_1 = m_2 = 4$ basis functions and a cubic degree, the number of interior knots evaluates exactly to zero. Without interior boundaries to restrict their domain, every marginal function remains nonzero across the entire unit interval. Consequently, the resulting tensor product functions are smooth, broad surfaces that overlap heavily across the unit square.

9.0.1.2 Linear margins with interior knots

```
include_graphics("pracs/x2/figures/x1_linear_4x4.png")
```

Tensor product of B-spline basis functions

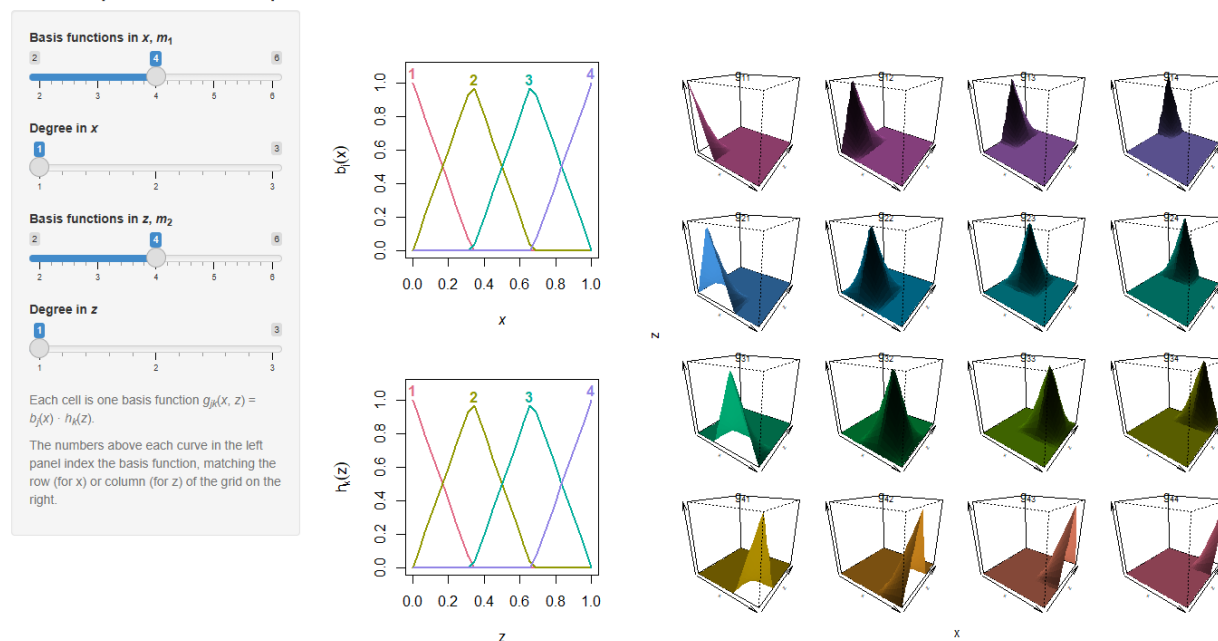


Figure 9.2: Application output with $m_1 = m_2 = 4$ and linear degree in both margins.

Holding the number of basis functions fixed at four and lowering the degree to one places two interior knots at $1/3$ and $2/3$, rendering each marginal function piecewise linear with compact support. The tensor product basis directly inherits this localized structure. Because the surfaces are formed by multiplying these piecewise linear margins, they exhibit sharp, visible kinks along the knot grid and remain strictly zero outside of their immediate rectangular neighborhoods.

9.0.1.3 Unequal margins

```
include_graphics("pracs/x2/figures/x1_unequal_5x3.png")
```

Tensor product of B-spline basis functions

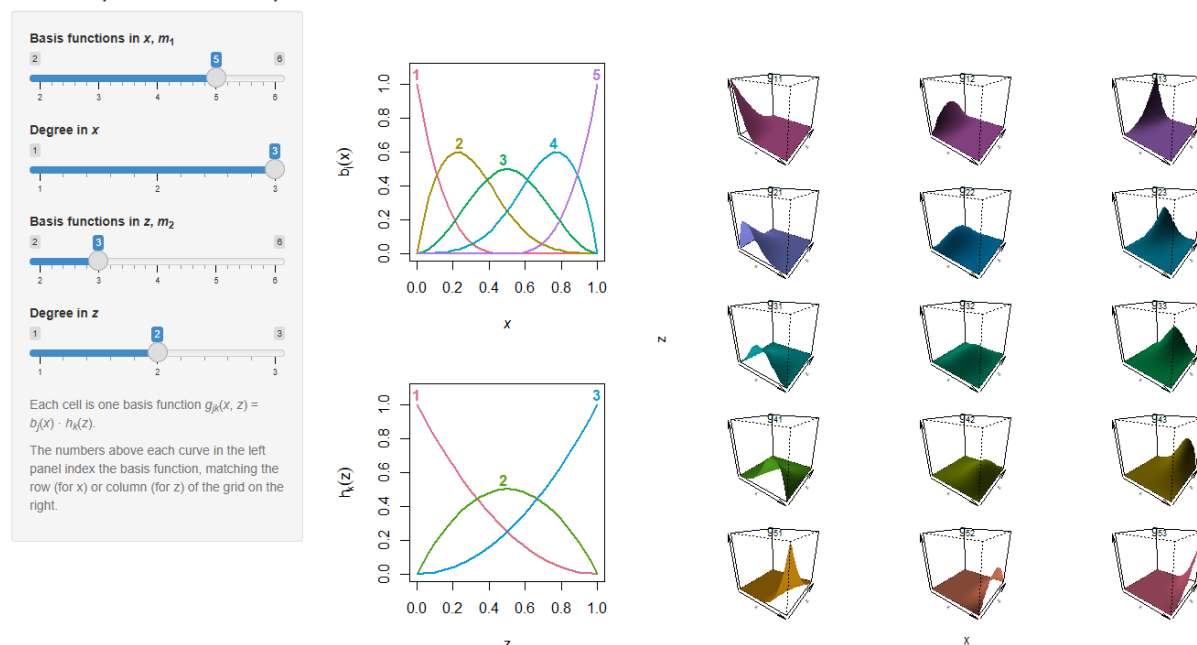


Figure 9.3: Application output with $m_1 = 5$ and cubic degree in x , and $m_2 = 3$ and quadratic degree in z .

The final configuration uses different sizes and degrees in the two margins, with five cubic functions in x and three quadratic functions in z , so that the grid of surfaces is rectangular with $5 \times 3 = 15$ panels. This is a situation that motivates the tensor product in practice, because the two predictors need not be measured on comparable scales nor require comparable flexibility. The construction accommodates this by allowing each margin its own basis. It also makes plain that the cost of the construction is the product of the margin sizes, with this being that adding a single function to one margin adds a whole row or column of surfaces, and therefore adds as many coefficients as there are functions in the other margin.

9.0.2 Appendix: Application Code

The following code is the complete, self-contained application. The application is intentionally narrow. It displays only the basis and not a fitted surface, so it shows what the model is built from but not how the coefficients θ_{jk} combine these surfaces into an estimate or how a penalty would regularise them; the knots are equally spaced on the unit square, which sidesteps the questions of knot placement and of scaling the two predictors. It is also limited to two dimensions, which is where the visual grid remains legible.

```
library(shiny)
library(splines)

# Evaluation grid on [0, 1] shared by both margins
```

```

ng <- 30
gg <- seq(0, 1, length.out = ng)

# B-spline basis
marg_basis <- function(x, m, degree) {
  nk <- m - degree - 1
  kn <- seq(0, 1, length.out = nk + 2)[-c(1, nk + 2)]
  B <- bs(x, knots = kn, degree = degree, intercept = TRUE,
          Boundary.knots = c(0, 1))
  matrix(B, nrow = length(x), ncol = m)
}

# Marginal B-splines; each curve tagged with its
# index at its peak so the reader can match a line to the corresp row/col
plot_marginals <- function(B1, B2, p) {
  op <- par(mfrow = c(2, 1), mar = c(4, 4.5, 3, 1))
  on.exit(par(op))

  col1 <- hcl.colors(p$m1, "Dark 3")
  matplot(gg, B1, type = "l", lty = 1, lwd = 2, ylim = c(0, 1.08),
          col = col1,
          xlab = expression(italic(x)), ylab = expression(b[j](x))
          )
  for (j in 1:p$m1) {
    imax <- which.max(B1[, j])
    text(gg[imax], B1[imax, j] + 0.06, labels = j,
         col = col1[j], cex = 1.0, font = 2)
  }

  col2 <- hcl.colors(p$m2, "Dark 3")
  matplot(gg, B2, type = "l", lty = 1, lwd = 2, ylim = c(0, 1.08),
          col = col2,
          xlab = expression(italic(z)), ylab = expression(h[k](z))
          )
  for (k in 1:p$m2) {
    imax <- which.max(B2[, k])
    text(gg[imax], B2[imax, k] + 0.06, labels = k,
         col = col2[k], cex = 1.0, font = 2)
  }
}

# one cell per tensor-product basis function
plot_tensor_persp <- function(m1, d1, m2, d2) {
  B1 <- marg_basis(gg, m1, d1)

```

```

B2 <- marg_basis(gg, m2, d2)

op <- par(mfrow = c(m1, m2),
          mar = c(0.3, 0.3, 1.4, 0.3),
          oma = c(2.5, 2.5, 3.8, 0.5))
on.exit(par(op))

cols <- hcl.colors(m1 * m2, "Dark 3", rev = TRUE)

idx <- 0
for (j in 1:m1) {
  for (k in 1:m2) {
    idx <- idx + 1
    Z <- outer(B1[, j], B2[, k]) #  $g_{jk}(x, z) = b_j(x) * h_k(z)$ 

    persp(gg, gg, Z,
          theta = 35, phi = 22,
          col = cols[idx],
          shade = 0.7,
          border = NA,
          ticktype = "simple", # ticks but no numeric labels
          cex.axis = 0.5,
          cex.lab = 0.55,
          xlab = "x", ylab = "z", zlab = "",
          zlim = c(0, 1),
          box = TRUE)

    mtext(bquote(g[.(j) * .(k)]),
          side = 3, line = -1.1, cex = 0.8, font = 2)
  }
}

mtext("x", side = 1, outer = TRUE, line = 1.0, cex = 0.9)
mtext("z", side = 2, outer = TRUE, line = 1.0, cex = 0.9)
}

ui <- fluidPage(
  titlePanel("Tensor product of B-spline basis functions"),

  sidebarLayout(
    sidebarPanel(
      width = 3,
      sliderInput("m1", HTML("Basis functions in  $x$ ,  $m_1$ "),
                  min = 2, max = 6, value = 4, step = 1),

```

```

    sliderInput("d1", HTML("Degree in <i>x</i>"),
                min = 1, max = 3, value = 3, step = 1),
    sliderInput("m2", HTML("Basis functions in <i>z</i>, <i>m</i><sub>2</sub>"),
                min = 2, max = 6, value = 4, step = 1),
    sliderInput("d2", HTML("Degree in <i>z</i>"),
                min = 1, max = 3, value = 3, step = 1),

    helpText("Each cell is one basis function ",
              HTML("<i>g<sub>jk</sub></i>(<i>x</i>, <i>z</i>) = ",
                    "<i>b<sub>j</sub></i>(<i>x</i>) \u00b7 <i>h<sub>k</sub></i>(<i>z</i>)",
    helpText("The numbers above each curve in the left panel index the ",
              "basis function, matching the row (for x) or column ",
              "(for z) of the grid on the right."),
),

mainPanel(
  width = 9,
  fluidRow(
    column(4, plotOutput("marg", height = "700px")),
    column(8, plotOutput("tp", height = "700px"))
  )
)
)
)

server <- function(input, output, session) {

  # Validate once; every downstream reactive inherits the message
  pars <- reactive({
    validate(need(input$m1 > input$d1 && input$m2 > input$d2,
                  "The number of basis functions must exceed the degree in each margin."),
    list(m1 = input$m1, d1 = input$d1, m2 = input$m2, d2 = input$d2)
  })

  B1 <- reactive(marg_basis(gg, pars()$m1, pars()$d1))
  B2 <- reactive(marg_basis(gg, pars()$m2, pars()$d2))

  output$marg <- renderPlot(plot_marginals(B1(), B2(), pars()), res = 96)

  output$tp <- renderPlot({
    p <- pars()
    plot_tensor_persp(p$m1, p$d1, p$m2, p$d2)
  }, res = 96)
}

```

```
shinyApp(ui, server)
```

9.0.3 References

Wickham, H. (2021). *Mastering Shiny*. O'Reilly Media. <https://mastering-shiny.org>

Part II

Section B

Chapter 10

Tutorial 1

10.1 Question 1

Consider the linear model $y = X\beta + \varepsilon$ where y is an N -dimensional vector, X is an $N \times (p+1)$ design matrix, and ε is an N -dimensional random vector with mean 0 and covariance matrix $\sigma^2 I$.

10.1.1 (a)

Show that minimising the residual sum of squares is equivalent to finding the residual vector $\hat{\varepsilon}$ such that $\hat{\varepsilon} \perp C(X)$.

Forward direction:

$$RSS(\beta) = (y - X\beta)'(y - X\beta)$$

$$\frac{\partial RSS}{\partial \beta} = -2X'(y - X\beta) = 0$$

At the minimiser $\hat{\beta}$:

$$X'(y - X\hat{\beta}) = 0$$

Let $\hat{\varepsilon} = y - X\hat{\beta}$:

$$\Rightarrow X'\hat{\varepsilon} = 0$$

$$\therefore \hat{\varepsilon} \perp C(X)$$

Converse direction:

For any candidate parameter β :

$$y - X\beta = (y - X\hat{\beta}) + X(\hat{\beta} - \beta) = \hat{\varepsilon} + X(\hat{\beta} - \beta)$$

$$RSS(\beta) = \|y - X\beta\|^2$$

$$= \|\hat{\varepsilon} + X(\hat{\beta} - \beta)\|^2$$

$$= \|\hat{\varepsilon}\|^2 + 2\hat{\varepsilon}'X(\hat{\beta} - \beta) + \|X(\hat{\beta} - \beta)\|^2$$

If $\hat{\varepsilon} \perp C(X) \implies \hat{\varepsilon}'X = 0$:

$$= \|\hat{\varepsilon}\|^2 + 0 + \|X(\hat{\beta} - \beta)\|^2$$

$$= \|\hat{\varepsilon}\|^2 + \|X(\hat{\beta} - \beta)\|^2$$

Since $\|X(\hat{\beta} - \beta)\|^2 \geq 0$:

$$RSS(\beta) \geq \|\hat{\varepsilon}\|^2$$

Equality holds iff $\beta = \hat{\beta}$.

$$\therefore \text{Minimising } RSS \iff \hat{\varepsilon} \perp C(X)$$

10.1.2 (b)

Show that $\frac{(N-p-1)\hat{\sigma}^2}{\sigma^2} \sim \chi_{N-p-1}^2$ where $\hat{\sigma}^2 = \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{N-p-1}$.

Let $H = X(X'X)^{-1}X'$, the projection matrix onto $C(X)$. The fitted residual vector is:

$$\hat{\varepsilon} = y - X\hat{\beta} = (I - H)y = (I - H)(X\beta + \varepsilon) = (I - H)\varepsilon$$

since $(I - H)X = 0$. The residual sum of squares expands as follows:

$$\hat{\varepsilon}'\hat{\varepsilon} = ((I - H)\varepsilon)'((I - H)\varepsilon)$$

$$\hat{\varepsilon}'\hat{\varepsilon} = \varepsilon'(I - H)'(I - H)\varepsilon$$

Since H is symmetric ($H' = H$), $(I - H)' = I - H$:

$$\hat{\varepsilon}'\hat{\varepsilon} = \varepsilon'(I - H)(I - H)\varepsilon$$

Since H is idempotent ($H^2 = H$), $(I - H)(I - H) = I - H$:

$$\hat{\varepsilon}'\hat{\varepsilon} = \varepsilon'(I - H)\varepsilon$$

For the symmetric idempotent matrix $I - H$, its rank r equals its trace. Using the cyclic property of the trace ($\text{tr}(ABC) = \text{tr}(CAB)$):

$$r = \text{rank}(I - H)$$

$$r = \text{tr}(I_N - H)$$

$$r = \text{tr}(I_N) - \text{tr}(X(X'X)^{-1}X')$$

$$r = N - \text{tr}(X'X(X'X)^{-1})$$

$$r = N - \text{tr}(I_{p+1})$$

$$r = N - (p + 1) = N - p - 1$$

Given $\varepsilon \sim N(0, \sigma^2 I)$, if $z \sim N(0, I)$ and M is symmetric idempotent of rank r , then $z'Mz \sim \chi_r^2$. Applying this with $z = \varepsilon/\sigma$ and $M = I - H$:

$$\frac{\varepsilon'(I - H)\varepsilon}{\sigma^2} = \frac{\hat{\varepsilon}'\hat{\varepsilon}}{\sigma^2} \sim \chi_{N-p-1}^2$$

Since $\hat{\varepsilon}'\hat{\varepsilon} = \sum_{i=1}^N (y_i - \hat{y}_i)^2 = (N - p - 1)\hat{\sigma}^2$, this gives $\frac{(N - p - 1)\hat{\sigma}^2}{\sigma^2} \sim \chi_{N-p-1}^2$.

10.1.3 (c)

Show by derivation how the mean squared error of the least squares estimator $\hat{\beta}$ can be decomposed into its two components, and comment on the relevance of this decomposition for model selection.

By definition, $MSE(\hat{\beta}) = E[(\hat{\beta} - \beta)(\hat{\beta} - \beta)']$.

Adding and subtracting $E[\hat{\beta}]$:

$$\hat{\beta} - \beta = (\hat{\beta} - E[\hat{\beta}]) + (E[\hat{\beta}] - \beta)$$

Expanding the quadratic form yields four terms:

$$\begin{aligned} (\hat{\beta} - \beta)(\hat{\beta} - \beta)' &= (\hat{\beta} - E[\hat{\beta}])(\hat{\beta} - E[\hat{\beta}])' + (E[\hat{\beta}] - \beta)(E[\hat{\beta}] - \beta)' \\ &\quad + (\hat{\beta} - E[\hat{\beta}])(E[\hat{\beta}] - \beta)' + (E[\hat{\beta}] - \beta)(\hat{\beta} - E[\hat{\beta}])' \end{aligned}$$

Apply the expectation operator. Since $E[\hat{\beta}] - \beta$ is a non-random constant vector and $E[\hat{\beta} - E[\hat{\beta}]] = 0$, the cross-terms vanish:

$$E[(\hat{\beta} - E[\hat{\beta}])(E[\hat{\beta}] - \beta)'] = E[\hat{\beta} - E[\hat{\beta}]](E[\hat{\beta}] - \beta)' = 0$$

$$E[(E[\hat{\beta}] - \beta)(\hat{\beta} - E[\hat{\beta}])'] = (E[\hat{\beta}] - \beta)E[\hat{\beta} - E[\hat{\beta}]]' = 0$$

The expectations of the remaining two terms are the definitions of variance and squared bias:

$$E[(\hat{\beta} - E[\hat{\beta}])(\hat{\beta} - E[\hat{\beta}])'] = \text{Var}(\hat{\beta})$$

$$E[(E[\hat{\beta}] - \beta)(E[\hat{\beta}] - \beta)'] = \text{Bias}(\hat{\beta})\text{Bias}(\hat{\beta})'$$

Substituting these back yields the matrix decomposition:

$$MSE(\hat{\beta}) = \text{Var}(\hat{\beta}) + \text{Bias}(\hat{\beta})\text{Bias}(\hat{\beta})'$$

To convert this to the scalar expected squared Euclidean distance, trace both sides using $\text{tr}(uv') = v'u = \|u\|^2$:

$$E\|\hat{\beta} - \beta\|^2 = \text{tr}(MSE(\hat{\beta}))$$

$$= \text{tr}(\text{Var}(\hat{\beta})) + \text{tr}(\text{Bias}(\hat{\beta})\text{Bias}(\hat{\beta})')$$

$$\therefore E\|\hat{\beta} - \beta\|^2 = \text{tr}(\text{Var}(\hat{\beta})) + \|\text{Bias}(\hat{\beta})\|^2$$

10.1.4 (d)

Consider fitting the linear model subject to the constraint $A\beta = C$, where A is $q \times (p+1)$ of rank $p+1$... actually of rank q , and C a vector of constants. Show that the constrained least squares estimator is

$$\hat{\beta}_C = \hat{\beta} - (X'X)^{-1}A'[A(X'X)^{-1}A']^{-1}(A\hat{\beta} - C),$$

where $\hat{\beta}$ is the unconstrained least squares estimator.

Lagrangian:

$$L(\beta, \lambda) = (y - X\beta)'(y - X\beta) + \lambda'(A\beta - C)$$

$$= y'y - y'X\beta - \beta'X'y + \beta'X'X\beta + \lambda'A\beta - \lambda'C$$

Since $y'X\beta$ is a scalar, it is equal to its transpose $\beta'X'y$.

$$\therefore L(\beta, \lambda) = y'y - 2\beta'X'y + \beta'X'X\beta + \lambda'A\beta - \lambda'C$$

$$\Rightarrow \frac{\partial L}{\partial \beta} = 0 - 2X'y + 2X'X\beta + A'\lambda = 0$$

$$2X'X\hat{\beta}_C = 2X'y - A'\lambda$$

$$\hat{\beta}_C = (X'X)^{-1}X'y - \frac{1}{2}(X'X)^{-1}A'\lambda$$

$$\therefore \hat{\beta}_C = \hat{\beta} - \frac{1}{2}(X'X)^{-1}A'\lambda$$

To find λ , impose the constraint $A\hat{\beta}_C = C$ by pre-multiplying the equation by A such that

$$A\hat{\beta}_C = A\hat{\beta} - \frac{1}{2}A(X'X)^{-1}A'\lambda$$

$$C = A\hat{\beta} - \frac{1}{2}A(X'X)^{-1}A'\lambda$$

we see

$$\frac{1}{2}A(X'X)^{-1}A'\lambda = A\hat{\beta} - C$$

$$\frac{1}{2}\lambda = [A(X'X)^{-1}A']^{-1}(A\hat{\beta} - C)$$

$$\lambda = 2[A(X'X)^{-1}A']^{-1}(A\hat{\beta} - C)$$

hence

$$\hat{\beta}_C = \hat{\beta} - \frac{1}{2}(X'X)^{-1}A' \left(2[A(X'X)^{-1}A']^{-1}(A\hat{\beta} - C) \right)$$

$$\therefore \hat{\beta}_C = \hat{\beta} - (X'X)^{-1}A'[A(X'X)^{-1}A']^{-1}(A\hat{\beta} - C)$$

10.1.5 (e)

Show that the difference between the residual sum of squares for the constrained and unconstrained models is

$$RSS_C - RSS = (A\hat{\beta} - C)'[A(X'X)^{-1}A']^{-1}(A\hat{\beta} - C)$$

By definition,

$$RSS_C = (y - X\hat{\beta}_C)'(y - X\hat{\beta}_C)$$

Substitute $\hat{\beta}_C = \hat{\beta} - (X'X)^{-1}A'[A(X'X)^{-1}A']^{-1}(A\hat{\beta} - C)$.

Let $W = (X'X)^{-1}A'[A(X'X)^{-1}A']^{-1}(A\hat{\beta} - C)$ such that $\hat{\beta}_C = \hat{\beta} - W$.

$$RSS_C = (y - X(\hat{\beta} - W))'(y - X(\hat{\beta} - W))$$

$$RSS_C = ((y - X\hat{\beta}) + XW)'((y - X\hat{\beta}) + XW)$$

Let $\hat{\varepsilon} = y - X\hat{\beta}$.

$$RSS_C = (\hat{\varepsilon} + XW)'(\hat{\varepsilon} + XW)$$

$$RSS_C = \hat{\varepsilon}'\hat{\varepsilon} + \hat{\varepsilon}'XW + W'X'\hat{\varepsilon} + W'X'XW$$

Note $X'\hat{\varepsilon} = X'(y - X\hat{\beta}) = X'y - X'X(X'X)^{-1}X'y = X'y - X'y = 0$.

$$\therefore \hat{\varepsilon}'X = 0 \text{ and } X'\hat{\varepsilon} = 0$$

$$RSS_C = \hat{\varepsilon}'\hat{\varepsilon} + W'X'XW$$

Since $RSS = \hat{\varepsilon}'\hat{\varepsilon}$:

$$\therefore RSS_C - RSS = W'X'XW$$

Substitute W :

$$W'X'XW = \left((X'X)^{-1}A'[A(X'X)^{-1}A']^{-1}(A\hat{\beta} - C) \right)' X'X \left((X'X)^{-1}A'[A(X'X)^{-1}A']^{-1}(A\hat{\beta} - C) \right)$$

Note $(ABC)' = C'B'A'$ and $[A(X'X)^{-1}A']^{-1}$ is symmetric.

$$W' = (A\hat{\beta} - C)'[A(X'X)^{-1}A']^{-1}A(X'X)^{-1}$$

$$W'X'XW = (A\hat{\beta} - C)'[A(X'X)^{-1}A']^{-1}A(X'X)^{-1}X'X(X'X)^{-1}A'[A(X'X)^{-1}A']^{-1}(A\hat{\beta} - C)$$

$$W'X'XW = (A\hat{\beta} - C)'[A(X'X)^{-1}A']^{-1}(A(X'X)^{-1}A')[A(X'X)^{-1}A']^{-1}(A\hat{\beta} - C)$$

$$\therefore W'X'XW = (A\hat{\beta} - C)'[A(X'X)^{-1}A']^{-1}(A\hat{\beta} - C)$$

$$\therefore RSS_C - RSS = (A\hat{\beta} - C)'[A(X'X)^{-1}A']^{-1}(A\hat{\beta} - C)$$

10.1.6 (f)

Hence show that $F = \frac{(RSS_C - RSS)/q}{RSS/(N - p - 1)} \sim F_{q, N-p-1}$ under $H_0 : A\beta = C$.

Under $H_0 : A\beta = C$, we can substitute C with $A\beta$:

$$A\hat{\beta} - C = A\hat{\beta} - A\beta = A(\hat{\beta} - \beta)$$

Given the unconstrained estimator distribution $\hat{\beta} \sim N(\beta, \sigma^2(X'X)^{-1})$, the expected value is:

$$E[A\hat{\beta} - C] = A(E[\hat{\beta}] - \beta) = A(\beta - \beta) = 0$$

The variance is transformed via $\text{Var}(Az) = A\text{Var}(z)A'$:

$$\text{Var}(A\hat{\beta} - C) = \text{Var}(A\hat{\beta}) = A\text{Var}(\hat{\beta})A' = A(\sigma^2(X'X)^{-1})A' = \sigma^2 A(X'X)^{-1}A'$$

$$\therefore A\hat{\beta} - C \sim N(0, \sigma^2 A(X'X)^{-1}A')$$

For a multivariate normal vector $z \sim N(0, \Sigma)$, the quadratic form standardising z by its covariance matrix follows $z'\Sigma^{-1}z \sim \chi^2_{\text{rank}(\Sigma)}$. Let $z = A\hat{\beta} - C$ and $\Sigma = \sigma^2 A(X'X)^{-1}A'$:

$$z'\Sigma^{-1}z = (A\hat{\beta} - C)'[\sigma^2 A(X'X)^{-1}A']^{-1}(A\hat{\beta} - C)$$

$$z'\Sigma^{-1}z = \frac{1}{\sigma^2}(A\hat{\beta} - C)'[A(X'X)^{-1}A']^{-1}(A\hat{\beta} - C)$$

Substitute the result from part (e) for the numerator:

$$\frac{RSS_C - RSS}{\sigma^2} \sim \chi^2_q$$

using $q = \text{rank}(A)$.

From part (b), $RSS/\sigma^2 \sim \chi^2_{N-p-1}$. To prove that F follows an F-distribution, the numerator and denominator chi-square variables must be independent. We show this by proving $\hat{\beta}$ and $\hat{\varepsilon}$ are independent. For jointly normal variables, zero covariance implies independence. i.e.

$$\text{Cov}(\hat{\beta}, \hat{\varepsilon}) = \text{Cov}((X'X)^{-1}X'y, (I - H)y)$$

Using $\text{Cov}(Ay, By) = A\text{Var}(y)B'$ and noting $I - H$ is symmetric,

$$\text{Cov}(\hat{\beta}, \hat{\varepsilon}) = (X'X)^{-1}X'\text{Var}(y)(I - H)$$

$$\text{Cov}(\hat{\beta}, \hat{\varepsilon}) = (X'X)^{-1}X'(\sigma^2 I)(I - H) = \sigma^2(X'X)^{-1}X'(I - H)$$

Expanding $X'(I - H)$

$$X'(I - H) = X' - X'H = X' - X'X(X'X)^{-1}X' = X' - IX' = X' - X' = 0$$

$$\therefore \text{Cov}(\hat{\beta}, \hat{\varepsilon}) = \sigma^2(X'X)^{-1}(0) = 0$$

Since $\hat{\beta}$ and $\hat{\varepsilon}$ are independent, $RSS_C - RSS$ (a function of $\hat{\beta}$) and RSS (a function of $\hat{\varepsilon}$) are independent chi-square variables.

The ratio of two independent chi-square variables, each divided by its degrees of freedom, is F-distributed. The σ^2 terms cancel out, giving

$$F = \frac{\frac{RSS_C - RSS}{\sigma^2}/q}{\frac{RSS}{\sigma^2}/(N - p - 1)}$$

$$\therefore F = \frac{(RSS_C - RSS)/q}{RSS/(N - p - 1)} \sim F_{q, N-p-1}$$

10.2 Question 2

Consider Model 1, $Y = \beta_0 \mathbf{1} + \varepsilon$, and Model 2, $Y = X\beta + \varepsilon$, with $\varepsilon \sim N(0, \sigma^2 I)$ and $\mathbf{1}$ an n -vector of ones. Let $p = k - 1$, let $\hat{\beta}$ be the OLS estimate for Model 2, and let $\bar{y}$ be the sample mean of Y .

10.2.1 (a)

Show that $\hat{\beta}_0 = \bar{y}$.

For Model 1, $Y = \beta_0 \mathbf{1} + \varepsilon$. Using $\hat{\beta} = (X'X)^{-1}X'Y$ with $X = \mathbf{1}$:

$$\hat{\beta}_0 = (\mathbf{1}'\mathbf{1})^{-1}\mathbf{1}'Y$$

$$\mathbf{1}'\mathbf{1} = \sum_{i=1}^n 1 = n$$

$$\mathbf{1}'Y = \sum_{i=1}^n Y_i$$

$$\Rightarrow \hat{\beta}_0 = n^{-1} \sum_{i=1}^n Y_i = \bar{y}$$

Alternatively, minimize $S = \sum_{i=1}^n (Y_i - \beta_0)^2$:

$$\frac{\partial S}{\partial \beta_0} = -2 \sum_{i=1}^n (Y_i - \beta_0) = 0$$

$$\Rightarrow \sum_{i=1}^n Y_i - \sum_{i=1}^n \hat{\beta}_0 = 0$$

$$\sum_{i=1}^n Y_i - n\hat{\beta}_0 = 0$$

$$n\hat{\beta}_0 = \sum_{i=1}^n Y_i$$

$$\therefore \hat{\beta}_0 = \frac{1}{n} \sum_{i=1}^n Y_i = \bar{y}$$

10.2.2 (b)

For Model 1, $Y = \beta_0 \mathbf{1} + \varepsilon$ with $\hat{\beta}_0 = \bar{y}$.

$$\hat{\varepsilon}_1 = Y - \bar{y}\mathbf{1}$$

$$SSE_1 = \hat{\varepsilon}_1^T \hat{\varepsilon}_1 = \sum_{i=1}^n (Y_i - \bar{y})^2$$

Model 1 has 1 parameter, so $df_1 = n - 1$.

$$\therefore \hat{\sigma}_1^2 = \frac{SSE_1}{n-1} = \frac{1}{n-1} \sum_{i=1}^n (Y_i - \bar{y})^2$$

10.2.3 (c)

Show that the residual sum of squares for Model 2 is $SSE_2 = Y'Y - \hat{\beta}'X'Y$.

For Model 2, $Y = X\beta + \varepsilon$.

$$SSE_2 = (Y - X\hat{\beta})^T(Y - X\hat{\beta})$$

$$SSE_2 = Y^TY - Y^TX\hat{\beta} - \hat{\beta}^TX^TY + \hat{\beta}^TX^TX\hat{\beta}$$

Note $Y^TX\hat{\beta} = (Y^TX\hat{\beta})^T = \hat{\beta}^TX^TY$.

Using the normal equations $X^TX\hat{\beta} = X^TY$:

$$SSE_2 = Y^TY - 2\hat{\beta}^TX^TY + \hat{\beta}^T(X^TY)$$

$$\therefore SSE_2 = Y^TY - \hat{\beta}^TX^TY$$

10.2.4 (d)

Provide the null and the alternate hypotheses in order to assess whether a significant regression model exists.

$$H_0 : \beta_1 = \beta_2 = \dots = \beta_p = 0 \quad \text{vs} \quad H_a : \beta_j \neq 0 \text{ for at least one } j \in \{1, \dots, p\}.$$

10.2.5 (e)

Use your above answers to show that the required test statistic when undertaking the above hypothesis test is $F = \frac{\hat{\beta}^TX^TY - n\bar{y}^2}{pS_2^2}$.

Using the general F-test for nested models:

$$F = \frac{(SSE_1 - SSE_2)/(df_1 - df_2)}{SSE_2/df_2}$$

Expand SSE_1 :

$$SSE_1 = \sum_{i=1}^n (Y_i - \bar{y})^2 = \sum_{i=1}^n Y_i^2 - 2\bar{y} \sum_{i=1}^n Y_i + n\bar{y}^2$$

$$SSE_1 = Y^TY - 2\bar{y}(n\bar{y}) + n\bar{y}^2 = Y^TY - n\bar{y}^2$$

Difference in sum of squares:

$$SSE_1 - SSE_2 = (Y^T Y - n\bar{y}^2) - (Y^T Y - \hat{\beta}^T X^T Y)$$

$$SSE_1 - SSE_2 = \hat{\beta}^T X^T Y - n\bar{y}^2$$

Degrees of freedom: Model 2 has $p + 1$ parameters, so $df_2 = n - p - 1$.

$$df_1 - df_2 = (n - 1) - (n - p - 1) = p$$

By definition, the estimated residual variance of Model 2 is $S_2^2 = \frac{SSE_2}{df_2}$.

$$\therefore F = \frac{(\hat{\beta}^T X^T Y - n\bar{y}^2)/p}{S_2^2}$$

10.2.6 (f)

where S_2^2 is the estimated residual variance of Model 2. Explain your reasoning. State any assumptions made where needed.

$S_2^2 = SSE_2/(n - p - 1)$. The reasoning is that $\hat{\beta}^T X^T Y - n\bar{y}^2$ measures how much of the total variability in Y is explained by the predictors beyond the intercept, and comparing this, scaled by its degrees of freedom p , against the residual variance S_2^2 tests whether that explained variability is large relative to the background noise level. This mirrors the general constrained versus unconstrained argument of Question 1, with Model 1 playing the role of the restricted model under $A\beta = C$. Assumptions made: Normal, homoscedastic, independent errors i.e. $\varepsilon \sim N(0, \sigma^2 I)$ and that X is of full column rank.

10.2.7 (g)

By the same argument as Question 1(f), applied with $q = p$ restrictions and $N - p - 1$ residual degrees of freedom, $F \sim F_{p, n-p-1}$ under H_0 .

10.3 Question 3

Find an orthonormal basis for $C(X)$, the column space of

$$X = \begin{pmatrix} 2 & -5 & 3 & 1 \\ 0 & 4 & -3 & -1 \\ -2 & -2 & 3 & 0 \end{pmatrix}.$$

Let $x_1 = (2, 0, -2)'$, $x_2 = (-5, 4, -2)'$, $x_3 = (3, -3, 3)'$, $x_4 = (1, -1, 0)'$.

$$\det[x_1 \ x_2 \ x_3] = 2(12 - 6) + 5(0 - 6) + 3(0 + 8) = 12 - 30 + 24 = 6$$

$$\det[x_1 \ x_2 \ x_3] \neq 0 \implies x_1, x_2, x_3 \text{ are linearly independent}$$

$$\therefore C(X) = \mathbb{R}^3 \text{ and } x_4 \text{ is redundant}$$

Applying Gram-Schmidt to x_1, x_2, x_3 :

$$u_1 = x_1 = (2, 0, -2)' \implies \|u_1\|^2 = 4 + 0 + 4 = 8$$

$$x'_2 u_1 = -10 + 0 + 4 = -6$$

$$u_2 = x_2 - \frac{x'_2 u_1}{u'_1 u_1} u_1 = (-5, 4, -2)' - \frac{-6}{8} (2, 0, -2)' = \left(-\frac{7}{2}, 4, -\frac{7}{2}\right)'$$

$$\implies \|u_2\|^2 = \frac{49}{4} + 16 + \frac{49}{4} = \frac{81}{2}$$

$$\text{Note } x'_3 u_1 = 6 + 0 - 6 = 0 \implies x_3 \perp u_1.$$

$$x'_3 u_2 = -\frac{21}{2} - 12 - \frac{21}{2} = -33$$

$$u_3 = x_3 - \frac{x'_3 u_2}{u'_2 u_2} u_2 = (3, -3, 3)' - \frac{-33}{81/2} \left(-\frac{7}{2}, 4, -\frac{7}{2}\right)' = \left(\frac{4}{27}, \frac{7}{27}, \frac{4}{27}\right)'$$

$$\implies \|u_3\|^2 = \frac{16}{729} + \frac{49}{729} + \frac{16}{729} = \frac{81}{729} = \frac{1}{9}$$

Normalising each $u_i \implies e_i = \frac{u_i}{\|u_i\|}$:

$$e_1 = \frac{1}{\sqrt{8}} (2, 0, -2)' = \frac{1}{\sqrt{2}} (1, 0, -1)'$$

$$e_2 = \frac{1}{\sqrt{81/2}} \left(-\frac{7}{2}, 4, -\frac{7}{2}\right)' = \frac{\sqrt{2}}{18} (-7, 8, -7)'$$

$$e_3 = \frac{1}{\sqrt{1/9}} \left(\frac{4}{27}, \frac{7}{27}, \frac{4}{27} \right)' = \frac{1}{9} (4, 7, 4)'$$

$\therefore \{e_1, e_2, e_3\}$ is an orthonormal basis for $C(X)$.

Chapter 11

Tutorial 2

11.1 Question 1

11.1.1 (a)

Model: $y = X\beta + \varepsilon$

y : $N \times 1$ response vector, X : $N \times (p + 1)$ design matrix, β : $(p + 1) \times 1$ coefficient vector, ε : $N \times 1$ error vector with $E[\varepsilon] = 0$ and $\text{Var}(\varepsilon) = \sigma^2 I$

Minimise the penalised residual sum of squares for $\lambda \geq 0$:

$$RSS_{ridge}(\beta) = (y - X\beta)'(y - X\beta) + \lambda\beta'\beta$$

$$RSS_{ridge}(\beta) = y'y - 2\beta'X'y + \beta'X'X\beta + \lambda\beta'\beta$$

$$\frac{\partial RSS_{ridge}}{\partial \beta} = -2X'y + 2X'X\beta + 2\lambda\beta = 0$$

$$\implies X'X\beta + \lambda\beta = X'y$$

$$\implies (X'X + \lambda I)\beta = X'y$$

$$\therefore \hat{\beta}^{ridge} = (X'X + \lambda I)^{-1}X'y$$

11.1.2 (b)

Show that $\hat{\beta}^{ridge}$ is a biased estimator when $\lambda > 0$.

$$\begin{aligned} E[\hat{\beta}^{ridge}] &= E[(X'X + \lambda I)^{-1} X' y] \\ &= (X'X + \lambda I)^{-1} X' E[y] \end{aligned}$$

Given $E[y] = X\beta$:

$$= (X'X + \lambda I)^{-1} X' X \beta$$

For $\lambda > 0$, $(X'X + \lambda I)^{-1} X' X \neq I \implies E[\hat{\beta}^{ridge}] \neq \beta$.

By definition, $\text{Bias}(\hat{\beta}^{ridge}) = E[\hat{\beta}^{ridge}] - \beta$:

$$\begin{aligned} \text{Bias}(\hat{\beta}^{ridge}) &= (X'X + \lambda I)^{-1} X' X \beta - \beta \\ &= ((X'X + \lambda I)^{-1} X' X - I) \beta \end{aligned}$$

Substitute $I = (X'X + \lambda I)^{-1} (X'X + \lambda I)$:

$$\begin{aligned} &= ((X'X + \lambda I)^{-1} X' X - (X'X + \lambda I)^{-1} (X'X + \lambda I)) \beta \\ &= (X'X + \lambda I)^{-1} (X'X - X'X - \lambda I) \beta \\ &= (X'X + \lambda I)^{-1} (-\lambda I) \beta \end{aligned}$$

$$\therefore \text{Bias}(\hat{\beta}^{ridge}) = -\lambda (X'X + \lambda I)^{-1} \beta$$

For $\lambda > 0$ and $\beta \neq 0$, $\text{Bias}(\hat{\beta}^{ridge}) \neq 0 \implies \hat{\beta}^{ridge}$ is biased.

11.1.3 (c)

Find $\text{Var}[\hat{\beta}^{ridge}]$, the variance of the ridge regression estimator.

$$\text{Var}(\hat{\beta}^{ridge}) = \text{Var}((X'X + \lambda I)^{-1} X' y)$$

Using $\text{Var}(Ay) = A\text{Var}(y)A'$:

$$= (X'X + \lambda I)^{-1} X' \text{Var}(y) ((X'X + \lambda I)^{-1} X')'$$

$$= (X'X + \lambda I)^{-1} X' \text{Var}(y) X (X'X + \lambda I)^{-1}$$

Given $\text{Var}(y) = \sigma^2 I$:

$$= (X'X + \lambda I)^{-1} X' (\sigma^2 I) X (X'X + \lambda I)^{-1}$$

$$\therefore \text{Var}(\hat{\beta}^{ridge}) = \sigma^2 (X'X + \lambda I)^{-1} X' X (X'X + \lambda I)^{-1}$$

11.1.4 (d)

As λ increases from zero, the ridge estimator moves further away from the unbiased OLS solution, so its bias grows, while the penalty stabilises the estimator by pulling the coefficients toward zero, so its variance shrinks. At $\lambda = 0$ ridge reduces exactly to OLS, which is unbiased but can have large variance when the columns of X are highly collinear. As $\lambda \rightarrow \infty$ the coefficients are shrunk all the way to zero, giving maximal bias and minimal variance. Because of this trade off, choosing an intermediate value of λ can give a smaller overall mean squared error than the unbiased OLS estimator, even though the resulting estimator is biased.

11.2 Question 2

11.2.1 (c)

How does the ridge regression estimator compare to the ordinary least squares estimator in the following cases?

i. $\lambda \rightarrow 0$

$$\hat{\beta}^{ridge} = (X'X + \lambda I)^{-1} X'y$$

As $\lambda \rightarrow 0$:

$$(X'X + \lambda I)^{-1} \rightarrow (X'X)^{-1}$$

$$\Rightarrow \hat{\beta}^{ridge} \rightarrow (X'X)^{-1} X'y$$

$$\therefore \hat{\beta}^{ridge} \rightarrow \hat{\beta}^{ls}$$

ii. The columns of the input matrix are orthonormal.

$$X'X = I$$

$$\Rightarrow \hat{\beta}^{ls} = (I)^{-1}X'y = X'y$$

$$\hat{\beta}^{ridge} = (I + \lambda I)^{-1}X'y$$

$$= ((1 + \lambda)I)^{-1}X'y$$

$$= \frac{1}{1 + \lambda}X'y$$

$$\therefore \hat{\beta}^{ridge} = \frac{1}{1 + \lambda}\hat{\beta}^{ls}$$

11.2.2 (d)

In ridge regression estimation, what happens when $\lambda \rightarrow \infty$?

$$\hat{\beta}^{ridge} = (X'X + \lambda I)^{-1}X'y$$

As $\lambda \rightarrow \infty$, the term λI dominates $X'X$:

$$(X'X + \lambda I)^{-1} \rightarrow (\lambda I)^{-1} = \frac{1}{\lambda}I$$

As $\frac{1}{\lambda}I \rightarrow 0$:

$$\Rightarrow \hat{\beta}^{ridge} \rightarrow 0$$

$\therefore$ Every coefficient is shrunk fully to zero, and the fitted model collapses to having no predictors.

11.3 Question 3

11.3.1 (a)

Use the Gram-Schmidt procedure to find an orthogonal basis z_1, z_2, z_3 for the given vector space. Let x_1, x_2, x_3 be a non-orthogonal basis.

Apply the Gram-Schmidt procedure:

$$z_1 = x_1$$

$$z_2 = x_2 - \frac{z'_1 x_2}{z'_1 z_1} z_1$$

$$z_3 = x_3 - \frac{z'_1 x_3}{z'_1 z_1} z_1 - \frac{z'_2 x_3}{z'_2 z_2} z_2$$

Check orthogonality ($z'_i z_j = 0$ for $i \neq j$):

$$\begin{aligned} z'_2 z_1 &= \left(x_2 - \frac{z'_1 x_2}{z'_1 z_1} z_1 \right)' z_1 \\ &= x'_2 z_1 - \frac{z'_1 x_2}{z'_1 z_1} z'_1 z_1 \\ &= x'_2 z_1 - x'_2 z_1 = 0 \\ &\implies z_2 \perp z_1 \end{aligned}$$

By the exact same expansion for z_3 :

$$\begin{aligned} z'_3 z_1 &= x'_3 z_1 - \frac{z'_1 x_3}{z'_1 z_1} z'_1 z_1 - 0 = x'_3 z_1 - x'_3 z_1 = 0 \\ z'_3 z_2 &= x'_3 z_2 - 0 - \frac{z'_2 x_3}{z'_2 z_2} z'_2 z_2 = x'_3 z_2 - x'_3 z_2 = 0 \\ &\implies z_3 \perp z_1 \text{ and } z_3 \perp z_2 \end{aligned}$$

Since each z_i is formed by adding or subtracting a linear combination of vectors already in the span:

$$\text{span}(z_1, z_2, z_3) = \text{span}(x_1, x_2, x_3)$$

$\therefore \{z_1, z_2, z_3\}$ is an orthogonal basis.

11.3.2 (b)

Use your answer in part (a) to show that the matrix X with columns x_1, x_2, x_3 , can be written as $X = QR$, where R is an upper triangular matrix, and the columns of Q are orthonormal.

Rearrange the Gram-Schmidt equations:

$$x_1 = z_1$$

$$x_2 = \frac{z'_1 x_2}{z'_1 z_1} z_1 + z_2$$

$$x_3 = \frac{z'_1 x_3}{z'_1 z_1} z_1 + \frac{z'_2 x_3}{z'_2 z_2} z_2 + z_3$$

Let $c_{ij} = \frac{z'_i x_j}{z'_i z_i}$ and let $Z = [z_1 \ z_2 \ z_3]$. In matrix form:

$$X = Z \begin{pmatrix} 1 & c_{12} & c_{13} \\ 0 & 1 & c_{23} \\ 0 & 0 & 1 \end{pmatrix}$$

$$X = Z\tilde{R}$$

where $\tilde{R}$ is an upper triangular matrix.

To make the columns of Z orthonormal, let $D = \text{diag}(\|z_1\|, \|z_2\|, \|z_3\|)$ and define $Q = ZD^{-1}$.

$$\implies Z = QD$$

Substitute Z into the matrix equation:

$$X = (QD)\tilde{R}$$

$$X = Q(D\tilde{R})$$

Let $R = D\tilde{R}$. The product of a diagonal matrix D and an upper triangular matrix $\tilde{R}$ remains upper triangular.

$$\therefore X = QR$$

11.3.3 (c)

(c) Find the least squares estimator $\hat{\beta}^{ls}$ and the predicted response $\hat{y}$ in terms of Q and R .
Given $X = QR$ and $Q'Q = I$ (since columns of Q are orthonormal):

$$X'X = (QR)'(QR) = R'Q'QR = R'IR = R'R$$

$$X'y = (QR)'y = R'Q'y$$

Substitute into $\hat{\beta}^{ls} = (X'X)^{-1}X'y$:

$$\hat{\beta}^{ls} = (R'R)^{-1}R'Q'y$$

$$= R^{-1}(R')^{-1}R'Q'y$$

$$= R^{-1}IQ'y$$

$$\therefore \hat{\beta}^{ls} = R^{-1}Q'y$$

For the predicted response $\hat{y} = X\hat{\beta}^{ls}$:

$$\hat{y} = (QR)(R^{-1}Q'y)$$

$$= Q(RR^{-1})Q'y$$

$$= QIQ'y$$

$$\therefore \hat{y} = QQ'y$$

11.4 Question 4

11.4.1 (a)

Ridge uses an L_2 penalty ($\lambda \sum \beta_j^2$). The penalty is smooth and differentiable. It shrinks coefficients continuously toward zero but never exactly to zero $\Rightarrow$ retains all predictors in the model.

Lasso uses an L_1 penalty ($\lambda \sum |\beta_j|$). The penalty is non-differentiable at zero (has a “corner”). This forces some coefficients to become exactly zero $\Rightarrow$ performs variable selection alongside shrinkage.

11.4.2 (b)

As regression shrinkage methods, what is the main difference between Lasso and Ridge regression? Elaborate.

Give in Lagrangian form, the expression of the Lasso estimator $\hat{\beta}^{lasso}$.

$$\hat{\beta}^{lasso} = \arg \min_{\beta} \left\{ (y - X\beta)'(y - X\beta) + \lambda \sum_{j=1}^p |\beta_j| \right\} = \arg \min_{\beta} \{ \|y - X\beta\|^2 + \lambda \|\beta\|_1 \}$$

11.4.3 (c)

An advantage is that automatic variable selection. By driving some coefficients exactly to zero, it produces a sparser, more interpretable model.

A disadvantage is that No closed-form solution. Because the L_1 penalty is not differentiable at zero, the estimates cannot be solved algebraically and require numerical optimisation methods.

Chapter 12

Extension III: Local Likelihood and Locally Linear Poisson Regression

```
library(knitr)
library(latex2exp)
```

This extension builds upon the local likelihood formulation introduced in the course notes and detailed in Section 6.5 of Hastie, Tibshirani and Friedman (2009, p. 205). The central premise is that any parametric model can be localized provided that the fitting algorithm accommodates observation weights, with maximum likelihood estimation forming an important class of such models. Writing $\ell(\beta \mid \mathbf{y}_i; x_i)$ for the log-likelihood contribution of observation i , the global log-likelihood in which a single parameter vector β serves every covariate value x is given by

$$\ell(\beta \mid \mathbf{y}; \mathbf{X}) = \sum_{i=1}^N \ell(\beta \mid \mathbf{y}_i; x_i),$$

while the log-likelihood local to a target point x_0 is defined as

$$\ell(\beta(x_0) \mid \mathbf{y}; \mathbf{X}) = \sum_{i=1}^N K_\lambda(x_0, x_i) \ell(\beta(x_0) \mid \mathbf{y}_i; x_i),$$

where $K_\lambda(x_0, x)$ represents a kernel function that decays with the distance between x and x_0 , and λ denotes its bandwidth. By allowing the parameters to vary as a function of the target point, a model that is linear globally is only required to satisfy linearity locally.

This extension extends this theoretical framework to a concrete regression problem with a non-Gaussian response by supplying a complete fitting algorithm, a likelihood-based cross-validation bandwidth selection rule, and a worked case study using call centre data. Following Hastie et al. (2009), we employ the tri-cube kernel $D(t) = (1 - |t|^3)^3$ for $|t| \leq 1$ and zero

otherwise, setting $K_\lambda(x_0, x) = D(|x - x_0|/\lambda)$ so that λ corresponds to the radius of the local window. The parametric model of choice is the Poisson log-linear model $\log \mu(x) = \beta_0 + \beta_1 x$. Its local counterpart is fitted using kernel weights within a weighted generalized linear model framework, and the bandwidth is selected via cross-validation across whole days.

12.0.1 Local Poisson Regression

For a count response, let $y_i \mid x_i \sim \text{Poisson}(\mu(x_i))$ with $\log \mu(x) = \eta(x)$. The global log-linear model specifies $\eta(x) = \beta_0 + \beta_1 x$. Its local counterpart takes the form $\eta(x) = \beta_0(x_0) + \beta_1(x_0)(x - x_0)$ for x near x_0 , yielding the local log-likelihood (up to a term free of β):

$$\ell(\beta(x_0) \mid \mathbf{y}; \mathbf{X}) = \sum_{i=1}^N K_\lambda(x_0, x_i) \{y_i \eta_i - e^{\eta_i}\}, \quad \eta_i = \beta_0(x_0) + \beta_1(x_0)(x_i - x_0).$$

Centring the covariate at x_0 ensures that $\beta_0(x_0)$ equals the fitted linear predictor at the target point, meaning the estimated mean function is given by $\hat{\mu}(x_0) = \exp\{\hat{\beta}_0(x_0)\}$, while the slope acts as a nuisance parameter capturing local trends. Differentiating with respect to the parameters yields the score equations:

$$\sum_{i=1}^N K_\lambda(x_0, x_i) (y_i - \hat{\mu}_i) \begin{pmatrix} 1 \\ x_i - x_0 \end{pmatrix} = 0,$$

which match ordinary Poisson score equations where kernel weights act as prior weights. Consequently, each local problem reduces to a weighted Poisson regression solved via iteratively reweighted least squares, allowing a weighted call to `glm.fit` to serve as the entire fitting engine. Furthermore, the intercept equation guarantees that kernel-weighted residuals sum to zero, ensuring local calibration within every neighbourhood—a property the global model possesses only across the sample globally.

The bandwidth λ controls the bias-variance trade-off. A small λ assigns appreciable weight to very few observations, allowing the estimate to track local features at the expense of higher variance, whereas a large λ flattens the weights across the sample, causing $\hat{\mu}(x_0)$ to converge toward the global log-linear fit. The global model thus represents the limiting member of a family indexed by λ .

12.0.1.1 Implementation

```
tricube <- function(u) ifelse(abs(u) < 1, (1 - abs(u)^3)^3, 0)

# Local linear Poisson likelihood at a single target point x0.
loclik_fit <- function(x0, x, y, lambda) {
  d <- x - x0
  w <- tricube(d / lambda)
```

```

k <- w > 0
glm.fit(cbind(1, d[k]), y[k], weights = w[k],
        family = poisson())$coefficients
}

# refitting once per distinct value of xnew
loclik_predict <- function(xnew, x, y, lambda) {
  ug <- unique(xnew)
  beta <- sapply(ug, loclik_fit, x = x, y = y, lambda = lambda)
  exp(beta[1, match(xnew, ug)])
}

pois_dev <- function(y, mu) 2 * sum(ifelse(y > 0, y*log(y/mu), 0) - (y-mu))

```

12.0.2 Case Study: Call Arrivals at a Bank Call Centre

The dataset records call volumes received by a major North American commercial bank in five-minute intervals between 7:00 and 21:00 on weekdays from 3 March to 24 October 2003, comprising 164 days of 169 intervals each. Available as `bank_calls` in the `fpp3` package, the response variable represents arrival counts in a fixed window, making the Poisson model a natural baseline. The single covariate is the time of day expressed in hours, focusing the analysis on the arrival rate profile.

Pooling all weekdays simplifies the problem to a single covariate. Day-of-week effects and daily baseline variations are intentionally set aside, with implications for dispersion addressed below. Data splitting occurs at the day level rather than the row level (70% training, 30% testing) to prevent information leakage across correlated intraday intervals.

```

data("bank_calls", package = "fpp3")
calls <- as.data.frame(bank_calls)

calls$day <- format(calls$DateTime, "%Y-%m-%d", tz = "UTC")
calls$hr <- as.numeric(format(calls$DateTime, "%H", tz = "UTC")) +
  as.numeric(format(calls$DateTime, "%M", tz = "UTC")) / 60

set.seed(1409)
days <- sort(unique(calls$day))
train_day <- sample(days, round(0.7 * length(days)))
train <- calls[calls$day %in% train_day, ]
test <- calls[!(calls$day %in% train_day), ]

# Global parametric baselines
fit_lin <- glm(Calls ~ hr, data = train, family = poisson)
fit_quad <- glm(Calls ~ hr + I(hr^2), data = train, family = poisson)

```

```

prof <- tapply(train$Calls, train$hr, mean)
tt    <- as.numeric(names(prof))

plot(NA, xlim = range(tt), ylim = range(train$Calls),
     xlab = "Time of day (hours)", ylab = "Calls per 5 minutes")
for (d in sample(train_day, 25)) {
  s <- train[train$day == d, ]
  lines(s$hr, s$Calls, col = adjustcolor("grey50", 0.4))
}
lines(tt, prof, lwd = 2)
lines(tt, predict(fit_lin, data.frame(hr = tt), type = "response"),
      col = "red", lwd = 2)
lines(tt, predict(fit_quad, data.frame(hr = tt), type = "response"),
      col = "forestgreen", lwd = 2)
legend("topright", c("Training mean", "Global log-linear",
                    "Global log-quadratic"),
      col = c("black", "red", "forestgreen"), lwd = 2, bty = "n")

```

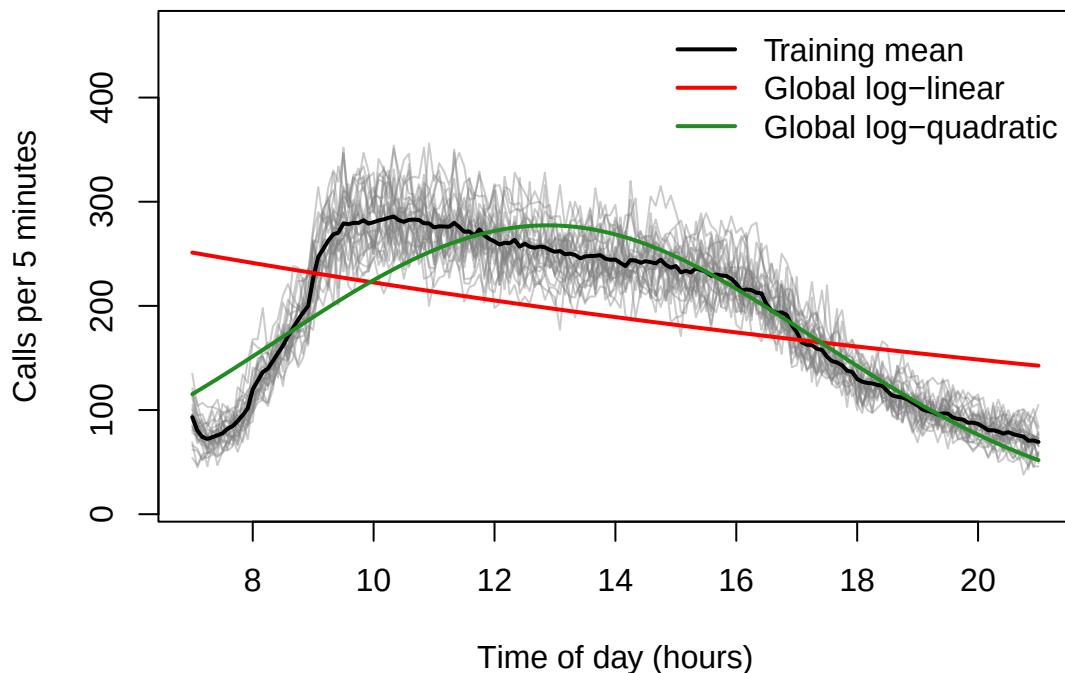


Figure 12.1: Calls per five minute interval against time of day. Grey curves are 25 randomly chosen training days, the black curve is the mean across all training days, and the coloured curves are the two global Poisson regressions.

The empirical mean profile exhibits a steep rise between 8:00 and 9:30, plateaus near 10:30, declines gradually through the afternoon with a sharper drop between 16:00 and 17:30, and displays a minor opening-interval spike. The global log-linear model imposes a downward trend and fails entirely to capture this non-linear shape. While the global log-quadratic model does a reasonable job capturing the overall macro-shape, particularly between hours 16 and 20, it fails to properly represent the initial rapid morning rise and the exact inflection point where the afternoon decline begins. Furthermore, because it is symmetric on the log scale, it cannot reconcile the asymmetry between the steep morning ascent and the gentler afternoon descent.

```
par(mfrow = c(1, 2))
for (x0 in c(8.75, 13)) {
  lam <- 1
  beta <- loclik_fit(x0, train$hr, train$Calls, lam)
  w <- tricube((tt - x0) / lam)
  win <- w > 0
  plot(tt[win], log(prof[win]), pch = 20, cex = 0.3 + 2 * w[win],
       xlab = "Time of day (hours)", ylab = "log(mean calls)",
       main = TeX(paste0("Target  $x_0$  = ", x0)))
  lines(tt[win], beta[1] + beta[2] * (tt[win] - x0), lwd = 2, col = "blue")
  points(x0, beta[1], pch = 19, col = "red", cex = 1.5)
}
```

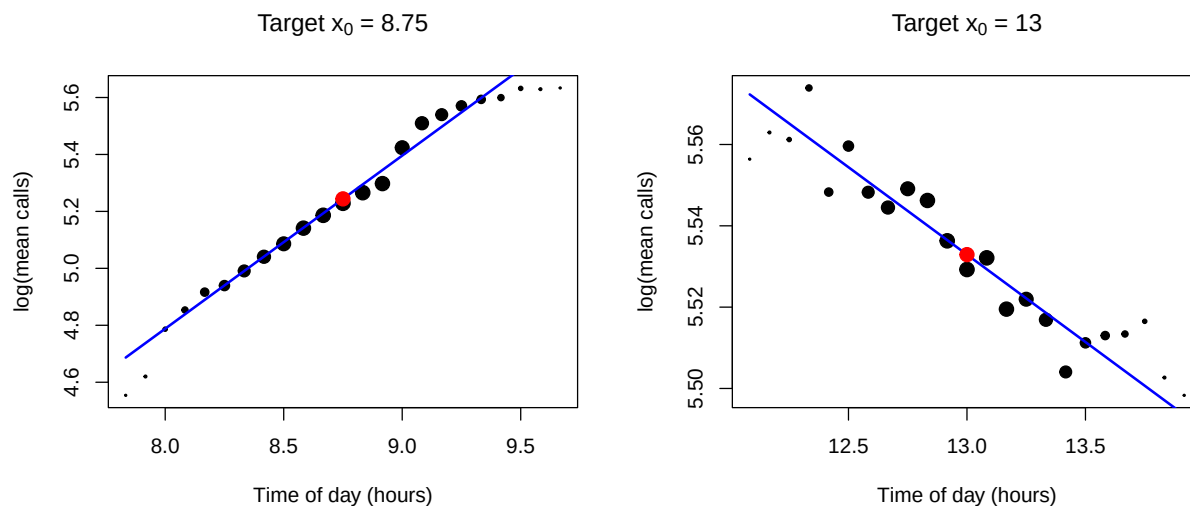


Figure 12.2: Local linear Poisson fits at two target points with $\lambda = 1$ hour. Point size is proportional to the tri-cube weight, points are training-mean call counts shown on the log scale, the line is the local linear predictor, and the red dot is the fitted value at the target.

```
par(mfrow = c(1, 1))
```

This isolated local regression for target points x_0 provides a clear illustration of the mechanism.

At a target of 8:45, the local window falls along the steep morning ramp, yielding a large, positive local slope. Conversely, at 13:00, the window rests on the afternoon decline, resulting in a negative local slope. A global log-linear model enforces a single compromise slope across the entire day, misrepresenting local dynamics at both points. The local model requires only approximate linearity on the log scale within the active window, extracting the fitted value directly from the local intercept.

```
K <- 5
lambdas <- c(0.25, 0.5, 0.75, 1, 1.5, 2, 3, 4)

# Folds are formed from whole days
fold_of_day <- sample(rep(1:K, length.out = length(train_day)))
train$fold <- fold_of_day[match(train$day, train_day)]

loclik_cv <- function(dat, lambdas, K) {
  sapply(lambdas, function(l) {
    dev <- 0
    for (k in 1:K) {
      tr <- dat[dat$fold != k, ]
      va <- dat[dat$fold == k, ]
      dev <- dev + pois_dev(va$Calls, loclik_predict(va$hr, tr$hr,
                                                    tr$Calls, l))
    }
    dev / nrow(dat)
  })
}

cv_dev <- loclik_cv(train, lambdas, K)
lambda_hat <- lambdas[which.min(cv_dev)]

par(mfrow = c(1, 2))
plot(lambdas, cv_dev, type = "b", pch = 20, log = "x", xaxt = "n",
     xlab = TeX("Bandwidth $\\lambda$ (hours, log scale)"),
     ylab = "CV deviance per observation")
axis(1, at = lambdas, labels = lambdas)
abline(v = lambda_hat, lty = 2, col = "blue")

plot(tt, prof, pch = 20, cex = 0.5, xlab = "Time of day (hours)",
     ylab = "Calls per 5 minutes")
cols <- c("blue", "darkorange", "red")
for (i in seq_along(cols)) {
  l <- c(lambda_hat, 1, 4)[i]
  lines(tt, loclik_predict(tt, train$hr, train$Calls, l), col = cols[i],
        lwd = 2)
}
```

```
legend("topright", paste0(TeX("\\lambda$ = "), c(lambda_hat, 1, 4)),
      col = cols, lwd = 2, bty = "n")
```

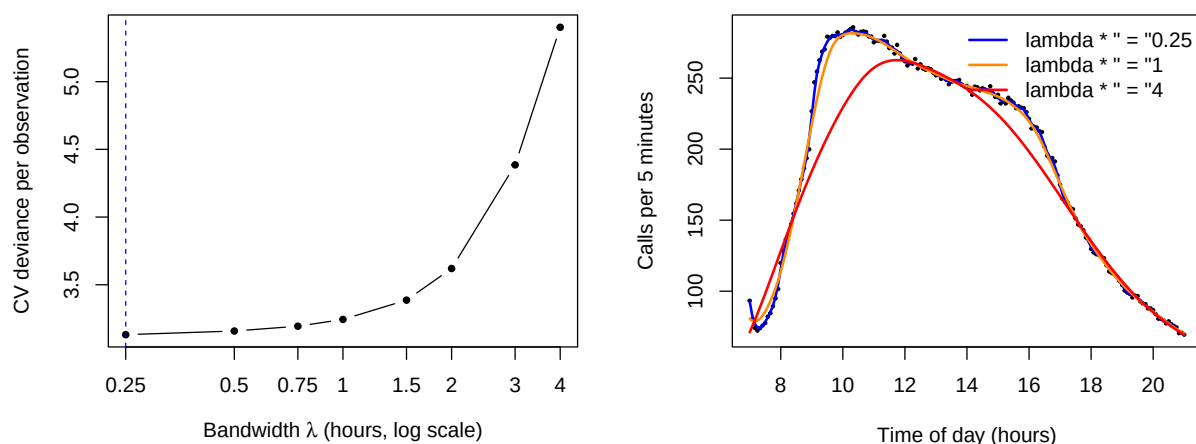


Figure 12.3: Left: five-fold cross-validated Poisson deviance per observation against bandwidth, with folds formed from whole days. Right: local linear fits at three bandwidths against the training mean profile.

```
par(mfrow = c(1, 1))
```

The cross-validated deviance increases monotonically over the tested grid, selecting the minimum evaluated bandwidth ($\lambda = 0.25$ hours). Due to high computational costs associated with the local likelihood setup, a finer grid was omitted, but this boundary selection highlights key properties of the design. The bandwidth $\lambda = 4$ represents the point where we see the CV error growing rapidly relative to the lower bandwidths; even so, $\lambda = 4$ gives a much better representation than our quadratic likelihood, which is remarkable, though it still misses the rapid rise and decline just before midday. Meanwhile, $\lambda = 1$ fits really well to the data, capturing every local kink. However, the minimum bandwidth ($\lambda = 0.25$) seems to pretty much interpolate every point and thus could be prone to overfitting.

12.0.3 References

Hastie, T., Tibshirani, R. and Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd edition. Springer.

Chapter 13

Extension IV: Ill-Posed Inverse Problem in Scalar-on-Function Regression

```
library(fda.usc)
library(knitr)
library(pracma)
library(latex2exp)
```

This extension explores the functional linear model $Y = \alpha + \langle X, \beta \rangle + \epsilon$, framing the estimation of the slope function β as an ill-posed inverse problem. This problem, along with the development of spectral truncation and Tikhonov regularisation estimators, follows the foundational framework established by Hall and Horowitz (2007). The overarching goal of that literature is to establish asymptotic guarantees for functional regression in infinite-dimensional Hilbert spaces. Rather than relying on heuristic tuning, Hall and Horowitz derive optimal nonparametric convergence rates and analyze the asymptotic behavior of regularised estimators under conditions of polynomial eigenvalue decay. The nature of their proofs centers on bounding the stochastic errors of empirical covariance operators and evaluating the mathematical trade-off between approximation bias and variance inflation as sample sizes grow.

To formalise this setting, The population normal equation for the β minimising $E(Y - \alpha - \langle X, \beta \rangle)^2$ is:

$$\Gamma\beta = r, \quad (\Gamma f)(s) = \int_T C(s, t)f(t) dt, \quad r(t) = E[(Y - \mu_Y)(X(t) - \mu(t))]$$

where Γ is the covariance operator of X (Hall and Horowitz, 2007, Sec.2). Formally $\hat{\beta} = \Gamma^{-1}r$. However, Γ is a compact operator, meaning it smooths and compresses high-frequency details down to zero ($\lambda_k \rightarrow 0$). Because reversing this compression requires an infinite stretch factor,

its inverse Γ^{-1} is unbounded, multiplying the k -th eigencomponent by $1/\lambda_k \rightarrow \infty$.

Despite the heavily theoretical lens of the literature, the primary aim of this extension is to implement the mathematical machinery largely from first principles for a relevant problem. Implementations in this extension, however, numerically approximate continuous integrals using `pracma::trapz`.

13.0.1 Regularisation Strategies

13.0.1.1 1. Spectral Truncation (FPCR)

Functional Principal Component Regression (FPCR) projects the data onto the first K eigenfunctions of the empirical covariance operator. Following Hall and Horowitz (2007, eq. 2.3 and 2.4), the resulting well-posed $K \times K$ problem is solved by:

$$\hat{\beta}_K(u) = \sum_{j=1}^K \hat{b}_j \hat{\phi}_j(u), \quad \hat{b}_j = \frac{\hat{g}_j}{\hat{\lambda}_j}, \quad \hat{g}_j = \int \hat{g}(u) \hat{\phi}_j(u) du$$

where $\hat{g}(u) = \frac{1}{n} \sum_{i=1}^n (Y_i - \bar{Y})(X_i(u) - \bar{X}(u))$.

In these formulas, $\hat{\phi}_j(u)$ is the j -th empirical eigenfunction, $\hat{\lambda}_j$ is the corresponding empirical eigenvalue, and $\hat{g}(u)$ is the sample cross-covariance function between X and Y . By terminating the sum exactly at K , this method applies a cutoff to the spectral expansion. It fully retains the first K principal components but discards the tail components whose sufficiently small eigenvalues would otherwise blow up the variance.

Computationally, $\hat{b}_j$ is exactly the OLS estimate of Y regressed onto the j -th functional principal component score.

```
# Trapezoidal quadrature weights
quad_w <- function(tt) {
  d <- diff(tt)
  c(d[1] / 2, (d[-1] + d[-length(d)]) / 2, d[length(d)] / 2)
}

fpcr_prep <- function(X, tt) {
  w <- quad_w(tt)
  xbar <- colMeans(X)
  Xc <- sweep(X, 2, xbar)
  sw <- sqrt(w)
  # Symmetrised operator: diag(sqrt(w)) C diag(sqrt(w)).
  G <- crossprod(Xc * rep(sw, each = nrow(Xc))) / nrow(X)
  ev <- eigen(G, symmetric = TRUE)
  list(phi = ev$vectors / sw, lam = ev$values, xbar = xbar, w = w, tt = tt)
}
```

```

fpcr_scores <- function(pp, X, K) {
  sweep(X, 2, pp$xbar) %*% (pp$phi[, 1:K, drop = FALSE] * pp$w)
}

fpcr_fit <- function(pp, X, y, K) {
  scores <- fpcr_scores(pp, X, K)

  # OLS
  Z <- cbind(1, scores)
  co <- solve(t(Z) %*% Z) %*% t(Z) %*% y

  alpha <- co[1]
  coef <- co[-1]

  # beta(t) = sum(b_k * phi_k(t))
  phi_K <- pp$phi[, 1:K, drop = FALSE]
  beta <- phi_K %*% coef

  list(alpha = alpha, coef = coef, beta = drop(beta), pp = pp, K = K)
}

fpcr_predict <- function(fit, Xnew) {
  scores <- fpcr_scores(fit$pp, Xnew, fit$K)
  y_hat <- fit$alpha + (scores %*% fit$coef)

  return(drop(y_hat))
}

fpcr_cv <- function(pp, X, y, Kmax) {
  n <- length(y)
  Zall <- cbind(1, fpcr_scores(pp, X, Kmax))

  cv_errors <- numeric(Kmax)
  cv_ses <- numeric(Kmax)

  for (K in 1:Kmax) {
    Z_K <- Zall[, 1:(K + 1), drop = FALSE]

    # Use QR decomp for numerical stability of the hat matrix
    q <- qr(Z_K)
    Q <- qr.Q(q)

    # Fitted values and h_ii

```

```

y_hat <- drop(Q %*% (t(Q) %*% y))
h_ii <- rowSums(Q^2)

# Leave-one-out residuals shortcut formula: (y - y_hat) / (1 - h_ii)
# no ways am i training and fitting N times
loo_res <- (y - y_hat) / (1 - h_ii)
e2 <- loo_res^2

cv_errors[K] <- mean(e2)
cv_ses[K] <- sd(e2) / sqrt(n)
}

data.frame(cv = cv_errors, se = cv_ses)
}

```

13.0.1.2 2. Tikhonov (Functional Ridge) Regularisation

Instead of a hard cutoff, the functional ridge estimator adds a penalty λI before inverting. Following Hall and Horowitz (2007, eq. 2.5), the estimator is defined as:

$$\hat{\beta}_{\lambda} = (\hat{\Gamma}_n + \lambda I)^{-1} \hat{r}$$

In the eigenbasis, this smoothly shrinks the k -th component by the factor $\frac{\hat{\mu}_k}{\hat{\mu}_k + \lambda}$ rather than discarding it entirely.

```

# any eigenvalue that has collapsed to numerical zero is dropped.
tik_Kmax <- function(pp, n) min(n - 1, sum(pp$lam > 1e-10 * pp$lam[1]))

tik_fit <- function(pp, X, y, lambda) {
  n <- nrow(X)
  Kmax <- tik_Kmax(pp, n)

  # Scores xi_ik = <X_i - Xbar, phi_k>_w
  xi <- fpcr_scores(pp, X, Kmax)
  mu <- pp$lam[1:Kmax]

  # b_k = xi_k' (y - ybar) / (n (mu_k + lambda))
  yc <- y - mean(y)
  coef <- drop(crossprod(xi, yc)) / n / (mu + lambda)

  alpha <- mean(y) - sum(colMeans(xi) * coef)
}

```

```

# beta(t) = sum_k coef_k * phi_k(t)
phi_K <- pp$phi[, 1:Kmax, drop = FALSE]
beta  <- phi_K %*% coef

list(alpha = alpha, coef = coef, beta = drop(beta), pp = pp, K = Kmax,
      mu = mu, shrink = mu / (mu + lambda))
}

tik_cv <- function(pp, X, y, lambdas) {
  n      <- nrow(X)
  Kmax <- tik_Kmax(pp, n)

  xi <- fpcr_scores(pp, X, Kmax)
  mu <- pp$lam[1:Kmax]
  yc <- y - mean(y)

  cv_errors <- numeric(length(lambdas))
  cv_ses     <- numeric(length(lambdas))

  for (i in seq_along(lambdas)) {
    lambda <- lambdas[i]

    H <- 1 / n + xi %*% ((1 / (n * (mu + lambda))) * t(xi))

    y_hat <- drop(H %*% yc)
    h_ii  <- diag(H)
    loo_res <- (yc - y_hat) / (1 - h_ii)
    e2      <- loo_res^2

    cv_errors[i] <- mean(e2)
    cv_ses[i]    <- sd(e2) / sqrt(n)
  }

  data.frame(cv = cv_errors, se = cv_ses)
}

```

13.0.2 Case Study: Tecator Benchmark

The empirical evaluation of these regularisation methods utilises the Tecator dataset, which consists of 215 continuous near-infrared absorbance spectra measured across 100 equally spaced wavelengths to predict the fat content of meat samples. Because adjacent spectral wavelengths exhibit extreme collinearity, attempting to regress the scalar response directly onto the discretised curves without regularisation triggers the catastrophic variance amplification inherent to compact operators.

```

data(tecator)
tt  <- tecator$absorp.fdata$argvals
Xall <- tecator$absorp.fdata$data
yall <- tecator$y$Fat

plot(tt, Xall[1, ], type = "l", col = adjustcolor("steelblue", 0.5),
      ylim = range(Xall), xlab = "Wavelength (nm)", ylab = "Absorbance",
      main = "Tecator NIR Absorbance Spectra")
for (i in 2:nrow(Xall)) {
  lines(tt, Xall[i, ], col = adjustcolor("steelblue", 0.5))
}

```

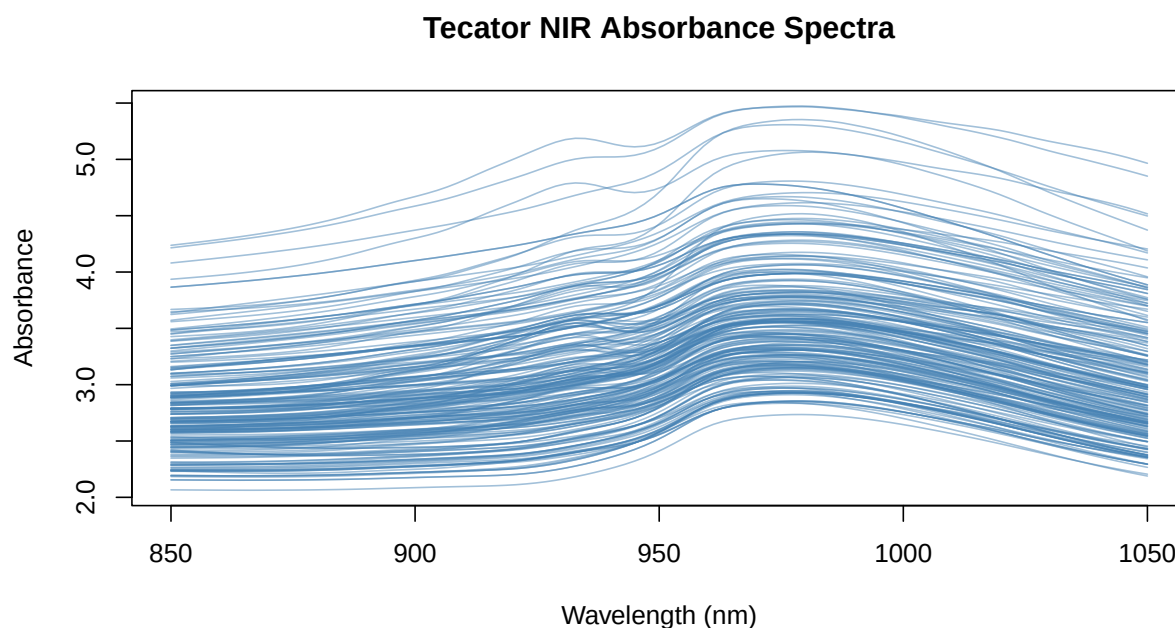


Figure 13.1: Observed Tecator near-infrared absorbance spectra. The curves are smooth, densely sampled, and highly collinear.

To demonstrate the mechanical benefit of regularisation, the data is partitioned into a 70/30 training and testing split. The analysis benchmarks the unregularised naive operator inversion against Functional Principal Component Regression (FPCR) and Tikhonov regularisation. Two FPCR fits are reported, one using the cross-validated minimum and one using the one-standard-error rule, because the choice of truncation rule turns out to matter as much as the choice of estimator. By either truncating or smoothly shrinking the smallest eigenvalues of the empirical covariance operator, regularisation prevents the unbounded inverse operator from multiplying microscopic sample noise by massive reciprocal eigenvalues.

```

n_tec <- nrow(Xall)
set.seed(1409)
train_idx <- sample(n_tec, round(0.7 * n_tec))
Xtr <- Xall[train_idx, ]; ytr <- yall[train_idx]
Xte <- Xall[-train_idx, ]; yte <- yall[-train_idx]

# lse the minimum
pick_lse <- function(cvdf) {
  m <- which.min(cvdf$cv)
  ok <- which(cvdf$cv <= cvdf$cv[m] + cvdf$se[m])
  min(ok)
}

# Prepare Data & Fit Unregularised Baseline
pp_tr <- fpcr_prep(Xtr, tt)
fit_naive <- fpcr_fit(pp_tr, Xtr, ytr, 40)

# FPCR
cv_K <- fpcr_cv(pp_tr, Xtr, ytr, 30)
K_min <- which.min(cv_K$cv)
K_lse <- pick_lse(cv_K)
fit_fpcr_min <- fpcr_fit(pp_tr, Xtr, ytr, K_min)
fit_fpcr_lse <- fpcr_fit(pp_tr, Xtr, ytr, K_lse)

# Tikhonov
lambdas <- 10^seq(-6, 0, by = 0.25)
cv_tik <- tik_cv(pp_tr, Xtr, ytr, lambdas)
fit_tik <- tik_fit(pp_tr, Xtr, ytr, lambdas[which.min(cv_tik$cv)])

preds <- list(
  "Unregularised (K=40)" = fpcr_predict(fit_naive, Xte),
  "FPCR (min)" = fpcr_predict(fit_fpcr_min, Xte),
  "FPCR (lse)" = fpcr_predict(fit_fpcr_lse, Xte),
  "Tikhonov" = fpcr_predict(fit_tik, Xte)
)

rmse <- sapply(preds, function(p) sqrt(mean((p - yte)^2)))
r2 <- sapply(preds,
  function(p) 1 - sum((p - yte)^2) / sum((yte - mean(yte))^2))

kable(data.frame(Model = names(preds),

```

```
RMSE = round(rmse, 3),
R2    = round(r2, 3))
```

Table 13.1: Predictive performance of unregularised and regularised estimators on the Tecator test set.

	Model	RMSE	R2
Unregularised (K=40)	Unregularised (K=40)	3.514	0.906
FPCR (min)	FPCR (min)	2.848	0.938
FPCR (1se)	FPCR (1se)	3.162	0.924
Tikhonov	Tikhonov	2.645	0.947

```
par(mfrow = c(2, 2))
yl_global <- range(c(fit_naive$beta, fit_fpcr_min$beta, fit_fpcr_1se$beta,
                    fit_tik$beta))

plot(tt, fit_naive$beta, type = "l", col = "red", ylim = yl_global,
     xlab = "Wavelength (nm)", ylab = expression(hat(beta)(t)),
     main = "Unregularised (K=40)")
abline(h = 0, lty = 3)

plot(tt, fit_fpcr_min$beta, type = "l", col = "forestgreen",
     ylim = yl_global, xlab = "Wavelength (nm)",
     ylab = expression(hat(beta)(t)),
     main = paste0("FPCR, min rule (K = ", K_min, ")"))
abline(h = 0, lty = 3)

plot(tt, fit_fpcr_1se$beta, type = "l", col = "darkorange",
     ylim = yl_global, xlab = "Wavelength (nm)",
     ylab = expression(hat(beta)(t)),
     main = paste0("FPCR, 1se rule (K = ", K_1se, ")"))
abline(h = 0, lty = 3)

plot(tt, fit_tik$beta, type = "l", col = "blue", ylim = yl_global,
     xlab = "Wavelength (nm)", ylab = expression(hat(beta)(t)),
     main = "Tikhonov")
abline(h = 0, lty = 3)
```

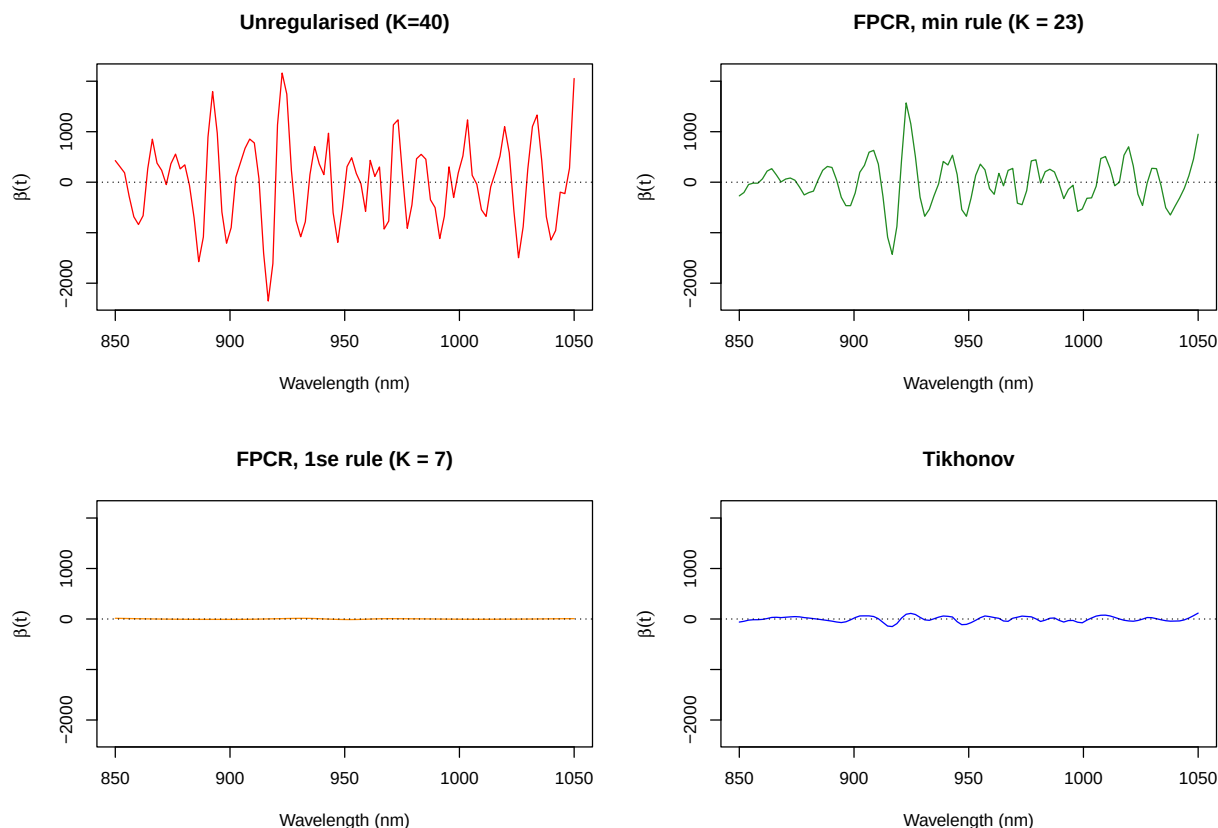


Figure 13.2: Estimated coefficient functions on a unified scale.

```
par(mfrow = c(1, 1))
```

The plots above make two features of truncation visible that the performance metrics alone do not. First, FPCR at the cross-validated minimum retains enough tail components that the reciprocal eigenvalues dominate the coefficient function, and the fitted curve is inflated to the same order of magnitude as the unregularised fit even though its out-of-sample predictions are stable. Second, the one-standard-error rule collapses the same estimator back to a curve of ordinary size, but only by discarding a noticeable block of components whose contribution to prediction was real. Truncation therefore forces an all-or-nothing decision on the tail: keep enough components to predict well and the coefficient function becomes uninterpretable, or shrink the coefficient function to an interpretable size and give up some predictive accuracy.

The two regularised fits are compared directly on their own scale in the next figure, with the unregularised fit omitted because its amplitude would compress everything else to a flat line.

```
plot(tt, fit_fpcr_1se$beta, type = "l", col = "darkorange", lwd = 2,
     ylim = range(c(fit_fpcr_1se$beta, fit_tik$beta)),
     xlab = "Wavelength (nm)", ylab = expression(hat(beta)(t)))
lines(tt, fit_tik$beta, col = "blue", lwd = 2)
abline(h = 0, lty = 3)
```

```
legend("topleft", c(paste0("FPCR, K = ", K_1se), "Tikhonov"),  
      col = c("darkorange", "blue"), lwd = 2, bty = "n")
```

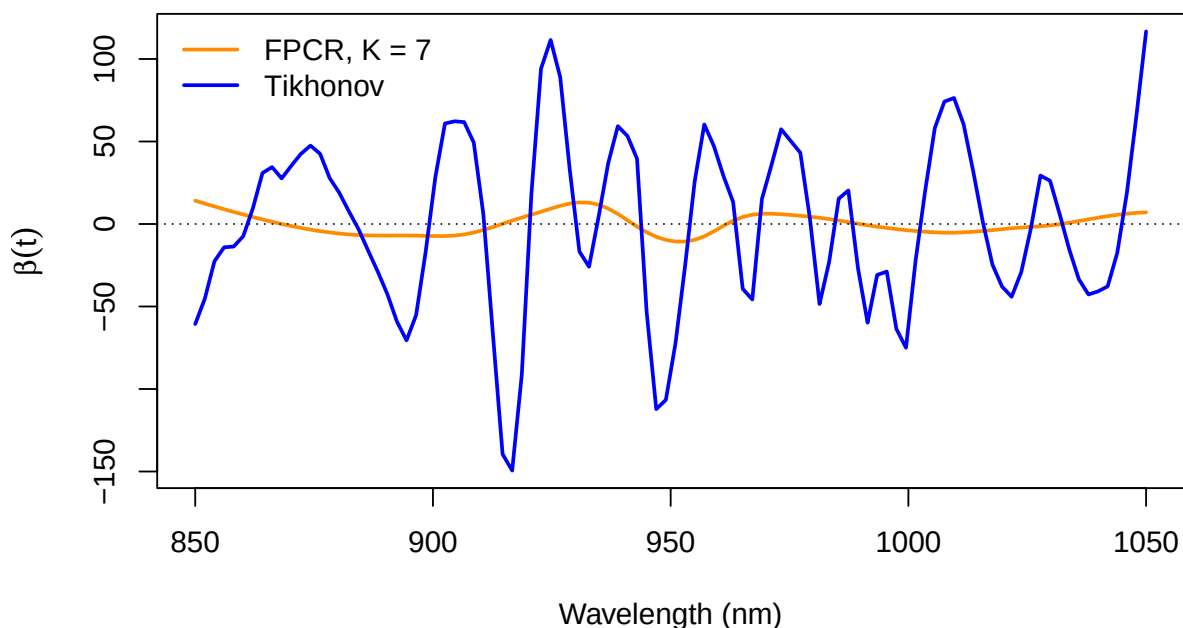


Figure 13.3: FPCR (1se) and Tikhonov on their own scale.

Comparing the two regularised fits on their own scale reveals a stark contrast in smoothness and structural interpretation. FPCR at the one-standard-error rule is remarkably smooth, driven by its strict confinement to the leading, low-frequency eigenfunctions. Interestingly, despite being drastically scaled down, it traces a nearly identical structural path to the inflated minimum-rule fit: both exhibit a distinct negative dip near 920 nm followed by a sharp terminal rise. This confirms that the core chemical signature is encoded in the leading components, while the discarded tail merely acts as an inflationary multiplier.

Tikhonov, by contrast, operates on a noticeably larger amplitude at this scale and exhibits high-frequency oscillation across the entire domain. Because it never completely zeroes out any eigendirection, the heavily down-weighted tail components manifest as a jagged ripple riding over the baseline function. This flips the conventional narrative on regularisation tradeoffs. While Tikhonov achieves the superior predictive accuracy with the lowest RMSE and highest R^2 , its coefficient function sacrifices chemical interpretability to high-frequency noise. FPCR at the one-standard-error rule willingly sacrifices a fraction of predictive power to deliver a smooth, readable coefficient function.

```

betas_abs <- list(
  "Unregularised" = abs(fit_naive$beta),
  "FPCR (min rule)" = abs(fit_fpcr_min$beta),
  "FPCR (1-SE rule)" = abs(fit_fpcr_1se$beta),
  "Tikhonov" = abs(fit_tik$beta)
)

five_num_stats <- data.frame(t(sapply(betas_abs, fivenum)))
colnames(five_num_stats) <- c("Min", "Q1", "Median", "Q3", "Max")

kable(five_num_stats, digits = 3)

```

Table 13.2: 5-Number Summary of Absolute Deviations from Zero

	Min	Q1	Median	Q3	Max
Unregularised	31.436	292.240	561.720	943.633	2353.137
FPCR (min rule)	0.540	139.588	274.187	462.313	1570.369
FPCR (1-SE rule)	0.011	2.646	5.056	7.024	14.104
Tikhonov	1.230	21.248	36.381	59.197	149.425

The summary statistics reinforce the visual impression of magnitude while highlighting the disconnect between amplitude and smoothness. The unregularised fit and the FPCR minimum-rule fit produce coefficient functions stretching into the thousands. The one-standard-error rule successfully collapses the FPCR median absolute deviation by two orders of magnitude. Tikhonov sits between these truncation extremes, making it larger in amplitude than the 1-SE FPCR fit but significantly smaller than the unregularised baseline. However, as the visual comparison proves, Tikhonov's intermediate magnitude does not translate to a smoother curve, underscoring that magnitude alone is an insufficient proxy for interpretability.

```

plot(fit_tik$mu, fit_tik$shrink, log = "x", type = "b", pch = 20,
     col = "blue", xlab = "Eigenvalue (log scale)",
     ylab = "Shrinkage Factor")

abline(v = fit_tik$mu[K_min], lty = 2, col = "forestgreen", lwd = 2)
abline(v = fit_tik$mu[K_1se], lty = 2, col = "darkorange", lwd = 2)

legend("bottomright",
      c("Tikhonov Shrinkage",
        paste0("FPCR min cutoff (K = ", K_min, ")"),
        paste0("FPCR 1se cutoff (K = ", K_1se, ")")),
      col = c("blue", "forestgreen", "darkorange"),
      lty = c(1, 2, 2), pch = c(20, NA, NA), bty = "n")

```

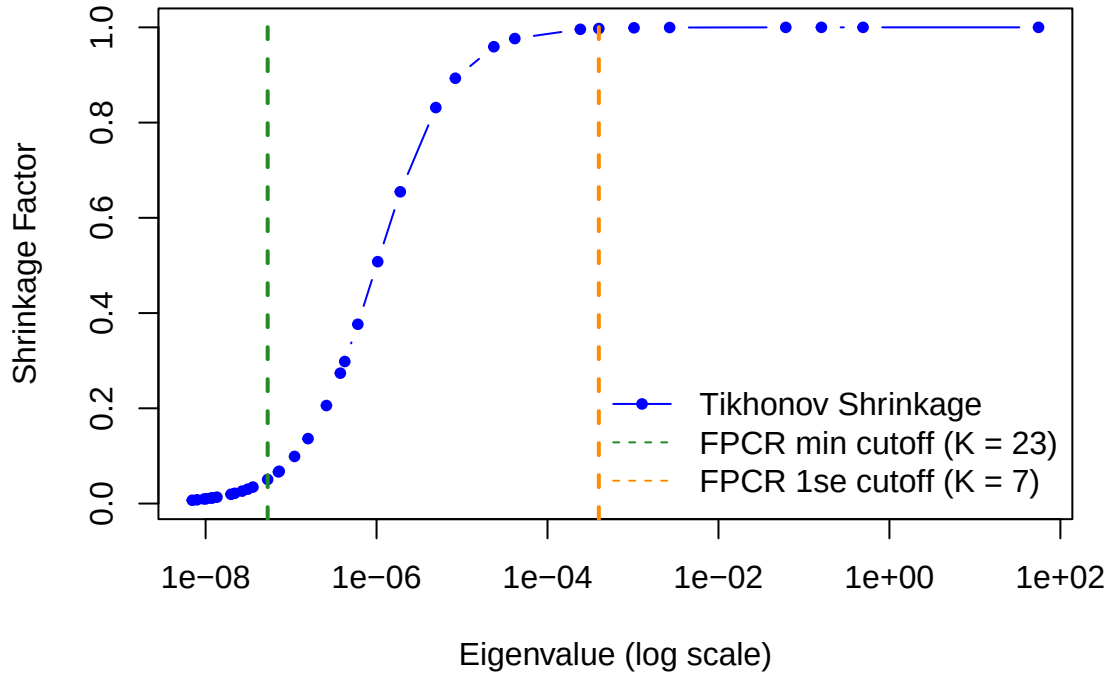


Figure 13.4: Tikhonov smoothly shrinks small eigenvalues toward zero, whereas FPCR applies a hard boundary. The two vertical lines mark the cutoffs chosen by the cross-validated minimum (green) and by the one-standard-error rule (orange); every component to the left of a line is discarded entirely by the corresponding FPCR fit.

The shrinkage curve plots the Tikhonov penalty weight, $\hat{\mu}_k/(\hat{\mu}_k + \lambda)$, applied to each component as a function of its eigenvalue. This factor dictates how much of a component's OLS coefficient is retained: large, stable eigenvalues on the right receive a weight near one, while noisy, near-zero eigenvalues on the left are smoothly shrunk toward zero. To get a good sense of comparison between what these two regularisation methods are doing, we can utilise this plot by mapping the FPCR cutoffs onto this space. This gives a reasonable explanation to the divergence in smoothness and predictive power: The one-standard-error rule places its hard boundary to the right, truncating the spectrum just before the shrinkage factor begins its rapid decline. By discarding everything to the left of this line, it acts as a strict filter that retains only the dominant signal to enforce a smooth coefficient function. Conversely, the minimum rule sets its boundary far to the left, deep into the region where the shrinkage factor has already tapered off near zero. Because FPCR retains everything to the right of its cutoff with full reciprocal weights ($1/\hat{\mu}_k$), the minimum rule allows this noisy tail to completely inflate the estimated function.

Ultimately, the choice between these estimators depends on the analytical priority: spectral

truncation seemingly tends to yield a structurally interpretable, smooth parameter, while Tikhonov tolerates a jagged coefficient function to maximize out-of-sample accuracy.

13.0.3 References

Hall, P. and Horowitz, J. L. (2007). Methodology and convergence rates for functional linear regression. *The Annals of Statistics*, 35(1), 70-91.

Part III

Supporting Documentation

AI Declaration

The use of AI in this portfolio was limited to the following:

- Code and LaTeX debugging, particularly for unnecessarily tedious problems such as plotting issues and random syntax errors.
- Helping to explore for verifiable sources and for different types of datasets.
- Converting handwritten solutions to LaTeX from scans, and similarly to save time in writing out the questions for the final setup.
- Serving as an engagement tool to clarify my own understanding and misconceptions, and to question various avenues, particularly ones with no clear answer ahead.

Main AI chatlogs can be found here: <https://gemini.google.com/app/be6ba84775cbe4e4>

Reflective Commentary

Learning achievements

I found the whole endeavour enjoyable. It was incredibly tedious at times, but it is the kind of tedium that feels rewarding once the work comes together. In a recent conversation with Dr Birgit, I said that creating these comprehensive bodies of work is, in a sense, an artwork, and I stand by that description. I spent a large amount of time working out how to compile the document as beautifully as possible. For example, anyone scrolling through the document will notice that I manually indented the code so that everything stays contained within the code margins, and that all figures and tables within sections are labelled using `#| fig-cap` and `#| tbl-cap`. My intention was to make the portfolio read much like a dissertation, with the same attention to detail, and to make each section self-contained, so that someone could read it, understand what is going on and learn from it in that way. Framing the work like this throughout helped me to keep a holistic view of what I wanted the project to look like. I knew from the start that I did not want to bundle together a collection of scattered pieces of work and stack them on top of each other without an overarching narrative.

Beyond the development of my own mental schema, getting immersed in the work gives a deeper understanding of, and feel for, the content. Ever since my honours year I have had the impression that everything is structured so that you get out as much as you are willing to pour in, and this portfolio has been no exception. As the work progresses you start to build a sense of confidence about how well you really know what is going on, and about what you are actually capable of. Despite how corny it may sound, at this level our aim is to show that we have mastered the subject of interest and can deploy what we know in applicable scenarios and areas of interest. The portfolio gave me a concrete way of testing that for myself.

Methodological challenges

This approach was mostly a blessing, but it did come with some curses. It is easy to get lost in random details and in the search for answers to largely irrelevant questions. This is good for understanding in my own mental framework, but it consumes a great deal of time. It also led me to set high expectations of how much I could get done in the time available. For example, my originally submitted draft of section A was a flow through two papers where $A \Rightarrow B$, effectively a mini-dissertation. I assumed that recreating simplified versions of these papers would not take very long, but as I progressed there were more and more additions to

include and nuances to explain and to understand fully myself. The draft ended up being far longer than intended, and the lesson I took from it was that expanding the scope is far easier than finishing it properly.

The most irritating part of the process was the bugs. I know this every time, yet I keep thinking it will get easier: debugging, and verifying that code works under the hood, takes up the majority of the time once a method is set up. If development is not properly staged, so that everything is confirmed to be correct at every step, it is easy to muddy the results with incorrect implementations whose effects only appear much later. This is why, for most of the tutorials, my main focus was to get the code right first and to worry about the cohesion of the project afterwards.

On top of this, tests and assignments from other courses put severe time constraints on what was actually plausible. Being honest with yourself about what is possible, and knowing when to discard certain avenues, is a valuable skill to retain, and it is one that I once again had to demonstrate here. After submitting the first draft, I realised that I could not pour that level of energy into the remaining projects, and I had to tone them down deliberately. Luckily, I am at a stage at masters level where I am comfortable with the nitty-gritty details alongside time pressure. At this point the work is mostly tedious at times rather than a source of pure confusion and stress, which is a considerable change from earlier years.

Ethical and reproducibility considerations

The elephant in the room is the remarkable capacity that AI tools have gained in such a short time. There is always a temptation to take as many shortcuts as possible, and such a tool can very easily be misused to the point where the work produced is far removed from the capabilities of the person submitting it. With this in mind, it is incredibly important that clear and honest transparency is presented, not only in an academic setting but also in the workforce. These issues have always existed. What has changed is that the avenues for acting on them are now so accessible, and produce results with little to no effort.

The concern is also not limited to AI use. It extends to the fabrication of results and plots, to the withholding of verifiable code so that results cannot be reproduced, and to work that has been plagiarised or restated without acknowledgement. For this reason I tried to do as much as I could on my own, and to rely on AI and other online sources for debugging help, as an engagement tool to verify my own understanding of the topics, and for finding important resources, all of which is documented in the AI chatlogs referred to in the declaration. The purpose of this is to ensure that I can wholeheartedly call whatever work I produce my own: that nothing in it is blindly generated, that it is a direct consequence of my active engagement, and that the narratives are always based on factual, verifiable and non-fabricated results.

Future areas for development

With respect to future development, I need to become comfortable with more technical but efficient ways of writing code. In many cases I have largely avoided them, even though

I am familiar with how they work, because the time and effort needed to get the hang of them first tends to be hard to commit to under deadline pressure. For example, the hyperparameter tuning for the local likelihood extension was restricted to an illustration rather than a full tuning exercise, because my code was largely sequential on a computation that is embarrassingly parallelisable, as my CSC2 lecturer used to say. A similar situation arose in the wavelets extension, where I restricted the simulation to 200 replications because of computational constraints, which could have been largely overcome with parallel computing. The same applies to the many opportunities I had to vectorise parts of my code. Making these techniques a routine part of my workflow, rather than something I postpone, is the most concrete next step I can identify.

Journal

Date	Progress
14-16 Aug	Officially started. At this stage I was getting familiar with the content and started with prac 1 on splines. Question 1 was what I really enjoyed to begin with. I felt it was one of those problems which, if done properly, leaves you fully understanding what splines are as a mathematical construction. I was not too focused on setting up the project template, I just wanted to start getting things done, and I had finished most of this first prac by the end of this period.
15 Aug	Started working on the first extension on wavelets. I found two interesting topics. I already knew about wavelet applications to finance, so I figured this would be a good one to put a lot of effort into something i'm actively being tested on in another course while applying it in something practical.
18-23 Aug	A confusing and disorganised period to be honest. I had a dilemma: I did not think I would have enough time to do all the tutorials and also complete a project extension. As a consequence, I decided to pour all my time into the wavelets extension and get that working. I figured that if I was going to submit something, I would rather get feedback on a project extension than on the tutorials, because the scope of the tutorials is already well defined and is mostly a matter of doing them correctly. As I mention in the reflective essay, I did get quite a bit of tunnel vision. I was reconstructing the ideas from two papers where $A \Rightarrow B$ and treated it as a mini-dissertation, except that I was obviously just recreating their work. It ended up being very long, with a couple of bug fixes still under the hood and a couple of sections incomplete. After submitting it I realised there was no way I could pour this much energy into the remaining three projects, and that I would have to tone them down.
2-5 Sept	I was absent from portfolio work for a week because of a massive test on 1 Sept. After resting for a bit, I slowly worked on completing the MDS prac and understanding some content for the future ones to come. I did not have much time to commit to the portfolio during this vacation period, because I had another assignment due the following week that was deliberately time consuming.

Date	Progress
9-16 Sept	The GAM, MARS and wavelets tutorials were done in this period. There is not much to note: my main focus throughout was to get the code right and to worry about the cohesion of the project later on. My overall sentiments on how the debugging and learning processes went are expressed in the reflection, and this period reflected them closely. I also started looking at the Shiny app extension around this time and got most of the code running as expected.
17-26 Sept	My main focus was writing up section B of the portfolio. This section was all tutorial questions, so the focus was on the maths and on converting it into LaTeX. I was also working on the two extensions. First I worked on the functional data analysis (FDA) one. I have a personal interest in functional analysis and wanted to do a project centred on material from an elective in the maths department in my honours year. However, I did not want to repeat the mistake I made in the first prac of focusing far too much on the underlying systems rather than on producing the relevant work, so I ended up working on a regularisation problem. I was swayed in this direction because from last year I knew that there were particular problems in this area that could be solved by one of the regularisation methods listed in the paper I eventually found and chose as my main source, although the link to the ill-posed inverse problem was never the original lens through which I thought about it. For the second, computational extension, I decided to dive deeper into one of the methods listed in the course that I was not particularly familiar with, and so I focused on developing the local likelihood architecture and illustrating a useful use case.
27 Sept-1 Oct	A very gruelling period. All the code cleaning and file compilation had me working on this project as if it were my job. I finished all the additional sections that I had left out and completed the mostly finished ones. Going through the projects from start to finish took a great deal of time: fixing my shorthand remarks and half sentences, adding proper structure to all of the qmds, and making sure that overall it was a coherent body of work. It was a love-hate relationship with the work. Honestly on multiple occasion I was glued to my laptop for 4+hours straight without food or water hyperfixiating on this because I had set myself the goal of finishing a certain section before doing anything else.
